

MCS-208 Data Structures and Algorithms

Solved question bank · every question from 10 term-end papers, December 2021 to June 2026

Trees

Q1. Write an algorithm for inorder traversal of a Binary Tree. [10 marks · J22]

Inorder traversal visits the left subtree, then the root, then the right subtree, producing the sequence Left-Root-Right. On a binary search tree it prints the keys in ascending order.

```
struct node {
    int info;
    struct node *left, *right;
};

void inorder(struct node *root)
{
    if (root != NULL) {
        inorder(root->left);      /* traverse left subtree */
        printf("%d ", root->info); /* visit the root */
        inorder(root->right);     /* traverse right subtree */
    }
}
```

Non-recursive version with an explicit stack:

```
void inorder_nr(struct node *root)
{
    struct node *stack[MAX], *p = root;
    int top = -1;
    while (1) {
        while (p != NULL) { stack[++top] = p; p = p->left; }
        if (top == -1) break;
        p = stack[top--];
        printf("%d ", p->info);
        p = p->right;
    }
}
```

Example on the tree with root A, A->left = B, A->right = C, B->left = D, B->right = E: the output is D B E A C.

Time complexity $O(n)$, space complexity $O(h)$ for the stack, where h is the height ($O(n)$ in the worst case of a skewed tree).

Q2. What is Binary Tree? Write algorithm for Inorder traversal of Binary Tree. [10 marks · J26]

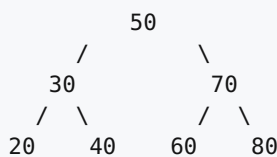
A binary tree is a finite set of nodes that is either empty, or consists of a root node together with two disjoint binary trees called the left subtree and the right subtree. Hence every node has at most two children, and the children are ordered — the left child is distinguishable from the right child.

Properties: a binary tree of height h has at most $2^{(h+1)} - 1$ nodes; a tree with n nodes has height at least $\lceil \log_2(n+1) \rceil - 1$; the number of leaf nodes equals the number of nodes of degree two plus one.

```
struct node { int info; struct node *left, *right; };

void inorder(struct node *root)
{
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->info);
    inorder(root->right);
}
```

Trace on the binary search tree formed by inserting 50, 30, 70, 20, 40, 60, 80:



Inorder output: 20 30 40 50 60 70 80 — sorted, which is the standard use of inorder traversal. Other uses are producing the infix form of an expression tree and copying a BST into a sorted array.

Time complexity $O(n)$, space complexity $O(h)$ for the recursion stack.

Q3. What is a Binary Tree? How does it differ from a tree? Write an algorithm for traversal of a Binary Tree. [10 marks · D22]

A binary tree is either empty or a root with two disjoint, ordered subtrees — the left and the right — so each node has degree at most two.

Tree (general tree)	Binary Tree
A node may have any number of children	A node has at most two children
Children are not ordered; there is no "left"/"right"	Children are ordered: left child and right child are distinct
Cannot be empty (at least a root exists in most definitions)	May be empty (the empty binary tree is valid)
No fixed bound on the number of nodes at a level	Level i holds at most 2^i nodes
Implemented with a variable list of child pointers or first-child/next-sibling	Implemented with exactly two pointers per node
Traversals: level-order, depth-first (preorder/postorder only)	Three natural traversals: inorder, preorder, postorder

```
void preorder(struct node *t) /* Root - Left - Right */
{ if (t) { printf("%d ", t->info); preorder(t->left); preorder(t->right); } }

void inorder(struct node *t) /* Left - Root - Right */
{ if (t) { inorder(t->left); printf("%d ", t->info); inorder(t->right); } }

void postorder(struct node *t) /* Left - Right - Root */
{ if (t) { postorder(t->left); postorder(t->right); printf("%d ", t->info); } }
```

On the tree root A with children B (left) and C (right), B having children D and E: preorder = A B D E C, inorder = D B E A C, postorder = D E B C A.

Each traversal is $O(n)$ in time and $O(h)$ in space.

Q4. What is a Binary Tree? Write an algorithm to traverse a Binary tree in post-order.

[10 marks · D23]

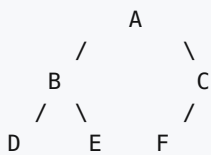
A binary tree is a finite set of nodes which is either empty or consists of a root plus a left binary subtree and a right binary subtree, so no node has more than two children and the two children are ordered.

Postorder traversal visits the left subtree, then the right subtree, then the root (Left-Right-Root). It is used to delete or free a tree (children are released before the parent) and to evaluate an expression tree bottom-up.

```
struct node { int info; struct node *left, *right; };

void postorder(struct node *root)
{
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->info);
}
```

Trace on:



postorder(A) → postorder(B) → postorder(D) prints D, postorder(E) prints E, prints B; then postorder(C) → postorder(F) prints F, prints C; finally prints A. Output: **D E B F C A**.

Time complexity $O(n)$ since every node is visited exactly once; space complexity $O(h)$ for the recursion stack, which is $O(\log n)$ for a balanced tree and $O(n)$ for a skewed tree.

Q5. Write **non-recursive** algorithm for preorder traversal of a Binary tree. Also, explain the algorithm with the help of an example. [10 marks · D24]

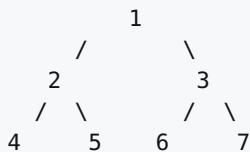
Preorder is Root-Left-Right. The iterative version replaces the recursion stack by an explicit stack: push the root; repeatedly pop a node, visit it, and push its right child first and its left child second, so that the left child is popped (and hence visited) before the right.

```

void preorder_nr(struct node *root)
{
    struct node *stack[MAX], *p;
    int top = -1;
    if (root == NULL) return;
    stack[++top] = root;
    while (top != -1) {
        p = stack[top--];          /* pop          */
        printf("%d ", p->info);    /* visit       */
        if (p->right != NULL) stack[++top] = p->right; /* right first */
        if (p->left  != NULL) stack[++top] = p->left;  /* left  on top */
    }
}

```

Example tree:



Step	Pop / Visit	Stack after the step (top on the right)
0	—	1
1	1	3, 2
2	2	3, 5, 4
3	4	3, 5
4	5	3
5	3	7, 6
6	6	7
7	7	(empty)

Output: **1 2 4 5 3 6 7**, identical to the recursive preorder.

Time complexity $O(n)$; space complexity $O(h)$, at most the height of the tree held on the stack.

Q6. Write **non-recursive** algorithm for postorder traversal of a binary tree. Explain with the help of an example. [10 marks · J25]

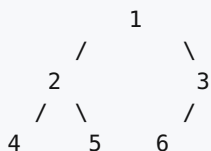
Postorder is Left-Right-Root. A node may be popped only after both its subtrees are done, so the single-stack version remembers the last node visited and only outputs a node when its right subtree has already been output.

```

void postorder_nr(struct node *root)
{
    struct node *stack[MAX], *p = root, *lastVisited = NULL, *peek;
    int top = -1;
    while (p != NULL || top != -1) {
        if (p != NULL) { /* go as far left as possible */
            stack[++top] = p;
            p = p->left;
        } else {
            peek = stack[top];
            if (peek->right != NULL && lastVisited != peek->right)
                p = peek->right; /* right subtree still pending */
            else {
                printf("%d ", peek->info);
                lastVisited = stack[top--];
            }
        }
    }
}

```

Example tree:



Step	Action	Stack (top right)	Output
1	push 1, 2, 4	1,2,4	—
2	4 has no right → visit 4	1,2	4
3	2's right = 5 → push 5	1,2,5	4
4	visit 5	1,2	4 5
5	2's right already visited → visit 2	1	4 5 2
6	1's right = 3 → push 3, then 6	1,3,6	4 5 2
7	visit 6	1,3	4 5 2 6
8	3's right is NULL → visit 3	1	4 5 2 6 3
9	visit 1	empty	4 5 2 6 3 1

Output: **4 5 2 6 3 1**.

Time complexity $O(n)$; space complexity $O(h)$.

Q7. Explain any two types of traversals of binary trees. [10 marks · D21]

Traversal means visiting every node of the tree exactly once in a systematic order. The two explained here are **inorder** and **preorder**.

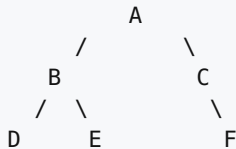
Inorder (Left-Root-Right). Traverse the left subtree in inorder, visit the root, then traverse the right subtree in inorder. Its defining property is that on a binary search tree it yields the keys in ascending order, and on an expression tree it yields the infix form of the expression. Recursive definition:

```
inorder(t): if (t != NULL) { inorder(t->left); visit(t); inorder(t->right); }
```

Preorder (Root-Left-Right). Visit the root first, then traverse the left subtree in preorder, then the right subtree in preorder. Because the root is emitted before its subtrees, preorder output can be used to re-create (copy) the tree, and on an expression tree it yields the prefix (Polish) form.

```
preorder(t): if (t != NULL) { visit(t); preorder(t->left); preorder(t->right); }
```

Illustration on:

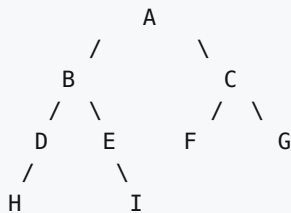


- Inorder: **D B E A C F**
- Preorder: **A B D E C F**

Both traversals touch each of the n nodes once, so both are $O(n)$ in time and $O(h)$ in stack space; they differ only in the position of the "visit the root" step, which is what changes the output order.

Q8. Write preorder and postorder traversals of the following tree: [10 marks · D25]

[The tree is a figure in the source paper and did not survive text extraction. Assumption: the tree below is used, with root A, $A \rightarrow (B, C)$, $B \rightarrow (D, E)$, $C \rightarrow (F, G)$, $D \rightarrow (H, -)$, $E \rightarrow (-, I)$.]



Preorder (Root-Left-Right). Visit A; then B's subtree: B, then D, then H, then E, then I; then C's subtree: C, F, G.

Preorder = **A B D H E I C F G**

Postorder (Left-Right-Root). In B's subtree: D's children first $\rightarrow$ H, then D; then E's subtree $\rightarrow$ I, then E; then B. In C's subtree: F, G, then C. Finally A.

Postorder = **H D I E B F G C A**

For completeness, inorder (Left-Root-Right) = **H D B E I A F C G**.

Step-by-step derivation of postorder, written as the recursion unwinds:

Call	Emits
postorder(H)	H
postorder(D)	D
postorder(I)	I
postorder(E)	E
postorder(B)	B
postorder(F)	F
postorder(G)	G
postorder(C)	C
postorder(A)	A

Each traversal takes $O(n)$ time and $O(h)$ space.

Q9. What is a Tree? How does it differ from a Binary Tree? [10 marks · D21]

A tree is a non-linear hierarchical data structure defined as a finite non-empty set of nodes with one distinguished node called the root, such that the remaining nodes are partitioned into zero or more disjoint sets, each of which is itself a tree (a subtree of the root). Equivalently, it is a connected acyclic graph with n nodes and $n-1$ edges. Terminology: degree of a node (number of children), leaf (degree 0), level of the root = 0, height = longest root-to-leaf path.

Tree	Binary Tree
A node can have any number (0, 1, 2, ... m) of children	A node has at most 2 children
Must contain at least one node (the root); it is non-empty by definition	Can be empty — the empty binary tree is a legal binary tree
Subtrees are an unordered collection	Subtrees are ordered: left subtree and right subtree
A single child is simply "the child"	A single child must be declared either the left child or the right child, and the two cases give different trees
Level i may hold any number of nodes	Level i holds at most 2^i nodes; a tree of height h has at most $2^{h+1} - 1$ nodes
Representation needs a list of children or the first-child/next-sibling scheme	Representation needs exactly two link fields per node
Only preorder, postorder and level-order are meaningful	Inorder is additionally defined

Example: a general tree whose root has three children cannot be a binary tree; after the first-child/next-sibling conversion it becomes a binary tree whose shape is completely different.

Q10. Explain the process of converting a Tree to a Binary Tree with an example. [10 marks · J22]

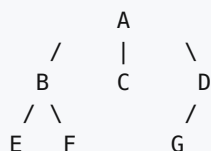
A general tree is converted into a binary tree by the **left-child / right-sibling** rule (also called the natural correspondence):

1. The root of the tree becomes the root of the binary tree.
2. For every node, its **first (leftmost) child** becomes its **left child** in the binary tree.

- Every other child is linked as the **right child of its immediate left sibling**; thus siblings form a right-going chain.
- The right pointer of the original root is set to NULL (when only one tree, not a forest, is converted).

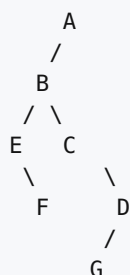
The resulting binary tree needs exactly two pointers per node instead of a variable-length child list, which is why the conversion is done.

Example general tree:



Applying the rule: A's first child B becomes A's left child; C becomes B's right child; D becomes C's right child. B's first child E becomes B's left child; F becomes E's right child. D's first child G becomes D's left child.

Converted binary tree:



Verification: the preorder of the general tree (A B E F C D G) equals the preorder of the converted binary tree, and the postorder of the general tree equals the inorder of the binary tree (E F B C G D A), which is the standard check that the conversion is correct. The conversion is $O(n)$.

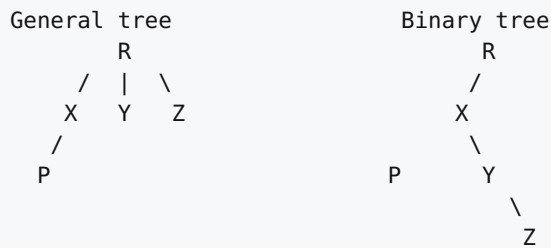
Q11. What are the differences between a tree and a binary tree? Explain the process of converting a tree into a binary tree. [10 marks · D23]

Tree	Binary Tree
Any number of children per node	At most two children per node
Cannot be empty	Can be empty
Children unordered	Left and right children strictly ordered
No bound on nodes per level	At most 2^i nodes at level i
Needs a child list / first-child-next-sibling representation	Needs two link fields per node
Inorder traversal is undefined	Inorder, preorder, postorder all defined

Conversion (left-child / right-sibling rule).

- Keep the root of the tree as the root of the binary tree.
- Make the leftmost child of each node its left child.
- Make each remaining sibling the right child of the sibling immediately to its left.
- Set the root's right pointer to NULL.

Example:



i.e. $R \rightarrow \text{left} = X$; $X \rightarrow \text{left} = P$; $X \rightarrow \text{right} = Y$; $Y \rightarrow \text{right} = Z$.

Check: preorder of the general tree R X P Y Z equals the preorder of the binary tree; postorder of the general tree P X Y Z R equals the inorder of the binary tree. The process visits each node once, so it is $O(n)$ in time and $O(1)$ extra space if done in place.

Q12. What is a Tree? Write an algorithm for implementation of a Binary Tree. [10 marks · J23]

A tree is a finite non-empty set of nodes with a distinguished root, the remaining nodes being partitioned into disjoint subtrees of the root; equivalently a connected acyclic graph with n nodes and $n-1$ edges.

A binary tree is implemented with a linked representation: each node holds the data and two pointers. Implementation means providing node creation and insertion so that a tree can be built, plus a traversal to display it. The insertion below follows the binary-search-tree rule so that the position of each key is well defined.

```
struct node {
    int info;
    struct node *left, *right;
};

struct node *getnode(int x)          /* create one node */
{
    struct node *p = (struct node *) malloc(sizeof(struct node));
    p->info = x; p->left = NULL; p->right = NULL;
    return p;
}

struct node *insert(struct node *root, int x)
{
    if (root == NULL)
        return getnode(x);          /* empty tree or empty subtree */
    if (x < root->info)
        root->left = insert(root->left, x);
    else if (x > root->info)
        root->right = insert(root->right, x);
    return root;                     /* duplicates ignored */
}

void create_tree(void)
{
    struct node *root = NULL;
    int n, i, x;
    scanf("%d", &n);
    for (i = 0; i < n; i++) { scanf("%d", &x); root = insert(root, x); }
    inorder(root);                   /* displays keys in sorted order */
}
```

Assumptions: keys are integers, duplicates are rejected, memory is available.

Example: inserting 40, 20, 60, 10 builds root 40 with left child 20 (whose left child is 10) and right child 60; inorder prints 10 20 40 60.

Time complexity of one insertion $O(h) \rightarrow O(\log n)$ if the tree stays balanced, $O(n)$ in the worst case; building the tree is $O(n \cdot h)$. Space complexity $O(n)$ for n nodes.

Q13. What is a binary tree? What are its properties? Explain how to create a binary tree with the help of a suitable code. [10 marks · D25]

A binary tree is either empty or a root node with two disjoint ordered binary subtrees, the left and the right subtree; every node therefore has degree at most two.

Properties. 1. Level i (root at level 0) contains at most 2^i nodes. 2. A binary tree of height h contains at most $2^{h+1} - 1$ nodes and at least $h+1$ nodes. 3. For n nodes the minimum height is $\lceil \log_2(n+1) \rceil - 1$. 4. If n_0 is the number of leaves and n_2 the number of nodes with two children, then $n_0 = n_2 + 1$. 5. In the array representation of a complete binary tree, the children of node i are at $2i+1$ and $2i+2$ and the parent is at $\lfloor (i-1)/2 \rfloor$. 6. A binary tree with n nodes has exactly $n-1$ edges and $n+1$ NULL links.

Creation code.

```
struct node { int info; struct node *left, *right; };

struct node *create(void) /* builds the tree interactively */
{
    int x; struct node *p;
    printf("Enter data (-1 for empty subtree): "); scanf("%d", &x);
    if (x == -1) return NULL; /* sentinel marks an absent child */
    p = (struct node *) malloc(sizeof(struct node));
    p->info = x;
    printf("Left child of %d:\n", x); p->left = create();
    printf("Right child of %d:\n", x); p->right = create();
    return p;
}
```

Entering 10, 20, -1, -1, 30, -1, -1 creates root 10 with left child 20 and right child 30. The function performs a preorder construction, calling itself once per node and once per NULL link.

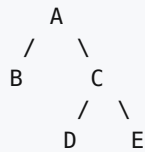
Time complexity $O(n)$; space complexity $O(n)$ for the nodes plus $O(h)$ recursion stack.

Q14. What is a full tree? What is a complete tree? Explain the implementation of a tree using linked list. [10 marks · J24]

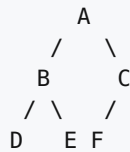
Full (strictly) binary tree. Every node has either 0 or 2 children — no node has exactly one child. A full binary tree of height h with all leaves at level h holds exactly $2^{h+1} - 1$ nodes and 2^h leaves, and is then also called a perfect binary tree.

Complete binary tree. All levels are completely filled except possibly the last, and the nodes of the last level are filled from left to right with no gaps. Consequence: a complete binary tree with n nodes has height $\lceil \log_2 n \rceil$ and can be stored in an array without wasted cells, which is exactly why heaps are complete trees. Every perfect tree is complete; a complete tree need not be full.

Full but not complete



Complete but not full



Implementation of a general tree using a linked list — the first-child / next-sibling representation. Each node stores the data, one pointer to its leftmost child and one pointer to its next sibling, so a node of any degree needs only two pointers.

```

struct tnode {
    int info;
    struct tnode *child;    /* leftmost child */
    struct tnode *sibling;  /* next sibling */
};

struct tnode *newnode(int x)
{
    struct tnode *p = (struct tnode *) malloc(sizeof(struct tnode));
    p->info = x; p->child = NULL; p->sibling = NULL; return p;
}

void addchild(struct tnode *parent, struct tnode *c)
{
    struct tnode *q;
    if (parent->child == NULL) parent->child = c;
    else {
        q = parent->child;
        while (q->sibling != NULL) q = q->sibling; /* walk the sibling chain */
        q->sibling = c;
    }
}
  
```

For a root R with children X, Y, Z: R->child = X, X->sibling = Y, Y->sibling = Z. Space is O(n) with exactly 2n pointers; adding a child at the end costs O(d) where d is the parent's degree, O(1) if the last child is also remembered.

Q15. Write a short note on Binary Search Trees. [10 marks · J23]

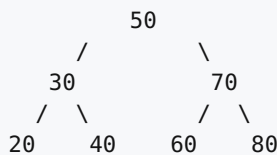
A **binary search tree (BST)** is a binary tree in which, for every node x, all keys in the left subtree of x are less than x->info and all keys in the right subtree are greater than x->info, and both subtrees are themselves BSTs. Duplicates are normally disallowed.

Key consequences. - The inorder traversal of a BST lists the keys in ascending order. - Search, insert and delete all follow a single root-to-leaf path, so each costs O(h) where h is the height — O(log n) when the tree is balanced, O(n) when the insertions arrive in sorted order and the tree degenerates into a linked list. - The minimum key is the leftmost node, the maximum the rightmost.

```

struct node *search(struct node *t, int key)
{
    while (t != NULL && t->info != key)
        t = (key < t->info) ? t->left : t->right;
    return t; /* NULL if key absent */
}
  
```

Example: inserting 50, 30, 70, 20, 40, 60, 80 gives



Inorder: 20 30 40 50 60 70 80. Searching for 40 compares with 50 (go left), 30 (go right), 40 (found) — 3 comparisons.

Deletion has three cases: a leaf is simply removed; a node with one child is replaced by that child; a node with two children is replaced by its inorder successor (leftmost node of the right subtree), which is then deleted recursively.

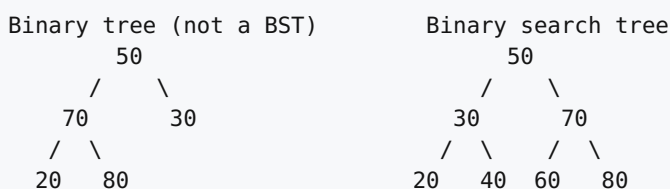
Uses. Dictionary and symbol-table implementation, maintaining a dynamic sorted set, and as the base of self-balancing variants (AVL, Red-Black) which guarantee $O(\log n)$.

Q16. What is a Binary Search Tree? How does it differ from a Binary Tree? [10 marks · D23]

A binary search tree is a binary tree that additionally satisfies the ordering (search-tree) property: for every node, every key in its left subtree is smaller than the node's key and every key in its right subtree is larger, and this holds recursively in both subtrees.

Binary Tree	Binary Search Tree
No relation is required among the keys of parent and children	Left subtree < node < right subtree, for every node
Searching for a key must examine, in the worst case, all n nodes — $O(n)$	Searching follows one root-to-leaf path — $O(h)$, i.e. $O(\log n)$ when balanced
Inorder traversal gives no particular order	Inorder traversal gives the keys in ascending order
Insertion can place a new node anywhere	The insertion position of a key is uniquely determined by comparisons
Used for hierarchy representation, expression trees, heaps (with the heap property instead)	Used for dictionaries, symbol tables and dynamic sorted sets
Every BST is a binary tree	Not every binary tree is a BST

Example — the same seven keys:



The left tree violates the property because 70 (greater than 50) sits in the left subtree. Searching 80 in it may need to examine most nodes; in the BST it needs 2 comparisons ($50 \rightarrow 70 \rightarrow 80$, i.e. 3 nodes examined along one path).

Note that an unbalanced BST built from sorted input degenerates to a chain and loses the $O(\log n)$ advantage — the motivation for AVL and Red-Black trees.

Q17. What are the properties of an AVL tree? Explain the possible rotations that are possible on an unbalanced AVL tree. [10 marks · D22]

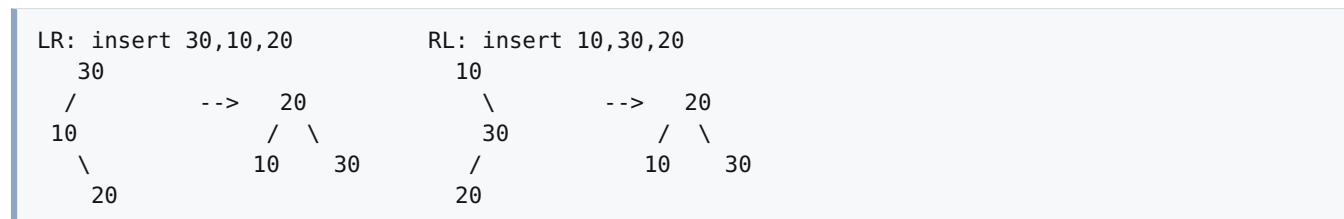
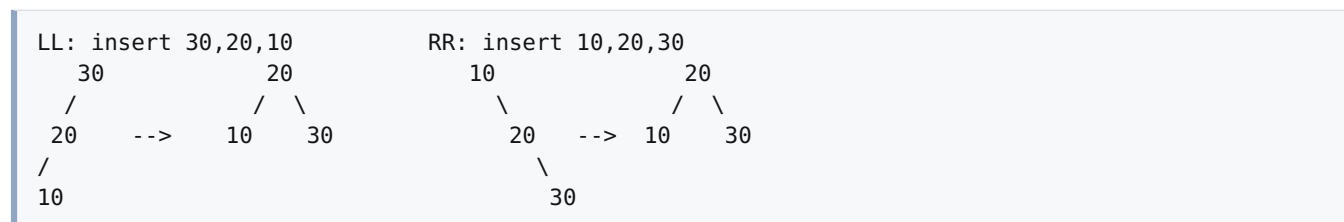
An **AVL tree** (Adelson-Velskii and Landis) is a height-balanced binary search tree.

Properties. 1. It is a binary search tree. 2. For every node, the **balance factor** = $\text{height}(\text{left subtree}) - \text{height}(\text{right subtree}) \in \{-1, 0, +1\}$. 3. A violation (balance factor ± 2) is repaired by a rotation immediately after the insertion or deletion that caused it. 4. The height of an AVL tree with n nodes is $O(\log n)$; more precisely $h \leq 1.44 \log_2(n+2) - 0.328$. 5. Search, insert and delete are therefore guaranteed $O(\log n)$, unlike a plain BST. 6. Insertion needs at most one single or double rotation; deletion may need up to $O(\log n)$ rotations along the path to the root.

The four rotations. Let A be the lowest node that became unbalanced.

Case	Where the new node went	Fix
LL	Left subtree of the left child of A	Single right rotation about A
RR	Right subtree of the right child of A	Single left rotation about A
LR	Right subtree of the left child of A	Double: left rotation about the left child, then right rotation about A
RL	Left subtree of the right child of A	Double: right rotation about the right child, then left rotation about A

Examples.



Each rotation only relinks a constant number of pointers, so a rotation is $O(1)$ and the whole rebalanced insertion remains $O(\log n)$.

Q18. Discuss the concept of AVL tree rotations and their role in balancing the tree. [10 marks · D24]

A plain binary search tree built from sorted keys degenerates into a linked list, making search $O(n)$. An AVL tree prevents this by keeping, at every node, the balance factor $\text{height}(\text{left}) - \text{height}(\text{right})$ in $\{-1, 0, +1\}$. An insertion or deletion can push one node's balance factor to $+2$ or -2 ; a **rotation** is the local, constant-time pointer rearrangement that restores the balance factor while preserving the inorder (sorted) sequence of keys — this preservation is the essential point, because the tree must remain a BST.

Role. After an insertion, walk back from the new leaf to the root updating heights; at the first (lowest) node A whose balance factor becomes ± 2 , apply the rotation for the matching case. One

single or double rotation suffices for an insertion; the subtree then regains its pre-insertion height and no ancestor can remain unbalanced. For deletion the rebalancing may have to be repeated at each ancestor, so up to $O(\log n)$ rotations.

The four cases. - **LL** — insertion in the left subtree of A's left child → single **right** rotation at A. - **RR** — insertion in the right subtree of A's right child → single **left** rotation at A. - **LR** — insertion in the right subtree of A's left child → left rotation at A → left, then right rotation at A. - **RL** — insertion in the left subtree of A's right child → right rotation at A → right, then left rotation at A.

Worked example — insert 10, 20, 30 in order. After 10 and 20 the tree is balanced. Inserting 30 gives node 10 a balance factor of -2 with the new node in the right subtree of its right child: an **RR** case. Left-rotating about 10 gives root 20 with children 10 and 30, all balance factors 0.

```
struct node *rotate_left(struct node *A)
{
    struct node *B = A->right;
    A->right = B->left; B->left = A;
    update_height(A);    update_height(B);
    return B;             /* B is the new subtree root */
}
```

A rotation is $O(1)$; insertion with rebalancing is $O(\log n)$ time and $O(\log n)$ stack space.

Q19. What are the characteristics of an AVL tree? How do rotations help in balancing an AVL tree when it becomes unbalanced? [10 marks · J25]

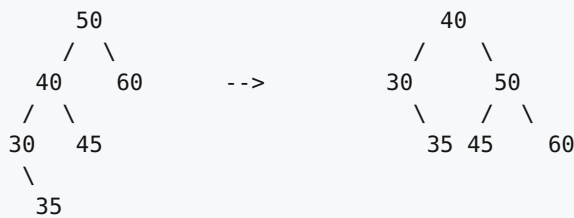
Characteristics. 1. It is a binary search tree, so inorder traversal is sorted. 2. Each node stores its height (or its balance factor). 3. Balance factor = $\text{height}(\text{left}) - \text{height}(\text{right})$ and must be -1 , 0 or $+1$ for every node. 4. Height is bounded by about $1.44 \log_2 n$, so search/insert/delete are guaranteed $O(\log n)$. 5. Rebalancing is performed by rotations, which are $O(1)$ each. 6. It is more rigidly balanced than a Red-Black tree, so lookups are faster but insert/delete do more rotations.

How rotations restore balance. A rotation changes the parent-child relation between a node and one of its children without changing the left-to-right order of the keys; therefore it reduces the height of the taller side by one and increases the shorter side by one, turning a balance factor of ± 2 back into 0 or ± 1 . The case is identified by the first two steps of the path from the unbalanced node A towards the newly inserted node:

Path from A	Case	Rotation
Left, Left	LL	single right rotation at A
Right, Right	RR	single left rotation at A
Left, Right	LR	left at A → left, then right at A
Right, Left	RL	right at A → right, then left at A

Worked example — insert 50, 40, 60, 30, 45, 35.

- After 50, 40, 60, 30, 45 the tree is balanced: root 50, left 40 (children 30, 45), right 60.
- Insert 35 as the right child of 30. Heights: node 40 now has left height 1 (30→35) and right height 0 → balance $+1$, fine. Node 50 has left height 2 and right height 0 → balance $+2$, unbalanced. The path from 50 to 35 goes Left (to 40) then Left (to 30): an **LL** case.
- Single right rotation at 50: 40 becomes the root, its right child becomes 50, and 40's old right child 45 becomes 50's left child.



All balance factors are now in $\{-1, 0, +1\}$.

Q20. What is AVL tree? Explain insertion of a node into an AVL tree. [5 marks · D25]

An AVL tree is a self-balancing binary search tree in which, for every node, the balance factor $\text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$ is $-1, 0$ or $+1$; this keeps the height $O(\log n)$ and all operations $O(\log n)$.

Insertion. (i) Insert the key exactly as in an ordinary BST, at a leaf position found by comparisons. (ii) Retrace the path from the new leaf to the root, updating heights. (iii) At the **lowest** node A whose balance factor becomes ± 2 , identify the case from the first two edges of the path towards the new node and rotate: LL $\rightarrow$ single right at A, RR $\rightarrow$ single left at A, LR $\rightarrow$ left at A $\rightarrow$ left then right at A, RL $\rightarrow$ right at A $\rightarrow$ right then left at A. One single or double rotation is always enough, because the rebalanced subtree regains its original height.

Example — insert 10, 20, 30: inserting 30 makes node 10 unbalanced (-2), an RR case; a single left rotation at 10 gives root 20 with children 10 and 30.

Time complexity $O(\log n)$ per insertion ($O(1)$ for the rotation itself); space $O(\log n)$ for the recursion.

Q21. What is AVL tree? Write algorithms for: (i) Insertion of a node into an AVL tree — 5, (ii) Deletion of a node from an AVL-tree — 5. [10 marks · J26]

An AVL tree is a height-balanced BST in which every node's balance factor, $\text{height}(\text{left}) - \text{height}(\text{right})$, lies in $\{-1, 0, +1\}$; rotations (LL, RR, LR, RL) restore the property after an update, keeping the height $O(\log n)$.

```

struct node { int info; struct node *left, *right; int ht; };

int height(struct node *p) { return (p == NULL) ? -1 : p->ht; }
int max(int a, int b)      { return (a > b) ? a : b; }
int bf(struct node *p)     { return height(p->left) - height(p->right); }

struct node *rotR(struct node *A)          /* single right rotation */
{
    struct node *B = A->left;
    A->left = B->right; B->right = A;
    A->ht = 1 + max(height(A->left), height(A->right));
    B->ht = 1 + max(height(B->left), height(B->right));
    return B;
}

struct node *rotL(struct node *A)          /* single left rotation */
{
    struct node *B = A->right;
    A->right = B->left; B->left = A;
    A->ht = 1 + max(height(A->left), height(A->right));
    B->ht = 1 + max(height(B->left), height(B->right));
    return B;
}

struct node *balance(struct node *t)
{
    t->ht = 1 + max(height(t->left), height(t->right));
    if (bf(t) > 1) { /* left heavy */
        if (bf(t->left) < 0) t->left = rotL(t->left); /* LR case */
        return rotR(t); /* LL case */
    }
    if (bf(t) < -1) { /* right heavy */
        if (bf(t->right) > 0) t->right = rotR(t->right); /* RL case */
        return rotL(t); /* RR case */
    }
    return t;
}

```

(i) Insertion

```

struct node *insert(struct node *t, int x)
{
    if (t == NULL) {
        t = (struct node *) malloc(sizeof(struct node));
        t->info = x; t->left = t->right = NULL; t->ht = 0;
        return t;
    }
    if (x < t->info) t->left = insert(t->left, x);
    else if (x > t->info) t->right = insert(t->right, x);
    else return t; /* duplicate: no change */
    return balance(t);
}

```

(ii) Deletion

```

struct node *minnode(struct node *t)
{ while (t->left != NULL) t = t->left; return t; }

struct node *delete(struct node *t, int x)
{
    struct node *tmp;
    if (t == NULL) return NULL;          /* key not present */
    if (x < t->info) t->left = delete(t->left, x);
    else if (x > t->info) t->right = delete(t->right, x);
    else {
        if (t->left == NULL || t->right == NULL) { /* 0 or 1 child */
            tmp = (t->left != NULL) ? t->left : t->right;
            free(t);
            return tmp;          /* may be NULL */
        }
        tmp = minnode(t->right); /* inorder successor */
        t->info = tmp->info;
        t->right = delete(t->right, tmp->info);
    }
    return balance(t);          /* rebalance on the way up */
}

```

Insertion needs at most one (single or double) rotation; deletion may need a rotation at every node on the path, i.e. up to $O(\log n)$ rotations. Both operations run in $O(\log n)$ time with $O(\log n)$ stack space.

Q22. What is a B-tree and what are its properties? Create a B-tree of order 5, if the following list of keys are inserted into the empty B-tree in the order:

25, 10, 15, 28, 40, 30, 5, 2, 35, 42, 50 [10 marks · D24]

A **B-tree** of order m is a balanced multiway search tree designed for block-oriented storage: one node is one disk block, so the tree is shallow and a search costs few disk accesses.

Properties of a B-tree of order m . 1. Every node has at most m children and at most $m-1$ keys. 2. Every node other than the root and the leaves has at least $\lceil m/2 \rceil$ children, i.e. at least $\lceil m/2 \rceil - 1$ keys. 3. The root has at least 2 children unless it is a leaf. 4. Keys inside a node are kept in ascending order, and a node with k keys has exactly $k+1$ child pointers; the subtree between two keys holds values between them. 5. All leaves appear at the same level — the tree is perfectly height-balanced. 6. Growth is upward: a node that overflows is split at its median, which is promoted to the parent; the height increases only when the root splits.

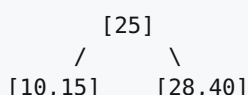
For order 5: maximum 4 keys and 5 children per node, minimum 2 keys in a non-root node, split when a 5th key arrives.

Insertion trace.

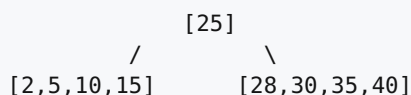
Step	Key	Tree after insertion
1	25	[25]
2	10	[10, 25]
3	15	[10, 15, 25]
4	28	[10, 15, 25, 28] — node now full
5	40	Overflow: 10,15, 25 ,28,40 → median 25 promoted; split
6	30	goes into the right leaf
7	5	goes into the left leaf
8	2	left leaf becomes full
9	35	right leaf becomes full
10	42	Overflow in the right leaf: 28,30, 35 ,40,42 → median 35 promoted; split
11	50	goes into the rightmost leaf

Trees drawn out:

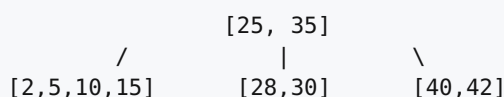
After step 5 (first split):



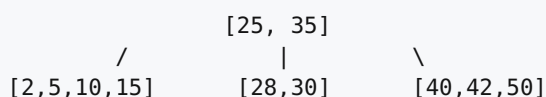
After step 9:



After step 10 (second split, 35 promoted into the root):



After step 11 (final B-tree):



Height = 1 (two levels), all leaves at the same level, every node within the 2-4 key limits. Search/insert/delete cost $O(\log_{\lfloor m/2 \rfloor} n)$ node accesses.

Q23. What are B-trees? What are its applications? [5 marks · D25]

A B-tree of order m is a height-balanced multiway search tree in which each node holds up to $m-1$ ordered keys and up to m child pointers, every non-root node holds at least $\lfloor m/2 \rfloor - 1$ keys, and all leaves lie at the same level. Overflow is handled by splitting a node at its median key and promoting that key to the parent, so the tree grows at the root and stays perfectly balanced. Because one node maps onto one disk block, a search touches only $O(\log_{\lfloor m/2 \rfloor} n)$ blocks — typically 3 or 4 for millions of records.

Applications. - Indexing in database management systems (the B+-tree variant is the standard index structure in Oracle, MySQL/InnoDB, PostgreSQL). - File-system directory and metadata indexes (NTFS, HFS+, ext4 htree, XFS). - Indexed sequential file organisation, where the index is a B-tree over the key field. - Multilevel indexing and external sorting/searching where the data is too large for main memory. - Any dictionary on secondary storage needing both random and ordered (range) access, since the leaves keep keys in sorted order.

Q24. What is Red-Black tree? Write its properties. [5 marks · J26]

A Red-Black tree is a self-balancing binary search tree in which each node carries one extra bit of colour information, red or black. The colouring rules force the longest root-to-leaf path to be at most twice the shortest, which bounds the height by $2 \log_2(n+1)$ and guarantees $O(\log n)$ search, insertion and deletion. Balance is restored after an update by recolouring plus at most two rotations, which is cheaper than AVL rebalancing, so Red-Black trees are preferred when insertions and deletions are frequent (they implement C++ `std::map` and Java `TreeMap`).

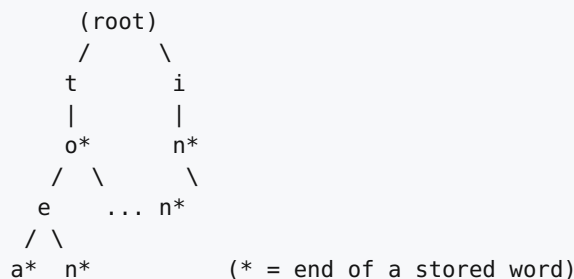
Properties. 1. Every node is either red or black. 2. The root is black. 3. Every leaf (the NULL sentinel nodes) is black. 4. A red node cannot have a red child — no two red nodes are adjacent, so red nodes never appear consecutively on a path. 5. Every path from a given node down to any descendant leaf contains the same number of black nodes (this count is the node's black height). 6. Consequently the height of a Red-Black tree with n internal nodes is at most $2 \log_2(n+1)$.

Insertion colours the new node red and then repairs violations of rule 4 by recolouring and rotating; deletion repairs violations of rule 5 in the same way.

Q25. What are Tries? List the main characteristics of tries. [10 marks · J24]

A **trie** (from *retrieval*; also called a prefix tree or digital search tree) is an ordered multiway tree used to store a set of strings, in which the position of a node in the tree — not any key stored in it — encodes the string. The root represents the empty string; each edge is labelled with one character; the string associated with a node is the concatenation of the edge labels on the path from the root to that node. A boolean flag (or an end-of-string marker such as `$`) in a node says whether that path spells a complete key.

Example — the keys **to**, **tea**, **ten**, **in**, **inn**:



Main characteristics. 1. Keys are stored by path, so all strings sharing a prefix share the same initial nodes — common prefixes are stored once. 2. Search, insertion and deletion take $O(L)$ time, where L is the length of the key, and are **independent of n** , the number of keys stored — unlike a BST where they cost $O(L \log n)$. 3. No collisions, unlike hashing, and the keys come out in lexicographic order by a preorder traversal — something a hash table cannot do. 4. Each node has up to R children, R being the alphabet size (26 for lowercase letters, 256 for bytes); a node is

typically an array of R pointers, which makes a trie space-hungry — $O(n \cdot L \cdot R)$ pointers in the worst case. 5. Space is reduced by compressed variants: the Patricia trie / radix tree merges every chain of single-child nodes into one edge, and the suffix tree is a compressed trie of all suffixes of a string. 6. No node stores a whole key, and the height equals the length of the longest key. 7. Supports prefix queries natively: "all keys starting with te " is just the subtree below the path $t-e$.

Applications. Dictionary and spell-checkers, autocomplete and predictive text, IP routing tables (longest-prefix match using a radix trie), word games, and string pattern matching via suffix trees.

Graphs

Q1. Write Kruskal's algorithm. [10 marks · asked 2x: D21, J23]

Kruskal's algorithm finds a minimum cost spanning tree of a connected, undirected, weighted graph $G = (V, E)$ by the greedy rule: repeatedly add the cheapest remaining edge that does not form a cycle, until $V - 1$ edges have been taken. The cycle test is done with a disjoint-set (union-find) structure: an edge is rejected exactly when both of its endpoints are already in the same set.

```
/* G has n vertices and m edges, edge[i] = (u, v, w) */
void kruskal(int n, int m, EDGE edge[], EDGE T[])
{
    int i, count = 0, cost = 0, ru, rv;
    int parent[MAX];

    sort(edge, m); /* ascending order of weight w */
    for (i = 0; i < n; i++)
        parent[i] = i; /* each vertex its own set */

    for (i = 0; i < m && count < n - 1; i++) {
        ru = find(parent, edge[i].u);
        rv = find(parent, edge[i].v);
        if (ru != rv) { /* no cycle is formed */
            T[count++] = edge[i];
            cost += edge[i].w;
            parent[ru] = rv; /* union of the two sets */
        }
    }
    if (count != n - 1) printf("Graph is not connected: no spanning tree\n");
    else printf("Minimum cost = %d\n", cost);
}

int find(int parent[], int x)
{ while (parent[x] != x) x = parent[x]; return x; }
```

Assumptions: the graph is connected and undirected, weights may be any reals, and ties in weight are broken arbitrarily (different MSTs of equal cost may result).

Time complexity $O(E \log E)$ — dominated by sorting the edges; the union-find operations add only $O(E \alpha(V))$, effectively linear. Space complexity $O(V + E)$.

Q2. Write Kruskal's algorithm to create minimum cost spanning tree. [10 marks · D23]

A spanning tree of a connected graph with n vertices is a subgraph that is a tree and includes all n vertices, hence exactly $n - 1$ edges. A minimum cost spanning tree (MCST) is a spanning tree

whose total edge weight is minimum. Kruskal's method builds it by scanning the edges in non-decreasing order of weight and accepting an edge only if it joins two different components.

```

Algorithm KRUSKAL(G)
Input : graph G = (V, E) with n = |V|, m = |E|, weights w[]
Output: set T of edges of a minimum cost spanning tree

1. T = empty; cost = 0; count = 0;
2. Sort all m edges in ascending order of weight;
3. for (i = 0; i < n; i++) parent[i] = i;          /* n singleton sets */
4. i = 0;
5. while (count < n - 1 && i < m) {
    (u, v) = edge[i]; i = i + 1;
    ru = find(u); rv = find(v);
    if (ru == rv) continue;                        /* would create a cycle */
    T = T union {(u, v)};
    cost = cost + w(u, v);
    count = count + 1;
    parent[ru] = rv;                               /* union */
}
6. if (count < n - 1) print "graph disconnected";
   else print T, cost;
7. stop.

```

The loop invariant is that T is always a subset of edges of some MCST (the cut property of greedy spanning-tree construction), which is why the greedy choice is safe.

Time complexity $O(E \log E) \approx O(E \log V)$; space complexity $O(V + E)$.

Q3. Explain Kruskal's algorithm for finding minimum cost spanning tree in a graph with the help of an example. [10 marks · J25]

Kruskal's algorithm is a greedy method. Sort all edges by weight; scan them from cheapest to costliest; accept an edge if its two endpoints lie in different connected components, otherwise reject it because it would close a cycle; stop when $n - 1$ edges have been accepted. The components are maintained by union-find.

Example graph (5 vertices, 7 edges):

edges: A-B = 2, A-D = 6, B-C = 3, B-D = 8, B-E = 5, C-E = 7, D-E = 9

Sorted edge list: A-B(2), B-C(3), B-E(5), A-D(6), C-E(7), B-D(8), D-E(9).

Step	Edge considered	Weight	Endpoints in same set?	Action	Accepted edges	Running cost
1	A-B	2	no	accept	{AB}	2
2	B-C	3	no	accept	{AB, BC}	5
3	B-E	5	no	accept	{AB, BC, BE}	10
4	A-D	6	no	accept	{AB, BC, BE, AD}	16
5	—	—	—	4 = n-1 edges reached, stop	—	16

Edges C-E(7), B-D(8), D-E(9) are never needed; had the scan continued, each would have been rejected as both endpoints are already connected.

Minimum cost spanning tree: A-B, B-C, B-E, A-D with **total cost 16**.

```
D --6-- A --2-- B --3-- C
              |
              5
              |
              E
```

Time complexity $O(E \log E)$ for the sort plus near-linear union-find; space $O(V + E)$.

Q4. Write Prim's Algorithm to find minimum cost spanning tree (MCST). [10 marks · J24]

Prim's algorithm grows a single tree from an arbitrary start vertex: at every step it adds the cheapest edge that crosses the cut between the vertices already in the tree and those outside it, until all n vertices are in the tree.

```
/* cost[i][j] = weight of edge (i,j), INF if absent; n vertices 0..n-1 */
void prim(int n, int cost[][MAX])
{
    int visited[MAX], near[MAX], dist[MAX];
    int i, j, k, min, total = 0;

    for (i = 0; i < n; i++) {
        visited[i] = 0;
        dist[i] = cost[0][i];      /* start from vertex 0 */
        near[i] = 0;
    }
    visited[0] = 1; dist[0] = 0;

    for (k = 1; k < n; k++) {      /* pick n-1 edges */
        min = INF; j = -1;
        for (i = 0; i < n; i++)    /* cheapest fringe vertex */
            if (!visited[i] && dist[i] < min) { min = dist[i]; j = i; }

        if (j == -1) { printf("Graph not connected\n"); return; }

        visited[j] = 1;
        total += min;
        printf("Edge (%d, %d) weight %d\n", near[j], j, min);

        for (i = 0; i < n; i++)    /* update the fringe */
            if (!visited[i] && cost[j][i] < dist[i]) {
                dist[i] = cost[j][i];
                near[i] = j;
            }
    }
    printf("Minimum cost = %d\n", total);
}
```

Assumptions: the graph is connected, undirected and weighted; `dist[i]` is the cheapest known edge from the tree to i and `near[i]` its tree endpoint.

Time complexity $O(V^2)$ with this adjacency-matrix version, improvable to $O(E \log V)$ with a binary heap as the priority queue. Space complexity $O(V^2)$ for the matrix, $O(V)$ for the auxiliary arrays.

Q5. Write Prim's algorithm to find Minimum Cost Spanning Tree (MCST) and explain it.

[10 marks · J26]

```

void prim(int n, int cost[][MAX], int start)
{
    int visited[MAX], dist[MAX], near[MAX], i, j, k, min, total = 0;

    for (i = 0; i < n; i++) { visited[i] = 0; dist[i] = cost[start][i]; near[i] = start; }
    visited[start] = 1;

    for (k = 1; k < n; k++) {
        min = INF; j = -1;
        for (i = 0; i < n; i++) if (!visited[i] && dist[i] < min) { min = dist[i]; j = i; }
        if (j == -1) { printf("not connected\n"); return; }
        visited[j] = 1; total += min;
        printf("(%d,%d) = %d\n", near[j], j, min);
        for (i = 0; i < n; i++)
            if (!visited[i] && cost[j][i] < dist[i]) { dist[i] = cost[j][i]; near[i] = j; }
    }
    printf("MCST cost = %d\n", total);
}

```

Explanation. The algorithm keeps two sets: the tree set (`visited`) and the remaining vertices. For each remaining vertex `i` it keeps `dist[i]`, the weight of the cheapest edge joining `i` to the tree, and `near[i]`, the tree-side endpoint of that edge. Each iteration selects the remaining vertex with the smallest `dist`, moves it into the tree, outputs the edge (`near[j]`, `j`), and then relaxes the fringe: any vertex `i` that is now closer to `j` than to the previous tree gets `dist[i] = cost[j][i]`. After $n - 1$ iterations every vertex is in the tree, and by the cut property the chosen edges form a minimum cost spanning tree.

Worked example on A-B 2, A-D 6, B-C 3, B-D 8, B-E 5, C-E 7, D-E 9 starting at A:

Iteration	Tree so far	dist (B, C, D, E)	Vertex chosen	Edge added	Cost
init	{A}	2, ∞ , 6, ∞	—	—	0
1	{A,B}	—, 3, 6, 5	B	A-B	2
2	{A,B,C}	—, —, 6, 5	C	B-C	3
3	{A,B,C,E}	—, —, 6, —	E	B-E	5
4	{A,B,C,E,D}	—	D	A-D	6

MCST = {A-B, B-C, B-E, A-D}, total cost **16**.

Time complexity $O(V^2)$ ($O(E \log V)$ with a heap); space $O(V)$ besides the graph.

Q6. What is spanning tree? Write and explain Prim's algorithm to find minimum cost spanning tree. Also, write the applications of spanning tree. [10 marks · D25]

Spanning tree. For a connected undirected graph $G = (V, E)$ with $|V| = n$, a spanning tree is a connected acyclic subgraph containing all n vertices and exactly $n - 1$ edges. A graph generally has many spanning trees; a complete graph on n vertices has n^{n-2} of them. A **minimum cost spanning tree** is one whose sum of edge weights is least.

Prim's algorithm.

```
void prim(int n, int cost[][MAX])
{
    int visited[MAX], dist[MAX], near[MAX], i, j, k, min, total = 0;
    for (i = 0; i < n; i++) { visited[i] = 0; dist[i] = cost[0][i]; near[i] = 0; }
    visited[0] = 1;
    for (k = 1; k < n; k++) {
        min = INF; j = -1;
        for (i = 0; i < n; i++) if (!visited[i] && dist[i] < min) { min = dist[i]; j = i; }
        visited[j] = 1; total += min;
        printf("Edge (%d,%d) cost %d\n", near[j], j, min);
        for (i = 0; i < n; i++)
            if (!visited[i] && cost[j][i] < dist[i]) { dist[i] = cost[j][i]; near[i] = j; }
    }
    printf("Total = %d\n", total);
}
```

Explanation. Start with any one vertex as the partial tree. At each of the $n - 1$ iterations, examine every edge with one endpoint inside the tree and one outside (the *cut*), pick the lightest, and add its outside endpoint to the tree. The `dist/near` arrays make this selection $O(V)$ rather than $O(E)$ per step. The greedy choice is correct by the cut property: the lightest edge crossing any cut belongs to some MCST.

Applications of spanning trees. - Least-cost network design: laying telephone, electrical, water, road or cable networks connecting all sites with minimum total length. - Designing LAN/WAN topology; the Spanning Tree Protocol (IEEE 802.1D) prevents switching loops in Ethernet. - Approximation algorithms, e.g. a 2-approximation for the travelling salesman problem. - Cluster analysis and image segmentation (single-linkage clustering is an MST computation). - Routing and broadcasting in networks, and circuit/VLSI wiring layout. - Civil-engineering pipeline and irrigation-channel planning.

Complexity $O(V^2)$, or $O(E \log V)$ with a priority queue.

Q7. What is meant by Minimum Cost Spanning Tree (MCST)? How can you find it? [10 marks · J22]

A spanning tree of a connected undirected graph $G = (V, E)$, $|V| = n$, is a subgraph which is a tree and contains all n vertices, so it has exactly $n - 1$ edges and no cycle. When the edges carry weights, the **cost** of a spanning tree is the sum of the weights of its $n - 1$ edges, and a **minimum cost spanning tree** is a spanning tree of least cost. It need not be unique — equal-weight edges can give several MCSTs of the same total cost — but its cost is unique.

Properties. An MCST contains no cycle; removing any of its edges disconnects it; adding any non-tree edge creates exactly one cycle, and in an MCST that added edge is never lighter than the heaviest edge of the cycle it creates. Both construction methods rely on the **cut property**: the lightest edge crossing any partition of V into two non-empty parts belongs to some MCST.

How it is found — two greedy algorithms.

Kruskal's algorithm	Prim's algorithm
Edge-based: sort all E edges by weight	Vertex-based: start from one arbitrary vertex
Repeatedly take the next cheapest edge if it joins two different components (union-find cycle test)	Repeatedly take the cheapest edge from the built tree to a vertex outside it
The intermediate structure is a forest	The intermediate structure is always one tree
$O(E \log E)$ — good for sparse graphs	$O(V^2)$ with a matrix, $O(E \log V)$ with a heap — good for dense graphs

Example on A-B 2, A-D 6, B-C 3, B-D 8, B-E 5, C-E 7, D-E 9: both methods select A-B(2), B-C(3), B-E(5), A-D(6) giving an MCST of cost **16**.

Q8. What is Depth First Search? Explain it with an example. [10 marks · D21]

Depth First Search (DFS) is a graph traversal that starts at a chosen source vertex, visits it, and then explores as far as possible along each branch before backtracking: from the current vertex it moves to any unvisited adjacent vertex and repeats; when no unvisited neighbour remains it backtracks to the previous vertex. A `visited[]` array prevents revisiting, and a **stack** (explicit, or the recursion stack) holds the backtracking path.

```
void dfs(int v, int n, int a[][MAX], int visited[])
{
    int w;
    visited[v] = 1;
    printf("%d ", v);
    for (w = 0; w < n; w++)
        if (a[v][w] == 1 && !visited[w])
            dfs(w, n, a, visited);
}
```

Example graph (adjacency lists):

```
A : B, C      D : B
B : A, D, E   E : B, F
C : A, F      F : C, E
```

Trace from A:

Step	Current vertex	Action	Stack (bottom→top)	Output
1	A	visit A, go to B	A	A
2	B	visit B, go to D	A B	A B
3	D	visit D, no unvisited neighbour → backtrack	A B D	A B D
4	B	go to E	A B	A B D
5	E	visit E, go to F	A B E	A B D E
6	F	visit F, go to C	A B E F	A B D E F
7	C	visit C, backtrack all the way	A B E F C	A B D E F C

DFS order: A B D E F C.

Uses: detecting cycles, finding connected components, topological sorting, strongly connected components, maze and puzzle solving, and finding articulation points/bridges.

Time complexity $O(V + E)$ with adjacency lists, $O(V^2)$ with an adjacency matrix; space complexity $O(V)$ for the visited array and the stack.

Q9. What is Breadth-First Search? Explain it with an example. [10 marks · J22]

Breadth First Search (BFS) visits the source vertex, then all vertices at distance 1 from it, then all at distance 2, and so on — a level-by-level traversal. It uses a **queue**: dequeue a vertex, visit all of its unvisited adjacent vertices and enqueue them, marking each as visited at the moment of enqueueing so that no vertex enters the queue twice.

```
void bfs(int start, int n, int a[][MAX])
{
    int queue[MAX], front = 0, rear = 0, visited[MAX], v, w;
    for (v = 0; v < n; v++) visited[v] = 0;

    visited[start] = 1; queue[rear++] = start;
    while (front < rear) {
        v = queue[front++];          /* dequeue */
        printf("%d ", v);
        for (w = 0; w < n; w++)
            if (a[v][w] == 1 && !visited[w]) {
                visited[w] = 1;
                queue[rear++] = w;    /* enqueue */
            }
    }
}
```

Example on the graph A:B,C | B:A,D,E | C:A,F | D:B | E:B,F | F:C,E, starting at A:

Step	Dequeued / visited	Newly enqueued	Queue after step	Output
0	—	A	A	—
1	A	B, C	B C	A
2	B	D, E	C D E	A B
3	C	F	D E F	A B C
4	D	—	E F	A B C D
5	E	—	F	A B C D E
6	F	—	empty	A B C D E F

BFS order: A B C D E F — level 0 is {A}, level 1 is {B, C}, level 2 is {D, E, F}.

Uses: shortest path in an unweighted graph (BFS gives the minimum number of edges from the source), testing connectivity and bipartiteness, finding all nodes within one "hop" distance (web crawlers, social-network degrees of separation), and broadcast routing.

Time complexity $O(V + E)$ with adjacency lists, $O(V^2)$ with a matrix; space complexity $O(V)$ for the queue and the visited array.

Q10. What is Breadth First Search? How does it differ from Depth First Search? [10 marks · D22]

BFS is a graph traversal that explores the graph level by level: it visits the source, then every vertex adjacent to the source, then every vertex at distance two, and so on, using a queue to hold the frontier and a visited array to avoid repetition.

Breadth First Search	Depth First Search
Auxiliary structure is a queue (FIFO)	Auxiliary structure is a stack (LIFO) or recursion
Explores level by level (nearest vertices first)	Explores one branch to its full depth, then backtracks
Finds the shortest path (fewest edges) from the source in an unweighted graph	Does not give shortest paths
Space $O(V)$ — the queue may hold a whole level, i.e. up to $O(b^d)$ in a wide graph	Space $O(h)$ — the stack holds only the current path, usually much less
Better when the target is near the source	Better when the target is deep, or for exhaustive exploration
Produces a BFS tree with cross edges	Produces a DFS tree with back edges — hence used for cycle detection
Applications: shortest path in unweighted graphs, bipartiteness test, peer-to-peer/network broadcast, web crawling	Applications: topological sort, strongly connected components, articulation points, maze solving, cycle detection
Time $O(V + E)$	Time $O(V + E)$

Same graph, both traversals. A:B,C | B:A,D,E | C:A,F | D:B | E:B,F | F:C,E, source A.

- BFS: **A B C D E F**
- DFS: **A B D E F C**

Q11. What is Depth First Search? How does it differ from Breadth First Search? [10 marks · J23]

DFS is a graph traversal that goes as deep as possible before backtracking: from the current vertex it recursively visits an unvisited neighbour; when none is left it returns to the previous vertex and tries its next unvisited neighbour. It is implemented with recursion or an explicit stack together with a `visited[]` array.

```
void dfs(int v) {
    visited[v] = 1; printf("%d ", v);
    for (w = 0; w < n; w++) if (a[v][w] && !visited[w]) dfs(w);
}
```

Depth First Search	Breadth First Search
Uses a stack / recursion (LIFO)	Uses a queue (FIFO)
Goes deep first, backtracks on a dead end	Goes wide first, level by level
Space $O(h)$, only the current path	Space $O(V)$, a whole level of the frontier
Can get trapped in a long branch; may not find the nearest solution first	Always reaches the closest vertices first, giving unweighted shortest paths
Edge classification: tree and back edges → cycle detection	Edge classification: tree and cross edges
Applications: topological sorting, SCCs, articulation points and bridges, cycle detection, puzzle/maze solving	Applications: shortest path in unweighted graphs, bipartiteness, broadcast, social-network degree queries
Time $O(V + E)$	Time $O(V + E)$

Example — graph A:B,C | B:A,D,E | C:A,F | D:B | E:B,F | F:C,E from A. DFS = **A B D E F C**; BFS = **A B C D E F**. The two agree only on the source and the first neighbour.

Q12. What is Depth First Search (DFS)? How does it differ from Breadth First Search (BFS)? Give examples for DFS and BFS. [10 marks · J25]

DFS is a systematic traversal of a graph that visits a vertex, then pushes deeper into one unvisited neighbour at a time, backtracking only when the current vertex has no unvisited neighbour left. The backtracking is handled by a stack or the recursion stack, and a visited array stops the traversal from looping on cycles.

DFS	BFS
Stack / recursion	Queue
Depth-wise, with backtracking	Level-wise (breadth-wise)
$O(h)$ auxiliary space (h = longest path)	$O(V)$ auxiliary space (widest level)
No shortest-path guarantee	Shortest path by edge count in an unweighted graph
Back edges reveal cycles	Cross edges; cycles found by level comparison
Topological sort, SCC, articulation points, mazes	Shortest paths, bipartite test, broadcast, crawling
$O(V + E)$ time	$O(V + E)$ time

Common example graph.

```

A --- B --- D
|       |
|       E --- F
C -----|
A:B,C   B:A,D,E   C:A,F   D:B   E:B,F   F:C,E

```

DFS from A. Visit $A \rightarrow B$ (first unvisited neighbour) $\rightarrow D$ (dead end, backtrack to B) $\rightarrow E \rightarrow F \rightarrow C$ (dead end, backtrack out). DFS order: **A, B, D, E, F, C**. Stack high-water mark: A B E F C.

BFS from A.

Dequeue	Enqueue	Queue	Output
A	B, C	B C	A
B	D, E	C D E	A B
C	F	D E F	A B C
D	—	E F	A B C D
E	—	F	A B C D E
F	—	—	A B C D E F

BFS order: **A, B, C, D, E, F**; levels $\{A\} / \{B, C\} / \{D, E, F\}$.

Q13. Write algorithm for Depth First Search. [5 marks · J26]

```

int visited[MAX];                /* 0 = not yet visited */

/* recursive version, adjacency matrix a[][], n vertices */
void dfs(int v, int n, int a[][MAX])
{
    int w;
    visited[v] = 1;
    printf("%d ", v);
    for (w = 0; w < n; w++)
        if (a[v][w] != 0 && visited[w] == 0)
            dfs(w, n, a);
}

/* iterative version with an explicit stack */
void dfs_iter(int start, int n, int a[][MAX])
{
    int stack[MAX], top = -1, v, w;
    for (v = 0; v < n; v++) visited[v] = 0;
    stack[++top] = start;
    while (top != -1) {
        v = stack[top--];
        if (visited[v]) continue;
        visited[v] = 1; printf("%d ", v);
        for (w = n - 1; w >= 0; w--)          /* push so lowest pops first */
            if (a[v][w] != 0 && !visited[w]) stack[++top] = w;
    }
}

void dfs_all(int n, int a[][MAX])          /* covers a disconnected graph */
{ int v; for (v = 0; v < n; v++) visited[v] = 0;
  for (v = 0; v < n; v++) if (!visited[v]) dfs(v, n, a); }

```

Time complexity $O(V + E)$ with adjacency lists, $O(V^2)$ with an adjacency matrix; space complexity $O(V)$.

Q14. Write Dijkstra's algorithm. [10 marks · D22]

Dijkstra's algorithm finds the shortest paths from one source vertex to every other vertex of a weighted graph with **non-negative** edge weights, by greedy relaxation: repeatedly finalise the unvisited vertex with the smallest tentative distance, then relax all edges leaving it.

```

void dijkstra(int n, int cost[][MAX], int source)
{
    int dist[MAX], pred[MAX], visited[MAX];
    int i, j, k, min;

    for (i = 0; i < n; i++) {
        dist[i] = cost[source][i];    /* INF where no edge exists */
        pred[i] = source;
        visited[i] = 0;
    }
    dist[source] = 0;  visited[source] = 1;

    for (k = 1; k < n; k++) {
        min = INF;  j = -1;
        for (i = 0; i < n; i++)          /* pick the nearest unvisited */
            if (!visited[i] && dist[i] < min) { min = dist[i]; j = i; }
        if (j == -1) break;              /* remaining vertices unreachable */
        visited[j] = 1;

        for (i = 0; i < n; i++)          /* relax edges out of j */
            if (!visited[i] && dist[j] + cost[j][i] < dist[i]) {
                dist[i] = dist[j] + cost[j][i];
                pred[i] = j;
            }
    }
    for (i = 0; i < n; i++)
        printf("shortest distance to %d = %d via %d\n", i, dist[i], pred[i]);
}

```

Assumptions: all weights ≥ 0 (the algorithm is incorrect for negative weights, because a finalised vertex could still be improved later); `cost[i][j] = INF` when the edge is absent; `pred[]` allows the path itself to be printed by walking back to the source.

Time complexity $O(V^2)$ with this matrix implementation, $O((V + E) \log V)$ with a binary min-heap. Space complexity $O(V)$ besides the graph.

Q15. Write Dijkstra's algorithm to find the single source shortest path in a graph. [10 marks · J24]

```

/* Single source shortest path; weights must be non-negative */
void dijkstra(int n, int cost[][MAX], int s)
{
    int dist[MAX], pred[MAX], S[MAX], i, u, v, min, k;

    for (v = 0; v < n; v++) { dist[v] = INF; pred[v] = -1; S[v] = 0; }
    dist[s] = 0;

    for (k = 0; k < n; k++) {
        min = INF; u = -1;
        for (v = 0; v < n; v++) if (!S[v] && dist[v] < min) { min = dist[v]; u = v; }
        if (u == -1) break;
        S[u] = 1;                                /* dist[u] is now final */
        for (v = 0; v < n; v++)
            if (!S[v] && cost[u][v] != INF && dist[u] + cost[u][v] < dist[v]) {
                dist[v] = dist[u] + cost[u][v];    /* relaxation */
                pred[v] = u;
            }
    }
}

```

Worked example — source A on the weighted graph A-B 2, A-D 6, B-C 3, B-D 8, B-E 5, C-E 7, D-E 9.

Iteration	Vertex finalised	dist[A]	dist[B]	dist[C]	dist[D]	dist[E]
init	—	0	∞	∞	∞	∞
1	A (0)	0	2	∞	6	∞
2	B (2)	0	2	5	6	7
3	C (5)	0	2	5	6	7
4	D (6)	0	2	5	6	7
5	E (7)	0	2	5	6	7

In iteration 2, D was not improved ($2 + 8 = 10 > 6$); in iterations 3 and 4, E was not improved ($5 + 7 = 12$ and $6 + 9 = 15$, both > 7).

Predecessors: $\text{pred}[B] = A$, $\text{pred}[C] = B$, $\text{pred}[D] = A$, $\text{pred}[E] = B$. Shortest paths: $A \rightarrow B = 2$, $A \rightarrow B \rightarrow C = 5$, $A \rightarrow D = 6$, $A \rightarrow B \rightarrow E = 7$.

Time complexity $O(V^2)$, or $O((V + E) \log V)$ with a heap; space $O(V)$.

Q16. Describe Dijkstra's algorithm for finding the shortest path in a weighted graph. How does it manage graph with negative edge costs? Explain with the help of an example. [10 marks · D24]

Description. Dijkstra's algorithm solves the single-source shortest path problem on a weighted graph with non-negative weights. It maintains $\text{dist}[v]$, the length of the best path to v found so far, a predecessor array, and a set S of vertices whose distance is final. Initially $\text{dist}[\text{source}] = 0$ and all others ∞ . Repeatedly: choose the vertex u outside S with the smallest dist , add it to S , and **relax** every edge (u, v) — if $\text{dist}[u] + w(u, v) < \text{dist}[v]$, update $\text{dist}[v]$ and $\text{pred}[v] = u$. After V iterations every dist is final.

```

dist[s] = 0; for all v != s dist[v] = INF; S = empty;
while (S != V) {
    u = vertex not in S with minimum dist[u];
    S = S union {u};
    for each v adjacent to u and not in S
        if (dist[u] + w(u,v) < dist[v]) { dist[v] = dist[u] + w(u,v); pred[v] = u; }
}

```

Negative edge costs. Dijkstra's algorithm **does not work** with negative edge weights. Its correctness rests on the assumption that once the nearest unvisited vertex is selected, no shorter route to it can exist — true only if adding more edges can never reduce a path length, i.e. only if all weights are ≥ 0 . With a negative edge, a vertex already finalised may later be reachable more cheaply, and the algorithm never reconsiders it. With a negative **cycle** no shortest path even exists, since the cycle can be traversed repeatedly.

Counter-example. Vertices A, B, C with $A \rightarrow B = 4$, $A \rightarrow C = 5$, $C \rightarrow B = -3$.

Iteration	Finalised	dist[A]	dist[B]	dist[C]
init	—	0	∞	∞
1	A	0	4	5
2	B (4, wrongly final)	0	4	5
3	C	0	4	5

Dijkstra reports $\text{dist}[B] = 4$, but the true shortest path is $A \rightarrow C \rightarrow B = 5 + (-3) = \mathbf{2}$. B was finalised in iteration 2 and never revisited.

Remedy. Use the **Bellman-Ford** algorithm, which relaxes every edge $V - 1$ times, costs $O(V \cdot E)$, handles negative edges and detects a negative cycle by a V th pass that still improves some distance; for all-pairs with negative edges use Floyd-Warshall, or Johnson's algorithm (reweighting followed by Dijkstra).

Q17. Write all pairs shortest paths algorithm. [10 marks · D23]

The all-pairs shortest path problem asks for the shortest distance between every ordered pair of vertices. **Floyd-Warshall** solves it by dynamic programming: consider the vertices $0, 1, \dots, n-1$ one at a time as a permitted intermediate vertex k , and keep the best distance using only $\{0 \dots k\}$ as intermediates.

Recurrence: $D_k[i][j] = \min(D_{(k-1)}[i][j], D_{(k-1)}[i][k] + D_{(k-1)}[k][j])$.

```
void floyd(int n, int cost[][MAX], int D[][MAX], int path[][MAX])
{
    int i, j, k;

    for (i = 0; i < n; i++)                /* initialise */
        for (j = 0; j < n; j++) {
            D[i][j] = (i == j) ? 0 : cost[i][j];    /* INF if no edge */
            path[i][j] = (cost[i][j] != INF && i != j) ? i : -1;
        }

    for (k = 0; k < n; k++)                /* intermediate vertex */
        for (i = 0; i < n; i++)            /* source */
            for (j = 0; j < n; j++)        /* destination */
                if (D[i][k] != INF && D[k][j] != INF &&
                    D[i][k] + D[k][j] < D[i][j]) {
                    D[i][j] = D[i][k] + D[k][j];
                    path[i][j] = path[k][j];
                }

    for (i = 0; i < n; i++)
        if (D[i][i] < 0) { printf("negative cycle detected\n"); return; }
}
```

Example — directed graph on $\{0,1,2\}$ with $0 \rightarrow 1 = 4$, $1 \rightarrow 2 = 3$, $0 \rightarrow 2 = 10$.

k	D[0][2]
init	10
k = 1 (via vertex 1)	$\min(10, 4 + 3) = \mathbf{7}$
k = 2	7

Properties: it works with negative edge weights (unlike Dijkstra), detects a negative cycle when any `D[i][i]` becomes negative, and `path[][]` reconstructs the actual routes.

Time complexity $\Theta(V^3)$; space complexity $\Theta(V^2)$.

Q18. What is graph? Write adjacency matrix representation of the following graph: [5 marks · D25]

[The graph is a figure in the source paper and did not survive extraction. Assumption: the 5-vertex weighted undirected graph below is used — A-B 2, A-D 6, B-C 3, B-D 8, B-E 5, C-E 7, D-E 9.]

A **graph** $G = (V, E)$ is a non-linear data structure consisting of a finite set of vertices V and a set of edges E , each edge being a pair of vertices. It is undirected if the pairs are unordered, directed (a digraph) if ordered, and weighted if each edge carries a cost.

Adjacency matrix. For $|V| = n$, an $n \times n$ matrix A where $A[i][j]$ = the weight of edge (i, j) if the edge exists, and 0 (or ∞ for shortest-path work) otherwise. For an undirected graph the matrix is symmetric, and the sum of row i gives the weighted degree of vertex i .

Graph: A --2-- B --3-- C
 | / | |
 6 8 5 7
 | / | |
 D -----+----- E
 (D-E = 9)

	A	B	C	D	E
A	0	2	0	6	0
B	2	0	3	8	5
C	0	3	0	0	7
D	6	8	0	0	9
E	0	5	7	9	0

The matrix is symmetric about the main diagonal and the diagonal is 0 because there are no self-loops. Space is $\Theta(V^2)$ irrespective of the number of edges, so it suits dense graphs; checking whether an edge exists is $O(1)$, but listing all neighbours of a vertex costs $O(V)$.

Q19. What is graph? Explain adjacency list representation of a graph with an example.

[5 marks · J26]

A graph $G = (V, E)$ is a collection of vertices together with edges connecting pairs of them; it may be directed or undirected, weighted or unweighted.

Adjacency list representation. An array (or vector) of $|V|$ header entries, one per vertex; the header for vertex i points to a singly linked list of the vertices adjacent to i , each list node optionally carrying the edge weight. In an undirected graph each edge (u, v) appears twice — once in u 's list, once in v 's.

```
struct adjnode { int vertex; int weight; struct adjnode *next; };
struct adjnode *head[MAX];          /* head[i] = adjacency list of vertex i */
```

Example — the undirected weighted graph A-B 2, A-D 6, B-C 3, B-D 8, B-E 5, C-E 7, D-E 9:

```

A -> B(2) -> D(6) -> NULL
B -> A(2) -> C(3) -> D(8) -> E(5) -> NULL
C -> B(3) -> E(7) -> NULL
D -> A(6) -> B(8) -> E(9) -> NULL
E -> B(5) -> C(7) -> D(9) -> NULL

```

Comparison with the adjacency matrix. Space is $O(V + 2E)$ instead of $O(V^2)$, so the list wins on sparse graphs; listing all neighbours of v costs $O(\deg v)$ instead of $O(V)$, which is why BFS and DFS are $O(V + E)$ with lists and $O(V^2)$ with a matrix. The drawback is that testing whether a particular edge (u, v) exists costs $O(\deg u)$ rather than $O(1)$.

Stacks and Expression Conversion

Q1. Convert the following expression to postfix: $(a + b) / (c - d)$ [10 marks · D22]

Precedence and associativity used throughout: $^$ (3, right-associative) $>$ $*$ / (2, left-associative) $>$ $+$ $-$ (1, left-associative). $($ is pushed unconditionally and has the lowest in-stack precedence.

Rules. Scan the infix string left to right. An operand goes straight to the output. $($ is pushed. $)$ pops and outputs operators until the matching $($, which is discarded. For an operator, pop and output every stack operator whose precedence is greater than or equal to it (for a left-associative operator), then push it. At the end, pop and output the whole stack.

Symbol	Operator stack (bottom → top)	Postfix output
$($	$($	
a	$($	a
$+$	$(+$	a
b	$(+$	$a b$
$)$	(empty)	$a b +$
$/$	$/$	$a b +$
$($	$/ ($	$a b +$
c	$/ ($	$a b + c$
$-$	$/ (-$	$a b + c$
d	$/ (-$	$a b + c d$
$)$	$/$	$a b + c d -$
end of input	(empty)	$a b + c d - /$

Postfix expression: $ab+cd- /$

For completeness, the prefix form of the same expression is $/+ab-cd$. The conversion makes one pass over n symbols, each pushed and popped at most once, so it is $O(n)$ in time and $O(n)$ in stack space.

Q2. Convert the following expression to postfix: $(a - b)(c + d)$ [10 marks · J23]

[Assumption: the juxtaposition of the two bracketed groups denotes multiplication, so the expression read is $(a - b) * (c + d)$.]

Precedence: $*$ / (2) above $+$ - (1), both left-associative.

Symbol	Operator stack	Postfix output
(	(	
a	(	a
-	(-	a
b	(-	a b
)	(empty)	a b -
*	*	a b -
(	* (	a b -
c	* (	a b - c
+	* (+	a b - c
d	* (+	a b - c d
)	*	a b - c d +
end	(empty)	a b - c d + *

Postfix expression: $ab-cd+*$

Check by evaluating with $a = 7, b = 3, c = 2, d = 4$: push 7, 3; $- \rightarrow 4$; push 2, 4; $+$ $\rightarrow 6$; $*$ $\rightarrow 24$. The infix value $(7-3)(2+4) = 4 \times 6 = 24$ agrees.

Prefix form: $*-ab+cd$. Time $O(n)$, space $O(n)$.

Q3. Convert the following expression to postfix: $a + (b * c + d) / e$ [10 marks · D23]

Symbol	Operator stack	Postfix output
a	(empty)	a
+	+	a
(	+ (	a
b	+ (	a b
*	+ (*	a b
c	+ (*	a b c
+	+ (+	a b c *
d	+ (+	a b c * d
)	+	a b c * d +
/	+ /	a b c * d +
e	+ /	a b c * d + e
end	(empty)	a b c * d + e / +

Notes on the two decision points: at the inner `+`, the stack top `*` has higher precedence so `*` is popped and output, then the scan stops at `(` and the `+` is pushed. At `/`, the stack top is `+` whose precedence (1) is lower than `/` (2), so nothing is popped and `/` is pushed.

Postfix expression: `abc*d+e/+`

Verification with $a = 1, b = 2, c = 3, d = 4, e = 5$: $2 \cdot 3 = 6, 6 + 4 = 10, 10 / 5 = 2, 1 + 2 = 3$; the infix expression gives $1 + (2 \cdot 3 + 4) / 5 = 1 + 2 = 3$.

Prefix form: `+a/+*bcde`. Time $O(n)$, space $O(n)$.

Q4. Convert the following expression to postfix: $a - b * c - d / e * f$ [10 marks · J24]

[Assumption: the two minus signs were lost by the PDF text layer and the expression is read as `a - b * c - d / e * f`, i.e. $((a - (bc)) - ((d/e)f))$, with `*` and `/` of equal higher precedence and left associativity.]

Symbol	Operator stack	Postfix output
a	(empty)	a
-	-	a
b	-	a b
*	- *	a b
c	- *	a b c
-	-	a b c *
	(pop *, prec 2 > 1; then pop -, equal prec and left-assoc; push -)	a b c * -
d	-	a b c * - d
/	- /	a b c * - d
e	- /	a b c * - d e
*	- *	a b c * - d e /
	(pop /, equal precedence, left-associative; push *)	
f	- *	a b c * - d e / f
end	(empty)	a b c * - d e / f * -

Postfix expression: `abc*-de/f*-`

Verification with $a = 20$, $b = 2$, $c = 3$, $d = 8$, $e = 4$, $f = 5$: $23 = 6$; $20 - 6 = 14$; $8/4 = 2$; $25 = 10$; $14 - 10 = 4$. Infix: $20 - 6 - 10 = 4$. Agrees.

Prefix form: `--a*bc*/def`. Time $O(n)$, space $O(n)$.

Q5. Convert the following infix expression into postfix and prefix expressions: $(a * (b + c)) / (d - e)$ [10 marks · D24]

Postfix conversion.

Symbol	Operator stack	Postfix output
(	(	
a	(	a
*	(*	a
(	(* (	a
b	(* (	a b
+	(* (+	a b
c	(* (+	a b c
)	(*	a b c +
)	(empty)	a b c + *
/	/	a b c + *
(	/ (	a b c + *
d	/ (	a b c + * d
-	/ (-	a b c + * d
e	/ (-	a b c + * d e
)	/	a b c + * d e -
end	(empty)	a b c + * d e - /

Postfix: `abc+*de-/`

Prefix conversion — reverse the infix string, swapping (with) , convert to postfix, then reverse the result. Reversed input:) e - d (/)) c + b (* a (.

Symbol	Operator stack	Output
((was))	(	
e	(	e
-	(-	e
d	(-	e d
)	(empty)	e d -
/	/	e d -
(	/(	e d -
(	/((	e d -
c	/((	e d - c
+	/((+	e d - c
b	/((+	e d - c b
)	/(	e d - c b +
*	/(*	e d - c b +
a	/(*	e d - c b + a
)	/	e d - c b + a *
end	(empty)	e d - c b + a */

Reversing ed-cb+a*/ gives **Prefix: /*a+bc-de**

Expression tree check: root / , left subtree * with children a and (+ b c), right subtree - with children d and e. Preorder of this tree = / * a + b c - d e and postorder = a b c + * d e - / , matching both answers. Time $O(n)$, space $O(n)$.

Q6. Convert the following expressions to postfix: (i) $a * b \oplus c / d \oplus e$ (ii) $a \oplus b c \oplus d \oplus e$

[10 marks · J25]

[Assumption: the operators printed as $\oplus$ were lost in extraction and are taken to be +, so (i) reads $a * b + c / d + e$ and (ii) reads $a + b * c + d + e$. The method shown is independent of which additive operator is used.]

(i) $a * b + c / d + e$

Symbol	Operator stack	Postfix output
a	(empty)	a
*	*	a
b	*	a b
+	+	a b *
c	+	a b * c
/	+ /	a b * c
d	+ /	a b * c d
+	+	a b * c d / +
	(pop / ; then pop +, equal precedence, left-assoc; push +)	
e	+	a b * c d / + e
end	(empty)	a b * c d / + e +

Postfix (i): `ab*cd/+e+`

(ii) `a + b * c + d + e`

Symbol	Operator stack	Postfix output
a	(empty)	a
+	+	a
b	+	a b
*	+ *	a b
c	+ *	a b c
+	+	a b c * +
d	+	a b c * + d
+	+	a b c * + d +
e	+	a b c * + d + e
end	(empty)	a b c * + d + e +

Postfix (ii): `abc*+d+e+`

Both conversions are a single scan: $O(n)$ time, $O(n)$ stack space.

Q7. Convert the following infix-expression into prefix and postfix expressions:

`( a + ( b * c ) * ( d / e ) )` [10 marks · J26]

[Assumption: the first operator was lost in extraction and is taken to be `+`, so the expression is `( a + ( b * c ) * ( d / e ) )`.]

Postfix conversion.

Symbol	Operator stack	Postfix output
(	(	
a	(	a
+	(+	a
(	(+ (	a
b	(+ (	a b
*	(+ (*	a b
c	(+ (*	a b c
)	(+	a b c *
*	(+ *	a b c *
(	(+ * (	a b c *
d	(+ * (	a b c * d
/	(+ * (/	a b c * d
e	(+ * (/	a b c * d e
)	(+ *	a b c * d e /
)	(empty)	a b c * d e / * +

Postfix: `abc*de/*+`

Prefix — by the reverse-and-convert method, or directly from the expression tree: the root is +, its left operand is a, its right operand is * whose children are (* b c) and (/ d e). Preorder of that tree:

Prefix: `+a**bc/de`

Verification with $a = 1, b = 2, c = 3, d = 8, e = 4$: $(23) = 6, (8/4) = 2, 62 = 12, 1+12 = 13$. Evaluating the postfix string left to right on a stack: push 1, 2, 3 $\rightarrow$ * gives 6 $\rightarrow$ push 8, 4 $\rightarrow$ / gives 2 $\rightarrow$ * gives 12 $\rightarrow$ + gives 13. Agrees.

Time $O(n)$ for each conversion; stack space $O(n)$.

Q8. Define "Stack". Explain the operations that can be performed on a stack. [10 marks · D21]

A **stack** is a linear, ordered list in which all insertions and deletions take place at one end only, called the **top**. It therefore obeys **LIFO** (Last In, First Out): the element inserted most recently is the one removed first. It is an abstract data type characterised by its operations rather than its storage, and it can be implemented with a one-dimensional array plus an integer `top`, or with a singly linked list whose head is the `top`.

Operations.

Operation	Meaning	Condition checked
CreateStack(S)	initialise an empty stack, <code>top = -1</code>	—
push(S, x)	insert x at the top	overflow if <code>top == MAX-1</code>
pop(S)	remove and return the top element	underflow if <code>top == -1</code>
peek(S) / top(S)	return the top element without removing it	underflow if empty
isEmpty(S)	true when <code>top == -1</code>	—
isFull(S)	true when <code>top == MAX-1</code> (array version only)	—

```
#define MAX 100
int stack[MAX], top = -1;

void push(int x)
{ if (top == MAX - 1) printf("Overflow\n"); else stack[++top] = x; }

int pop(void)
{ if (top == -1) { printf("Underflow\n"); return -1; } return stack[top--]; }
```

Example on an empty stack: push(10), push(20), push(30) gives stack = 10, 20, 30 with top = 2; pop() returns 30; peek() then returns 20.

Every operation is $O(1)$ in time; the array implementation uses $O(\text{MAX})$ space. Applications: expression conversion and evaluation, function-call/activation records and recursion, undo facilities, parenthesis matching, backtracking and DFS.

Q9. Explain the operations that are performed on Stacks. Write an algorithm to push an element to the stack. [10 marks · D22]

A stack is a LIFO linear list with access restricted to one end, the top. Its operations are:

- **push(x)** — insert x at the top; fails with *overflow* when the stack is full.
- **pop()** — delete and return the top element; fails with *underflow* when the stack is empty.
- **peek()/top()** — read the top element without deleting it.
- **isEmpty()** — test `top == -1`.
- **isFull()** — test `top == MAX - 1` (array implementation only).
- **create/initialise** — set `top = -1`.

All of these are $O(1)$ because only the top is touched; no element has to be shifted, unlike in a general list.

Push algorithm.

```
#define MAX 100
int stack[MAX];
int top = -1;           /* -1 denotes an empty stack */

void push(int item)
{
    if (top == MAX - 1) { /* Step 1: overflow check */
        printf("Stack Overflow: cannot push %d\n", item);
        return;
    }
    top = top + 1;        /* Step 2: advance the top */
    stack[top] = item;    /* Step 3: store the item */
    printf("%d pushed, top = %d\n", item, top);
}
```

Assumptions: the stack holds integers, MAX is its capacity, and `top` is a global index of the topmost occupied cell.

Example — MAX = 5, starting empty:

Call	top before	top after	Array contents
push(10)	-1	0	10
push(20)	0	1	10, 20
push(30)	1	2	10, 20, 30

Time complexity $O(1)$; space complexity $O(1)$ extra ($O(\text{MAX})$ for the array itself).

Q10. What is a Stack? Write an algorithm for implementation of a Stack. [10 marks · D22]

A stack is a linear data structure in which insertion and deletion are permitted at only one end, the top, giving Last In First Out behaviour.

Array implementation — complete set of operations.

```

#define MAX 100

struct stack {
    int item[MAX];
    int top;
};

void create(struct stack *s) { s->top = -1; }
int  isEmpty(struct stack *s) { return (s->top == -1); }
int  isFull (struct stack *s) { return (s->top == MAX - 1); }

void push(struct stack *s, int x)
{
    if (isFull(s)) { printf("Overflow\n"); return; }
    s->item[++(s->top)] = x;
}

int pop(struct stack *s)
{
    if (isEmpty(s)) { printf("Underflow\n"); return -1; }
    return s->item[(s->top)--];
}

int peek(struct stack *s)
{
    if (isEmpty(s)) { printf("Stack empty\n"); return -1; }
    return s->item[s->top];
}

void display(struct stack *s)
{
    int i;
    for (i = s->top; i >= 0; i--) printf("%d ", s->item[i]);    /* top first */
}

```

Assumptions: fixed capacity MAX; `top = -1` marks the empty stack; elements are integers.

Example trace with MAX = 4:

Operation	Array	top	Result
create	—	−1	empty
push(5)	5	0	
push(9)	5, 9	1	
pop()	5	0	returns 9
peek()	5	0	returns 5
pop()	—	−1	returns 5
pop()	—	−1	Underflow

Each operation is $O(1)$ in time; space is $O(\text{MAX})$.

Q11. Write an algorithm for implementation of a Stack using Arrays. [10 marks · J24]

```
#define MAX 10
int stack[MAX];
int top = -1;           /* empty stack */

/* 1. PUSH */
void push(int x)
{
    if (top == MAX - 1) { printf("Stack Overflow\n"); return; }
    top++;
    stack[top] = x;
}

/* 2. POP */
int pop(void)
{
    int x;
    if (top == -1) { printf("Stack Underflow\n"); return -1; }
    x = stack[top];
    top--;
    return x;
}

/* 3. PEEK */
int peek(void)
{
    if (top == -1) { printf("Stack is empty\n"); return -1; }
    return stack[top];
}

/* 4. EMPTY / FULL tests */
int isEmpty(void) { return (top == -1); }
int isFull (void) { return (top == MAX - 1); }

/* 5. DISPLAY from top to bottom */
void display(void)
{
    int i;
    if (top == -1) { printf("Empty\n"); return; }
    for (i = top; i >= 0; i--) printf("%d ", stack[i]);
}
```

Assumptions: the stack is a global integer array of fixed size MAX; `top` indexes the last inserted element and is `-1` when the stack is empty; the two boundary tests `top == MAX-1` (overflow) and `top == -1` (underflow) must precede every push and pop respectively.

Trace with MAX = 3: push(2) → [2], top = 0; push(4) → [2,4], top = 1; push(6) → [2,4,6], top = 2; push(8) → **Overflow**; pop() → 6, top = 1; pop() → 4; pop() → 2, top = -1; pop() → **Underflow**.

Time complexity O(1) per operation; space complexity O(MAX), fixed at compile time.

Q12. Explain stack implementation using an array. What are the advantages and limitations of this approach? Illustrate it with the help of an example. [10 marks · J25]

Implementation. A one-dimensional array `stack[MAX]` holds the elements and an integer `top` holds the index of the topmost element, initialised to `-1` for the empty stack. `push` first tests `top == MAX - 1` (overflow), then increments `top` and stores the element. `pop` first tests `top == -1`

(underflow), then returns `stack[top]` and decrements `top`. Nothing is ever shifted, so both operations touch exactly one cell.

```
#define MAX 5
int stack[MAX], top = -1;

void push(int x) { if (top == MAX-1) printf("Overflow\n"); else stack[++top] = x; }
int pop(void)    { if (top == -1) { printf("Underflow\n"); return -1; } return stack[top--]; }
int isEmpty(void){ return top == -1; }
```

Advantages. - Simple to code; no pointers and no dynamic memory management. - $O(1)$ push, pop and peek, with a very small constant. - No per-element pointer overhead, so memory per element is minimal. - Contiguous storage gives good cache locality and allows random inspection of `stack[i]` for debugging.

Limitations. - The size `MAX` is fixed at compile time: overflow occurs even when heap memory is free, so the capacity must be guessed in advance. - Unused cells are wasted when the stack stays small. - Growing the stack requires allocating a bigger array and copying — $O(n)$. - Only one stack can occupy an array efficiently (two can, by growing from both ends).

The linked-list implementation removes the fixed limit at the cost of one pointer per node.

Example with `MAX = 5`:

Operation	Array contents (index 0 →)	top	Comment
initial	—	-1	empty
push(10)	10	0	
push(20)	10, 20	1	
push(30)	10, 20, 30	2	
pop()	10, 20	1	returns 30
push(40)	10, 20, 40	2	reuses cell 2
pop()	10, 20	1	returns 40

Q13. What is Stack? Implement stack using array. [10 marks · D25]

A stack is a linear list where insertion (push) and deletion (pop) happen only at one end called the top; the structure is therefore Last In First Out. Typical uses: evaluating and converting expressions, handling function calls and recursion, parenthesis matching, undo operations and DFS.

```

#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1;

void push(int x)
{
    if (top == MAX - 1) { printf("Stack Overflow\n"); return; }
    stack[++top] = x;
}

int pop(void)
{
    if (top == -1) { printf("Stack Underflow\n"); return -1; }
    return stack[top--];
}

int peek(void)    { return (top == -1) ? -1 : stack[top]; }
int isEmpty(void) { return (top == -1); }
int isFull (void) { return (top == MAX - 1); }

void display(void)
{
    int i;
    if (isEmpty()) { printf("Stack is empty\n"); return; }
    printf("Top -> ");
    for (i = top; i >= 0; i--) printf("%d ", stack[i]);
    printf("<- Bottom\n");
}

int main(void)
{
    push(11); push(22); push(33);
    display(); /* Top -> 33 22 11 <- Bottom */
    printf("popped %d\n", pop()); /* 33 */
    printf("top is %d\n", peek()); /* 22 */
    return 0;
}

```

Assumptions: integer elements; capacity MAX fixed at compile time; `top = -1` denotes the empty stack; overflow and underflow are reported rather than causing an invalid access.

Each of push, pop, peek, isEmpty and isFull is $O(1)$ in time; display is $O(n)$. Space complexity is $O(\text{MAX})$, independent of how many elements are actually stored — the principal drawback of the array version.

Q14. Explain stack implementation using a **linked list**. [10 marks · D24]

In the linked representation each stack element is a dynamically allocated node holding the data and a pointer to the node below it. A single pointer `top` points to the most recently inserted node, which is the head of the list; the bottom node's link is NULL. Because insertion and deletion are both at the head, both are $O(1)$ and no shifting is needed.

```

struct node {
    int info;
    struct node *link;
};
struct node *top = NULL;          /* NULL => empty stack */

void push(int x)
{
    struct node *p;
    p = (struct node *) malloc(sizeof(struct node));
    if (p == NULL) { printf("Memory exhausted\n"); return; } /* only "overflow" */
    p->info = x;
    p->link = top;          /* new node points to the old top */
    top = p;               /* new node becomes the top */
}

int pop(void)
{
    struct node *p; int x;
    if (top == NULL) { printf("Stack Underflow\n"); return -1; }
    p = top;
    x = p->info;
    top = top->link;        /* second node becomes the top */
    free(p);               /* release the old top */
    return x;
}

int peek(void) { return (top == NULL) ? -1 : top->info; }
int isEmpty(void) { return (top == NULL); }

void display(void)
{ struct node *p = top; while (p != NULL) { printf("%d ", p->info); p = p->link; } }

```

Trace. push(10): top → [10 | NULL]. push(20): top → [20 | •] → [10 | NULL]. push(30): top → [30] → [20] → [10]. pop() frees the 30-node, returns 30 and leaves top → [20] → [10].

Comparison with the array version. There is no fixed capacity, so the stack grows until the heap is exhausted — there is no conventional overflow test; memory is allocated only for the elements actually present, so nothing is wasted. The costs are one extra pointer per element, the run-time cost of `malloc/free`, loss of cache locality, and no $O(1)$ random access.

Time complexity $O(1)$ for push, pop, peek and isEmpty; $O(n)$ for display. Space complexity $O(n)$ with 1 pointer of overhead per element.

Q15. What is stack? Write algorithm to check whether stack is empty or not. [5 marks · J26]

A stack is a linear data structure in which both insertion (push) and deletion (pop) are restricted to one end called the top, making it Last In First Out. The emptiness test must be executed before every pop or peek, since popping an empty stack is an underflow error.

```

/* Array implementation: top == -1 means empty */
#define MAX 100
int stack[MAX];
int top = -1;

int isEmpty(void)
{
    if (top == -1)
        return 1;          /* stack is empty */
    else
        return 0;          /* stack is not empty */
}

/* Linked-list implementation: a NULL top means empty */
struct node { int info; struct node *link; };
struct node *topptr = NULL;

int isEmpty_ll(void)
{
    return (topptr == NULL) ? 1 : 0;
}

```

Example: on a freshly initialised stack `top = -1`, so `isEmpty()` returns 1; after `push(7)`, `top = 0` and `isEmpty()` returns 0; after `pop()`, `top` is back to `-1` and it returns 1 again.

Time complexity $O(1)$ — a single comparison; space complexity $O(1)$.

Q16. Explain the process of implementing two stacks in a single dimensional array.

[10 marks · D23]

If two stacks are given half of an array each, one may overflow while the other is nearly empty. The efficient scheme lets the two stacks **grow towards each other from the two ends** of one array of size MAX, so overflow occurs only when the array as a whole is full.

Layout. Stack 1 occupies the low end and grows upward from index 0: `top1` starts at `-1` and is incremented on push. Stack 2 occupies the high end and grows downward from index `MAX-1`: `top2` starts at `MAX` and is decremented on push.

index:	0	1	2	3	4	5	6	7	8	9
	[A	B	C				Z	Y	X	]
	^		^					^		
			top1 = 2					top2 = 7		
	stack1 grows -->						<-- stack2 grows			

Conditions. The array is full — and *both* stacks are blocked — when `top1 + 1 == top2`. Stack 1 is empty when `top1 == -1`; stack 2 is empty when `top2 == MAX`.

```

#define MAX 10
int a[MAX];
int top1 = -1, top2 = MAX;

void push1(int x)
{ if (top1 + 1 == top2) printf("Overflow\n"); else a[++top1] = x; }

void push2(int x)
{ if (top1 + 1 == top2) printf("Overflow\n"); else a[--top2] = x; }

int pop1(void)
{ if (top1 == -1) { printf("Stack1 Underflow\n"); return -1; } return a[top1--]; }

int pop2(void)
{ if (top2 == MAX) { printf("Stack2 Underflow\n"); return -1; } return a[top2++]; }

```

Example with MAX = 6: push1(1), push1(2) → a = [1,2,,,], top1 = 1. push2(9), push2(8) → a = [1,2,,,8,9], top2 = 4. push1(3) → top1 = 2, and now top1+1 = 3 ≠ top2 = 4, so one more push is still possible; after push1(4), top1 = 3 and top1+1 == top2, so any further push on either stack reports overflow.

Advantage. The whole array is shared, so space is used fully: stack 1 may hold MAX elements if stack 2 is empty. All operations remain O(1); space O(MAX). The same trick does **not** extend naturally to three or more stacks, which need either fixed partitions with periodic shifting or linked lists.

Arrays, Structures, Matrices and Strings

Q1. Write an algorithm for addition of two matrices. [10 marks · D21]

Two matrices can be added only if they have identical dimensions; the sum is formed element by element: $C[i][j] = A[i][j] + B[i][j]$.

```

#define MAX 10

void matadd(int A[][MAX], int B[][MAX], int C[][MAX],
            int m1, int n1, int m2, int n2)
{
    int i, j;

    if (m1 != m2 || n1 != n2) { /* compatibility check */
        printf("Addition not possible: orders differ\n");
        return;
    }
    for (i = 0; i < m1; i++)
        for (j = 0; j < n1; j++)
            C[i][j] = A[i][j] + B[i][j];
}

void display(int C[][MAX], int m, int n)
{
    int i, j;
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) printf("%4d", C[i][j]);
        printf("\n");
    }
}

```

Assumptions: both matrices are of order $m \times n$, elements are integers, MAX bounds the declared size.

Example:

```
A = | 1  2 |      B = | 5  6 |      C = |  6   8 |
    | 3  4 |      | 7  8 |      | 10  12 |
```

$C[0][0] = 1+5 = 6$, $C[0][1] = 2+6 = 8$, $C[1][0] = 3+7 = 10$, $C[1][1] = 4+8 = 12$.

Time complexity $O(m \cdot n)$ — the inner statement executes exactly $m \cdot n$ times, which is optimal since every element must be read. Space complexity $O(m \cdot n)$ for the result matrix, $O(1)$ extra.

Q2. What are structures? Write an algorithm for addition of two matrices. [10 marks · J23]

A **structure** is a user-defined data type that groups a fixed number of logically related data items, possibly of different types, under one name. Each item is a **member** (or field), accessed by the dot operator on a structure variable or by `->` through a pointer. Unlike an array, whose elements are all of the same type and are selected by an index, a structure's members may be heterogeneous and are selected by name. Structures may be nested, passed to and returned from functions, and arrays of structures are the standard way to hold records. They are the building block of linked lists, trees and graphs, where a structure holds the data together with the pointers.

```
struct student {
    int    roll;
    char   name[30];
    float  marks;
};
struct student s1;          /* s1.roll, s1.name, s1.marks */
```

A matrix can itself be held in a structure that carries its order along with its elements:

```
#define MAX 10
struct matrix { int m, n; int a[MAX][MAX]; };

int matadd(struct matrix *A, struct matrix *B, struct matrix *C)
{
    int i, j;
    if (A->m != B->m || A->n != B->n)
        return 0;          /* orders incompatible */
    C->m = A->m; C->n = A->n;
    for (i = 0; i < A->m; i++)
        for (j = 0; j < A->n; j++)
            C->a[i][j] = A->a[i][j] + B->a[i][j];
    return 1;
}
```

Example with $A = [[2,3],[4,5]]$ and $B = [[1,1],[2,2]]$: $C = [[3,4],[6,7]]$.

Time complexity $O(m \cdot n)$; space complexity $O(m \cdot n)$ for the result.

Q3. What are arrays? Write an algorithm for addition of two matrices. [10 marks · J25]

An **array** is a finite ordered collection of homogeneous elements stored in contiguous memory locations and referenced by a common name together with one or more subscripts. A one-dimensional array A of n elements with lower bound L and element size w has $\text{address}(A[i]) = \text{base} + w \cdot (i - L)$, so any element is reached in $O(1)$ time — random access is the

defining property. The size is fixed at declaration (static allocation), insertion and deletion in the middle require shifting $O(n)$ elements, and a two-dimensional array (a matrix) is an array of arrays, linearised in memory in either row-major or column-major order.

```
#define MAX 10

void matrix_add(int A[][MAX], int B[][MAX], int C[][MAX], int m, int n,
               int p, int q)
{
    int i, j;
    if (m != p || n != q) { printf("Orders must be identical\n"); return; }

    for (i = 0; i < m; i++)          /* for every row */
        for (j = 0; j < n; j++)      /* for every column */
            C[i][j] = A[i][j] + B[i][j];
}
```

Worked example:

```
A = | 1  2  3 |      B = | 9  8  7 |      C = | 10 10 10 |
    | 4  5  6 |      | 6  5  4 |      | 10 10 10 |
```

Each $C[i][j]$ is computed by exactly one addition; for a 2×3 pair that is 6 additions.

Time complexity $O(m \cdot n)$ — the loop body runs $m \cdot n$ times, and no algorithm can do better since all $m \cdot n$ results must be produced. Space complexity $O(m \cdot n)$ for C, $O(1)$ auxiliary.

Q4. Write an algorithm for multiplication of two matrices. [10 marks · asked 2×: J22, J24]

Matrices A ($m \times n$) and B ($p \times q$) can be multiplied only if $n == p$; the product C is $m \times q$ with $C[i][j] = \sum_{k=0}^{n-1} A[i][k] \cdot B[k][j]$.

```
#define MAX 10

void matmul(int A[][MAX], int B[][MAX], int C[][MAX],
            int m, int n, int p, int q)
{
    int i, j, k;

    if (n != p) {                          /* compatibility check */
        printf("Multiplication not possible: cols(A) != rows(B)\n");
        return;
    }
    for (i = 0; i < m; i++)
        for (j = 0; j < q; j++) {
            C[i][j] = 0;                    /* reset the accumulator */
            for (k = 0; k < n; k++)
                C[i][j] = C[i][j] + A[i][k] * B[k][j];
        }
}
```

Assumptions: integer elements, A is $m \times n$, B is $p \times q$, the result C is $m \times q$, and the accumulator must be cleared for every (i, j) — forgetting this is the commonest error.

Worked example:

```
A (2x3) = | 1  2  3 |      B (3x2) = | 7  8 |
          | 4  5  6 |      | 9 10 |
                          | 11 12 |
```

$C[0][0] = 1 \cdot 7 + 2 \cdot 9 + 3 \cdot 11 = 7 + 18 + 33 = 58$
 $C[0][1] = 1 \cdot 8 + 2 \cdot 10 + 3 \cdot 12 = 8 + 20 + 36 = 64$
 $C[1][0] = 4 \cdot 7 + 5 \cdot 9 + 6 \cdot 11 = 28 + 45 + 66 = 139$
 $C[1][1] = 4 \cdot 8 + 5 \cdot 10 + 6 \cdot 12 = 32 + 50 + 72 = 154$

$C \ (2 \times 2) = \begin{vmatrix} 58 & 64 \\ 139 & 154 \end{vmatrix}$

Time complexity $O(m \cdot n \cdot q)$, i.e. $O(n^3)$ for square matrices of order n — the innermost statement executes $m \cdot q \cdot n$ times. Space complexity $O(m \cdot q)$ for the product.

Q5. What are Arrays? Write an algorithm to multiply two matrices. [10 marks · D22]

An array is an ordered set of a fixed number of homogeneous elements held in consecutive memory locations, accessed by a common name and an integer subscript. Its characteristics: homogeneous elements, contiguous allocation, size fixed at declaration time, $O(1)$ random access via the address formula $\text{base} + w \cdot (i - L)$, and $O(n)$ insertion/deletion because elements have to be shifted. A two-dimensional array stores a matrix and is linearised either row by row (row-major, as in C) or column by column (column-major, as in FORTRAN).

```

#define MAX 10

void multiply(int A[][MAX], int B[][MAX], int C[][MAX],
             int r1, int c1, int r2, int c2)
{
    int i, j, k, sum;

    if (c1 != r2) { printf("Not conformable\n"); return; }

    for (i = 0; i < r1; i++)
        for (j = 0; j < c2; j++) {
            sum = 0;
            for (k = 0; k < c1; k++)
                sum += A[i][k] * B[k][j];      /* row i of A . column j of B */
            C[i][j] = sum;
        }
}

```

Worked example:

$A = \begin{vmatrix} 1 & 2 \\ 3 & 4 \end{vmatrix}$
 $B = \begin{vmatrix} 5 & 6 \\ 7 & 8 \end{vmatrix}$
 $C = \begin{vmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{vmatrix} = \begin{vmatrix} 19 & 22 \\ 43 & 50 \end{vmatrix}$

Each of the 4 elements of C needs 2 multiplications and 1 addition, so 8 multiplications in total — matching $r1 \cdot c2 \cdot c1 = 2 \cdot 2 \cdot 2$.

Time complexity $O(r1 \cdot c1 \cdot c2) = O(n^3)$ for square matrices; space complexity $O(r1 \cdot c2)$. (Strassen's method reduces the exponent to about $n^{2.81}$ but is not required here.)

Q6. What is Row Major Representation? How does it differ from Column Major Representation?

[10 marks · J23]

A two-dimensional array is logically a table but memory is one-dimensional, so the elements must be linearised. **Row major representation** stores the array row by row: the whole of row 0, then the whole of row 1, and so on — this is what C, C++, Pascal and Python use. **Column major representation** stores it column by column: the whole of column 0, then column 1 — used by FORTRAN, MATLAB and R.

For an array $A[L1..U1][L2..U2]$ with $n1 = U1 - L1 + 1$ rows, $n2 = U2 - L2 + 1$ columns, base address B and element width w :

- Row major: $\text{addr}(A[i][j]) = B + w \cdot ((i - L1) \cdot n2 + (j - L2))$
- Column major: $\text{addr}(A[i][j]) = B + w \cdot ((j - L2) \cdot n1 + (i - L1))$

Layout of a 3×3 array $A[0..2][0..2]$, $B = 1000$, $w = 2$ bytes.

Row major order	Address	Column major order	Address
$A[0][0]$	1000	$A[0][0]$	1000
$A[0][1]$	1002	$A[1][0]$	1002
$A[0][2]$	1004	$A[2][0]$	1004
$A[1][0]$	1006	$A[0][1]$	1006
$A[1][1]$	1008	$A[1][1]$	1008
$A[1][2]$	1010	$A[2][1]$	1010
$A[2][0]$	1012	$A[0][2]$	1012
$A[2][1]$	1014	$A[1][2]$	1014
$A[2][2]$	1016	$A[2][2]$	1016

Check with the formulas for $A[2][1]$: row major $1000 + 2 \cdot (2 \cdot 3 + 1) = 1000 + 14 = 1014$; column major $1000 + 2 \cdot (1 \cdot 3 + 2) = 1000 + 10 = 1010$. The two differ except on the main diagonal and for $1 \times n$ or $n \times 1$ arrays.

Row major	Column major
Rows stored contiguously	Columns stored contiguously
Needs the number of columns ($n2$) in the formula	Needs the number of rows ($n1$) in the formula
Used by C, C++, Java, Pascal, Python	Used by FORTRAN, MATLAB, R, Julia
Row-wise traversal is cache-friendly	Column-wise traversal is cache-friendly

Address computation is $O(1)$ in both schemes.

Q7. What is array? Explain row major and column major representations of array with examples.

[10 marks · D25]

An **array** is a finite, ordered collection of homogeneous elements stored in contiguous memory and identified by a single name plus subscripts. Its size is fixed at declaration, any element is reachable in $O(1)$ by arithmetic on the subscript, and insertion/deletion in the middle costs $O(n)$. A one-dimensional array uses $\text{addr}(A[i]) = B + w \cdot (i - L)$.

Because memory is linear, a two-dimensional array $A[n1][n2]$ must be flattened, and there are exactly two natural orders.

Row major. Row 0 first, then row 1, and so on. The formula, for lower bounds $L1$ and $L2$ and $n2$ columns, is

$$\text{addr}(A[i][j]) = B + w \cdot ((i - L1) \cdot n2 + (j - L2))$$

Column major. Column 0 first, then column 1, and so on, with $n1$ rows:

$$\text{addr}(A[i][j]) = B + w \cdot ((j - L2) \cdot n1 + (i - L1))$$

Example. A[3][3] with elements

```
A = | 11 12 13 |  
    | 21 22 23 |  
    | 31 32 33 |
```

B = 2000, w = 4 bytes, L1 = L2 = 0.

Row major memory image:

Address	2000	2004	2008	2012	2016	2020	2024	2028	2032
Element	11	12	13	21	22	23	31	32	33

Column major memory image:

Address	2000	2004	2008	2012	2016	2020	2024	2028	2032
Element	11	21	31	12	22	32	13	23	33

Locating A[1][2] (value 23): row major $2000 + 4 \cdot (1 \cdot 3 + 2) = 2000 + 20 = \mathbf{2020}$; column major $2000 + 4 \cdot (2 \cdot 3 + 1) = 2000 + 28 = \mathbf{2028}$. Both tables confirm the addresses.

Practical consequence: in C (row major) a loop with j innermost walks memory sequentially and is markedly faster than one with i innermost, because of cache line reuse. Address computation itself is O(1) in both representations.

Q8. Explain Sparse matrix. How are sparse matrices stored efficiently in the computer's memory? Explain with the help of an example. [10 marks · D24]

A **sparse matrix** is a matrix in which most elements are zero; a common working rule is that fewer than one third of the $m \cdot n$ elements are non-zero. Storing it as a full two-dimensional array wastes $m \cdot n \cdot w$ bytes, most of them holding zeros, and makes every operation $O(m \cdot n)$ even when there is almost nothing to compute. Diagonal, tridiagonal, triangular and band matrices, and the adjacency matrices of sparse graphs, are typical examples.

Storage method 1 — the triplet (3-column) representation. Keep only the non-zero elements as (row, column, value) triples, sorted in row-major order, preceded by a header triple holding the number of rows, the number of columns and the number of non-zero elements.

Example.

```
A = | 0 0 3 0 |  
    | 0 5 0 0 |  
    | 0 0 0 7 |  
    | 2 0 0 0 |
```

4 x 4, only 4 non-zero elements

Triplet form:

	row	col	value
header	4	4	4
1	0	2	3
2	1	1	5
3	2	3	7
4	3	0	2

Space saving. Full array: $4 \times 4 = 16$ integers. Triplet: $(4 + 1) \times 3 = 15$ integers. For a larger case — a 100×100 matrix with 50 non-zero elements — the full array needs 10 000 integers and the triplet needs $(50+1) \cdot 3 = 153$, a saving of about 98%. In general the triplet form pays off whenever the number of non-zero elements t satisfies $3(t+1) < m \cdot n$.

```
struct term { int row, col, value; };
struct term a[MAX];          /* a[0] is the header */
```

Storage method 2 — linked list representation. Each non-zero element becomes a node {row, col, value, right, down}; nodes are threaded into one circular list per row and one per column, with an array of row heads and column heads. This suits matrices whose non-zero pattern changes at run time, because inserting or deleting a non-zero element costs $O(1)$ once the position is found, whereas the triplet array needs shifting. The cost is two extra pointers per element.

Operations on the triplet form run in $O(t)$ or $O(t \log t)$ instead of $O(m \cdot n)$ — for instance transposition is just a swap of the row and column fields followed by a sort.

Q9. What is sparse matrix? Explain how sparse matrix can be stored in memory for efficient performance, using an example. [10 marks · J26]

A sparse matrix is a matrix most of whose elements are zero, so that storing all $m \cdot n$ elements is wasteful both in space and in processing time. (A matrix with few zeros is called dense.)

Efficient storage — triplet / three-column form. Store only the non-zero elements, each as a triple (row index, column index, value), in row-major order, with a header triple giving the order of the matrix and the count of non-zero elements.

Example.

A =

	0	0	0	4	
	0	9	0	0	
	6	0	0	0	
	0	0	8	1	

 5 non-zero elements out of 16

	row	col	value
header	4	4	5
1	0	3	4
2	1	1	9
3	2	0	6
4	3	2	8
5	3	3	1

Space: full array 16 integers; triplet $(5+1) \times 3 = 18$ integers — in this small example the triplet form is actually larger, which shows the break-even point: the triplet form wins only when $3(t+1) < m \cdot n$. For a 1000×1000 matrix with 2000 non-zero elements it needs 6003 integers against 1 000 000, a saving of over 99%.

Linked-list form. struct snode { int row, col, value; struct snode *right, *down; }; Each non-zero element is a node linked into a circular list for its row (via right) and for its column (via down), with header arrays of row and column pointers. Advantages: a non-zero element can be inserted or removed in $O(1)$ without shifting, and row-wise and column-wise scans are both direct. Disadvantage: two extra pointers per element.

Other schemes used in practice: compressed sparse row (CSR), where a value array, a column-index array and a row-pointer array replace the triples; and for band or tridiagonal matrices, a single one-dimensional array with an index mapping, since the position of every non-zero element is known in advance.

Operations cost $O(t)$ in the number of non-zero elements rather than $O(m \cdot n)$.

Q10. Write an algorithm to reverse a string. [10 marks · D21]

A string in C is a character array terminated by the null character `'\0'`. Reversal exchanges the first character with the last, the second with the second last, and so on, until the two indices meet; the null terminator must be left in place.

```
void reverse(char s[])
{
    int i = 0, j = 0;
    char temp;

    while (s[j] != '\0') j++;          /* find the length */
    j = j - 1;                         /* j = index of the last character */

    while (i < j) {                    /* two-pointer swap */
        temp = s[i];
        s[i] = s[j];
        s[j] = temp;
        i++;
        j--;
    }
}
```

Assumptions: `s` is a null-terminated, writable character array; the reversal is done in place, so no second array is needed; a string of length 0 or 1 is returned unchanged because the loop condition fails immediately.

Trace on "DATA" (length 4, last index 3).

Iteration	i	j	Array before swap	Array after swap
1	0	3	D A T A	A A T D
2	1	2	A A T D	A T A D
3	—	—	loop ends ($i = 2, j = 1, i < j$ false)	ATAD

Time complexity $O(n)$ — $n/2$ swaps plus n steps to find the length, so about $1.5n$ character operations, which is $\Theta(n)$. Space complexity $O(1)$: only the three scalar variables `i`, `j` and `temp`, since the reversal is in place. A recursive version would be $O(n)$ in stack space, and a version that copies into a second array would be $O(n)$ in space.

Q11. Write an algorithm that accepts two strings, `S1` and `S2`, as input and then find whether `S1` is a substring of `S2`. [10 marks · J22]

`S1` is a substring of `S2` if the characters of `S1` occur consecutively somewhere in `S2`. The naive (brute-force) method aligns `S1` against every possible starting position of `S2` and compares character by character.

```

int length(char s[])
{ int n = 0; while (s[n] != '\0') n++; return n; }

/* returns the starting index of S1 inside S2, or -1 if absent */
int substring(char S1[], char S2[])
{
    int m = length(S1);          /* pattern length */
    int n = length(S2);          /* text length */
    int i, j;

    if (m == 0) return 0;         /* the empty string is a substring of all */
    if (m > n) return -1;         /* pattern longer than the text */

    for (i = 0; i <= n - m; i++) { /* every alignment */
        j = 0;
        while (j < m && S2[i + j] == S1[j]) /* compare forward */
            j++;
        if (j == m)                /* all m characters matched */
            return i;
    }
    return -1;
}

void main_test(void)
{
    char a[] = "DATA", b[] = "STRUCTURE AND DATA";
    int p = substring(a, b);
    if (p == -1) printf("S1 is NOT a substring of S2\n");
    else printf("S1 is a substring of S2, starting at index %d\n", p);
}

```

Assumptions: both strings are null-terminated; the comparison is case-sensitive; the first (leftmost) occurrence is reported.

Trace with $S1 = \text{"ab"}$, $S2 = \text{"caab"}$ ($m = 2$, $n = 4$):

i	S2[i..i+1]	Comparison	Result
0	"ca"	'c' $\neq$ 'a'	mismatch at j = 0
1	"aa"	'a' = 'a', 'a' $\neq$ 'b'	mismatch at j = 1
2	"ab"	'a' = 'a', 'b' = 'b'	match, return 2

Time complexity $O(m \cdot n)$ in the worst case (for example $S1 = \text{"aaab"}$, $S2 = \text{"aaaaaaab"}$), $O(n)$ in the average case for random text. Space complexity $O(1)$. The Knuth–Morris–Pratt algorithm improves this to $O(m + n)$ by pre-computing a failure function, at the cost of $O(m)$ extra space.

Complexity and asymptotic notation

Q1. Define "Time Complexity" and "Space Complexity". What are the differences between them?

[10 marks · D21]

Time complexity of an algorithm is the amount of computing time it requires, expressed as a function $T(n)$ of the input size n . It is measured by counting the number of elementary operations (comparisons, assignments, arithmetic) executed, not by a stopwatch, so the measure is machine-independent.

Space complexity is the amount of memory the algorithm requires, expressed as $S(n)$. It has two parts: the *fixed part* (code, constants, simple variables) and the *variable part* (dynamically allocated memory, the recursion stack, and space whose size depends on n). Usually only the variable (auxiliary) part is quoted.

Example — summing an array of n elements:

```
sum = 0;
for (i = 0; i < n; i++)
    sum = sum + a[i];
```

The loop body executes n times, so $T(n) = c_1 \cdot n + c_2 = O(n)$. The extra variables are only `sum` and `i`, so auxiliary space $S(n) = O(1)$.

Point	Time complexity	Space complexity
Measures	Number of operations / running time	Number of memory cells used
Function of	Input size n	Input size n
Typical unit	Elementary operation count	Words or bytes of storage
Reduced by	Better algorithm, pre-computation, extra tables	Reusing memory, in-place processing
Example	Bubble sort $O(n^2)$ time	Bubble sort $O(1)$ auxiliary space
Dependence	Grows with the number of steps	Grows with data stored at one instant

The two are often traded against each other: a hash table buys $O(1)$ average lookup time by spending $O(n)$ extra space.

Q2. What is a time complexity? How does it differ from space complexity? Give an example for each to calculate time complexity and space complexity. [10 marks · J25]

Time complexity $T(n)$ is the number of elementary operations an algorithm performs on an input of size n ; space complexity $S(n)$ is the number of memory locations it needs for that input. Time is consumed repeatedly, space is occupied simultaneously, which is why a loop executed n times costs $O(n)$ time but may still cost $O(1)$ space.

Time complexity example — linear search:

```
LinearSearch(a, n, key)
    for (i = 0; i < n; i++)
        if (a[i] == key) return i;
    return -1;
```

Best case: key at `a[0]`, 1 comparison. Worst case: key absent, n comparisons. Average: $(1+2+\dots+n)/n = (n+1)/2$. All are linear in n , hence $T(n) = O(n)$.

Space complexity example — recursive factorial:

```
Fact(n)
    if (n <= 1) return 1;
    return n * Fact(n-1);
```

Each call stores one activation record (parameter n plus return address), and the recursion depth is n , so $S(n) = O(n)$. The iterative version uses a single accumulator, so $S(n) = O(1)$ — the same $O(n)$ time with less space.

Point	Time complexity	Space complexity
Resource	CPU time / step count	Memory
Components	Comparisons, assignments, arithmetic	Fixed part + variable part
Reusable	No, time once spent is gone	Yes, memory can be freed and reused
Example value	Binary search $O(\log n)$	Binary search $O(1)$ iterative, $O(\log n)$ recursive

Q3. What is a time complexity of an algorithm? What is a time-and-space trade-off? Explain with the help of an example. [10 marks · D24]

Time complexity is the function $T(n)$ giving the number of elementary operations an algorithm performs on input of size n , usually quoted asymptotically as $O(f(n))$ for the worst case.

A **time-space trade-off** is the situation in which an algorithm can be made faster by using more memory, or made to use less memory at the cost of extra time. Neither resource can usually be minimised alone; the designer chooses the point on the curve that suits the machine and the application.

Example 1 — counting character frequencies in a string of n characters:

```
for (i = 0; i < 256; i++) count[i] = 0;
for (i = 0; i < n; i++) count[ s[i] ]++;
```

This takes $O(n)$ time but spends 256 extra cells. Without the table, each distinct character would be counted by re-scanning the string, giving $O(n^2)$ time but $O(1)$ space. Extra space bought a faster algorithm.

Example 2 — a lookup table of sines versus computing each sine from the series: the table is $O(1)$ time per query with $O(m)$ space; the series is $O(1)$ space with many multiplications per query.

Example 3 — storing a sparse matrix as a full 2-D array gives $O(1)$ element access with $O(n^2)$ space; the triplet representation needs only $O(k)$ space for k non-zeros but element access becomes a search.

Other standard instances: a hash table ($O(1)$ average lookup, $O(n)$ space) versus a sorted array with binary search ($O(\log n)$ lookup, no extra space); dynamic programming, which stores a table of sub-results to turn exponential recomputation into polynomial time; and compressed files, which save storage but cost decompression time.

Q4. What is Storage Complexity? How does it differ from Time Complexity? [10 marks · D22]

Storage (space) complexity $S(n)$ is the total memory an algorithm needs to run to completion on an input of size n . It is the sum of:

1. **Fixed part** — instruction space, constants, simple variables; independent of n .
2. **Variable part** — space for dynamically created structures, arrays whose size depends on n , and the recursion stack.

So $S(n) = c + S_p(n)$, where c is fixed and $S_p(n)$ is the instance-dependent part. Only $S_p(n)$ is normally quoted as the auxiliary space complexity.

Example — adding two $n \times n$ matrices requires $2n^2$ input cells plus n^2 for the result, hence $S(n) = O(n^2)$, while the loop counters are the fixed part.

Point	Storage complexity	Time complexity
Resource measured	Memory cells occupied	Operations executed
Formula	$S(n) = \text{fixed} + \text{variable part}$	$T(n) = \text{count of elementary steps}$
Bound set by	Peak simultaneous usage	Total cumulative work
Recursion effect	One stack frame per depth level	One call's work per level
Reclaimable	Yes, after deallocation	No
Example	Merge sort $O(n)$ extra space	Merge sort $O(n \log n)$ time

Both are expressed in asymptotic notation and both are functions of n , but memory is a reusable resource whereas time is not, so designers more often accept a larger $S(n)$ to reduce $T(n)$.

Q5. Explain the process of calculation of storage complexity with an example. [10 marks · J24]

Steps to compute storage complexity:

1. List every variable and structure the algorithm declares.
2. Classify each as fixed (size independent of n) or variable (size a function of n).
3. Add the recursion stack: depth $\times$ size of one activation record.
4. Take the **maximum simultaneous** requirement, not the sum over time, since freed memory is reused.
5. Express the total as $S(n) = c + S_p(n)$ and drop constants to get the O -bound.

Example A — iterative array sum of n integers, each integer one word:

Item	Words
Array $a[n]$	n
n, i, sum	3

$S(n) = n + 3 = O(n)$; auxiliary space (excluding the input) = $O(1)$.

Example B — matrix multiplication of two $n \times n$ matrices: A, B and C need $3n^2$ words, plus i, j, k and the accumulator = 4 words. $S(n) = 3n^2 + 4 = O(n^2)$.

Example C — recursive factorial of n : one frame per call, depth n , each frame holding the parameter and return address (2 words). $S(n) = 2n + c = O(n)$. The iterative version needs $O(1)$, which is why recursion is charged for stack space even though it declares no array.

Example D — merge sort of n elements: the auxiliary merge array is n words and the recursion depth is $\log_2 n$ frames, so $S(n) = O(n)$ — the array dominates the stack.

Q6. What is meant by Big-O notation? Explain with an example. [10 marks · J22]

Big-O gives an **asymptotic upper bound** on the growth of a function. Formally:

$f(n) = O(g(n))$ if there exist positive constants c and n_0 such that

$$0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0.$$

It states that beyond some input size, f grows no faster than a constant multiple of g . It therefore describes the *worst case* ceiling and ignores constant factors and lower-order terms, which are machine-dependent and irrelevant for large n .

Example 1: $f(n) = 3n^2 + 5n + 7$. Take $g(n) = n^2$, $c = 15$, $n_0 = 1$. Then $3n^2 + 5n + 7 \leq 3n^2 + 5n^2 + 7n^2 = 15n^2$ for all $n \geq 1$, so $f(n) = O(n^2)$.

Example 2: bubble sort on n elements makes $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$ comparisons $= (n^2 - n)/2 \leq n^2$ for $n \geq 1$, hence $O(n^2)$.

Example 3: binary search halves the interval each step; after k steps the interval length is $n/2^k$, and the search ends when $n/2^k = 1$, so $k = \log_2 n$ and the complexity is $O(\log n)$.

Properties: $O(c \cdot f) = O(f)$; $O(f + g) = O(\max(f, g))$; $O(f) \cdot O(g) = O(f \cdot g)$. Common order of growth: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$. Note that O is only an upper bound, so an $O(n^2)$ algorithm is also correctly $O(n^3)$; the convention is to quote the tightest bound known.

Q7. Explain various asymptotic notations for analysis of algorithms. [10 marks · J23]

Asymptotic notations describe the growth rate of an algorithm's cost function for large n , suppressing constants and machine details.

Big-O (upper bound). $f(n) = O(g(n))$ if $\exists c > 0, n_0 > 0$ with $0 \leq f(n) \leq c \cdot g(n) \forall n \geq n_0$. Example: $3n^2 + 5n + 7 = O(n^2)$ with $c = 15, n_0 = 1$. Worst case of insertion sort $= O(n^2)$.

Big-Omega (lower bound). $f(n) = \Omega(g(n))$ if $\exists c > 0, n_0 > 0$ with $0 \leq c \cdot g(n) \leq f(n) \forall n \geq n_0$. Example: $3n^2 + 5n + 7 = \Omega(n^2)$ with $c = 3, n_0 = 1$. Any comparison sort is $\Omega(n \log n)$.

Big-Theta (tight bound). $f(n) = \Theta(g(n))$ if $\exists c_1, c_2 > 0, n_0 > 0$ with $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \forall n \geq n_0$; equivalently $f = O(g)$ and $f = \Omega(g)$. Example: $3n^2 + 5n + 7 = \Theta(n^2)$ with $c_1 = 3, c_2 = 15, n_0 = 1$. Merge sort $= \Theta(n \log n)$ in every case.

Little-o (strict upper bound). $f(n) = o(g(n))$ if for every $c > 0$ there is n_0 with $f(n) < c \cdot g(n) \forall n \geq n_0$; equivalently $\lim f(n)/g(n) = 0$. Example: $2n = o(n^2)$, but $2n^2 \neq o(n^2)$.

Little-omega (strict lower bound). $f(n) = \omega(g(n))$ if for every $c > 0, f(n) > c \cdot g(n)$ for all large n ; equivalently $\lim f(n)/g(n) = \infty$. Example: $n^2 = \omega(n \log n)$.

Notation	Meaning	Analogy
$O(g)$	at most g	$\leq$
$\Omega(g)$	at least g	$\geq$
$\Theta(g)$	exactly g up to constants	$=$
$o(g)$	strictly less than g	$<$
$\omega(g)$	strictly greater than g	$>$

Growth order: $1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$.

Q8. What is meant by worst case time complexity and best case time complexity? Explain with an example. [10 marks · D23]

For the same input size n , different inputs make an algorithm do different amounts of work, so three cases are distinguished.

Best case is the minimum number of operations over all inputs of size n — the most favourable arrangement of data. It gives a lower bound on running time and is usually of little practical value because it is rarely met.

Worst case is the maximum number of operations over all inputs of size n . It is the guarantee that matters: the algorithm will never be slower than this. Quoted complexities default to the worst case.

Average case is the expected number of operations over all inputs of size n under an assumed probability distribution (usually uniform). It is the hardest to compute.

Example — linear search for `key` in an unsorted array of n elements:

```
for (i = 0; i < n; i++)
    if (a[i] == key) return i;
return -1;
```

- Best case: key is at $a[0] \rightarrow 1$ comparison $\rightarrow \Omega(1)$.
- Worst case: key is at $a[n-1]$ or absent $\rightarrow n$ comparisons $\rightarrow O(n)$.
- Average case (key present, equally likely at any position): $(1 + 2 + \dots + n)/n = (n+1)/2$ comparisons $\rightarrow O(n)$.

Example — bubble sort with the swap flag: best case is an already-sorted array, one pass of $n-1$ comparisons and no swaps $\rightarrow O(n)$; worst case is a reverse-sorted array, $n(n-1)/2$ comparisons and as many swaps $\rightarrow O(n^2)$.

Example — quick sort: best/average case is a pivot that splits the array in half, $T(n) = 2T(n/2) + O(n) = O(n \log n)$; worst case is an already-sorted array with the first element as pivot, giving splits of size $n-1$ and 0 , $T(n) = T(n-1) + O(n) = O(n^2)$.

Q9. What is an algorithm? What are its properties? Explain tradeoff between space and time complexity of algorithms. Explain time complexity of linear search algorithm. [10 marks · D25]

An **algorithm** is a finite, ordered set of unambiguous instructions that, for every legal input, terminates after a finite number of steps and produces the required output.

Properties: 1. **Finiteness** — terminates after finitely many steps. 2. **Definiteness** — every step is precisely and unambiguously stated. 3. **Input** — zero or more externally supplied quantities. 4. **Output** — one or more quantities with a specified relation to the input. 5. **Effectiveness** — every operation is basic enough to be carried out exactly, in finite time, with pencil and paper.

Space-time trade-off. An algorithm can usually be made faster by storing more, or made leaner by recomputing more. Examples: a hash table gives $O(1)$ average lookup for $O(n)$ extra space, while a linear scan of the same data needs no extra space but $O(n)$ per lookup; dynamic programming stores a table of sub-results to replace exponential recomputation; a sparse matrix stored as a full array has $O(1)$ access but $O(n^2)$ space, whereas the triplet form uses $O(k)$ space but needs a search per access.

Time complexity of linear search. The array is scanned from the first element until the key is found or the array ends:

```
LinearSearch(a, n, key)
    for (i = 0; i < n; i++)
        if (a[i] == key) return i;
    return -1;
```

The basic operation is the comparison `a[i] == key`. - Best case: key at `a[0]` $\rightarrow$ 1 comparison $\rightarrow O(1)$. - Worst case: key at `a[n-1]` or absent $\rightarrow$ n comparisons $\rightarrow O(n)$. - Average case: if the key is equally likely at each of the n positions, expected comparisons $= (1/n) \sum_{k=1}^n k = (n+1)/2 \rightarrow O(n)$.

Therefore $T(n) = O(n)$ and the auxiliary space is $O(1)$. Linear search needs no sorted input, which is its only advantage over binary search's $O(\log n)$.

Q10. What is algorithm? Explain the meaning of time complexity of an algorithm. Write binary search algorithm and calculate its time complexity. [10 marks · J26]

An algorithm is a finite sequence of definite, effective instructions that transforms a given input into the required output and halts. Its five properties are finiteness, definiteness, input, output and effectiveness.

Time complexity is the function $T(n)$ counting the elementary operations the algorithm performs on an input of size n . It is quoted asymptotically (O , Ω , Θ) so that it is independent of machine speed, language and compiler, and it is normally stated for the worst case.

Binary search. Precondition: the array must be **sorted in ascending order**.

```
BinarySearch(a, n, key)
{
    int low = 0, high = n - 1, mid;
    while (low <= high)
    {
        mid = (low + high) / 2;
        if (key == a[mid])
            return mid;           /* found */
        else if (key < a[mid])
            high = mid - 1;       /* search left half */
        else
            low = mid + 1;        /* search right half */
    }
    return -1;                   /* not found */
}
```

Derivation of the complexity. Each iteration performs $O(1)$ work and halves the candidate interval. Starting from n elements, after k iterations the interval holds $n/2^k$ elements. The loop ends when the interval has one element:

$$n / 2^k = 1 \implies 2^k = n \implies k = \log_2 n.$$

So the worst case does $\lceil \log_2 n \rceil + 1$ comparisons: $T(n) = O(\log_2 n)$. The recurrence form is $T(n) = T(n/2) + c$, $T(1) = c$, which unrolls to $c \cdot \log_2 n + c$.

Best case: the key is at the first `mid` $\rightarrow$ 1 comparison $\rightarrow O(1)$. Average case: $O(\log n)$. Space: $O(1)$ for the iterative version, $O(\log n)$ stack for the recursive version. For $n = 1024$ binary search needs at most 11 comparisons where linear search needs 1024.

Queues

Q1. Define "Queue". Explain the operations that can be performed on a Queue. [10 marks · J22]

A **queue** is a linear data structure in which insertions are made at one end, called the **rear**, and deletions at the other end, called the **front**. The element inserted first is removed first, so a queue

is a **FIFO** (First-In-First-Out) structure. Two index variables are kept: `front` points to the element to be removed next, `rear` to the last element inserted. Initially `front = rear = -1`.

Operations:

Operation	Effect	Condition checked	Cost
<code>createQueue()</code>	Initialise <code>front = rear = -1</code>	—	O(1)
<code>enqueue(x)</code> / insert	Increment <code>rear</code> , store <code>x</code> at <code>rear</code>	Overflow: <code>rear == MAX-1</code>	O(1)
<code>dequeue()</code> / delete	Return <code>q[front]</code> , increment <code>front</code>	Underflow: <code>front == -1</code> or <code>front > rear</code>	O(1)
<code>isEmpty()</code>	<code>front == -1</code> or <code>front > rear</code>	—	O(1)
<code>isFull()</code>	<code>rear == MAX-1</code>	—	O(1)
<code>peek()</code> / front	Return <code>q[front]</code> without removing it	Underflow	O(1)

```

enqueue(q, x)
{
    if (rear == MAX - 1) { print "Overflow"; return; }
    if (front == -1) front = 0;
    q[++rear] = x;
}

dequeue(q)
{
    if (front == -1 || front > rear) { print "Underflow"; return -1; }
    x = q[front++];
    if (front > rear) front = rear = -1;    /* queue became empty */
    return x;
}

```

Applications: CPU and printer scheduling, buffering between a fast producer and a slow consumer, breadth-first search, simulation of waiting lines, message queues.

Defect of the linear array queue: after repeated deletions `front` advances and the cells before it cannot be reused — the queue reports overflow while the array is largely empty. The circular queue removes this wastage.

Q2. Write the operations that are performed on queues. Write an algorithm to delete an element from the queue. [10 marks · D23]

The operations on a queue are `createQueue`, `enqueue` (insert at `rear`), `dequeue` (delete from `front`), `isEmpty`, `isFull` and `peek`. All are O(1). A queue is FIFO: the element that has been in the queue longest leaves first.

Assumptions: the queue is held in an array `q[MAX]`; `front` and `rear` are global integers initialised to `-1`; an empty queue is denoted by `front == -1`.

```

#define MAX 50
int q[MAX], front = -1, rear = -1;

int dequeue(void)
{
    int x;
    if (front == -1)                /* underflow test */
    {
        printf("Queue Underflow: queue is empty");
        return -1;
    }
    x = q[front];                  /* element to return */
    if (front == rear)             /* only one element */
        front = rear = -1;        /* queue becomes empty */
    else
        front = front + 1;         /* advance front */
    return x;
}

```

Trace on $q = [10, 20, 30]$ with $front = 0$, $rear = 2$: first `dequeue()` returns 10, $front = 1$; second returns 20, $front = 2$; third returns 30 and since $front == rear$ both are reset to -1 ; a fourth call reports underflow.

Time complexity $O(1)$, space complexity $O(1)$ auxiliary. Note that the deleted cells $q[0..front-1]$ are not reusable in a linear queue — this wastage is the reason the circular queue exists.

Q3. What is a queue? Write an algorithm for implementation of a queue. [10 marks · J23]

A queue is a linear FIFO list with insertion at the rear and deletion at the front. It is implemented here with a one-dimensional array.

Assumptions: fixed capacity MAX; `front` and `rear` initialised to -1 ; integer elements; -1 returned as the error value.

```

#define MAX 50
int q[MAX];
int front = -1, rear = -1;

void createQueue(void) { front = rear = -1; }

int isEmpty(void)      { return (front == -1); }

int isFull(void)       { return (rear == MAX - 1); }

void enqueue(int x)
{
    if (isFull()) { printf("Overflow"); return; }
    if (front == -1) front = 0;          /* first insertion */
    rear = rear + 1;
    q[rear] = x;
}

int dequeue(void)
{
    int x;
    if (isEmpty()) { printf("Underflow"); return -1; }
    x = q[front];
    if (front == rear) front = rear = -1; /* last element removed */
    else front = front + 1;
    return x;
}

int peek(void)
{
    if (isEmpty()) { printf("Queue empty"); return -1; }
    return q[front];
}

```

Example: enqueue 5, 9, 12 → q = [5, 9, 12], front = 0, rear = 2. dequeue → returns 5, front = 1. enqueue 20 → rear = 3, q = [5, 9, 12, 20] with cell 0 wasted.

Each operation is $O(1)$ in time; the structure uses $O(\text{MAX})$ space. Limitations: the capacity is fixed at compile time and deleted cells are not reclaimed, both cured by the circular queue or a linked-list queue.

Q4. Describe the implementation of a queue using a linked list. [10 marks · J25]

In a linked queue each element is a dynamically allocated node holding the data and a pointer to the next node. Two external pointers are kept: `front` to the first node and `rear` to the last. There is no overflow condition (other than exhaustion of heap memory) and no wasted cells, so the queue grows and shrinks on demand.

```

struct node {
    int data;
    struct node *next;
};
struct node *front = NULL, *rear = NULL;

void enqueue(int x)
{
    struct node *newnode;
    newnode = (struct node *) malloc(sizeof(struct node));
    if (newnode == NULL) { printf("Memory not available"); return; }
    newnode->data = x;
    newnode->next = NULL;
    if (rear == NULL)                /* queue was empty */
        front = rear = newnode;
    else {
        rear->next = newnode;        /* link at the tail */
        rear = newnode;
    }
}

int dequeue(void)
{
    struct node *temp; int x;
    if (front == NULL) { printf("Underflow: queue empty"); return -1; }
    temp = front;
    x = temp->data;
    front = front->next;
    if (front == NULL) rear = NULL; /* last node deleted */
    free(temp);
    return x;
}

int isEmpty(void) { return (front == NULL); }

```

Example: enqueue 7 → front = rear = N1(7). enqueue 11 → N1→N2, rear = N2. dequeue → returns 7, front = N2. dequeue → returns 11, front becomes NULL so rear is also set to NULL.

The special case to state explicitly: when the node being deleted is the only node, both `front` and `rear` must be set to NULL, otherwise `rear` dangles at freed memory.

Time complexity $O(1)$ per operation; space $O(n)$ with one extra pointer per element, which is the trade-off against the array version.

Q5. What is queue data structure? What are its properties? Implement queue using linked list. Make necessary assumptions. [10 marks · D25]

A queue is a linear list in which elements are inserted at the rear and deleted from the front, giving First-In-First-Out order.

Properties: FIFO discipline; two access points only (front and rear); insertion and deletion are $O(1)$; no random access — the k -th element cannot be reached without removing $k-1$ elements; the order of removal is exactly the order of arrival; a queue is empty when front is NULL (linked) or front == -1 (array).

Assumptions: elements are integers; nodes are allocated with `malloc`; `front` and `rear` are global pointers initialised to NULL; -1 is the error return.

```

struct node { int data; struct node *next; };
struct node *front = NULL, *rear = NULL;

void enqueue(int x)                /* insert at rear */
{
    struct node *p = (struct node *) malloc(sizeof(struct node));
    if (p == NULL) { printf("Heap full"); return; }
    p->data = x; p->next = NULL;
    if (rear == NULL) front = rear = p;
    else { rear->next = p; rear = p; }
}

int dequeue(void)                  /* delete from front */
{
    struct node *p; int x;
    if (front == NULL) { printf("Underflow"); return -1; }
    p = front; x = p->data;
    front = front->next;
    if (front == NULL) rear = NULL;
    free(p);
    return x;
}

void display(void)
{
    struct node *p = front;
    while (p != NULL) { printf("%d ", p->data); p = p->next; }
}

```

Trace: enqueue 4, 8, 15 → front→4→8→15←rear. dequeue returns 4. dequeue returns 8. dequeue returns 15 and resets both pointers to NULL.

All operations $O(1)$; space $O(n)$. Advantage over the array queue: dynamic size, no overflow, no wasted cells.

Q6. What is Queue? Write algorithm for addition of an element to the Queue. [5 marks · J26]

A queue is a linear FIFO structure with insertion at the rear and deletion at the front.

Assumptions: array `q[MAX]`, `front` and `rear` global and initialised to `-1`.

```

void enqueue(int x)
{
    if (rear == MAX - 1)                /* overflow */
    {
        printf("Queue Overflow");
        return;
    }
    if (front == -1)                    /* first element */
        front = 0;
    rear = rear + 1;
    q[rear] = x;
}

```

Example with `MAX = 5`: enqueue 10 → front = 0, rear = 0; enqueue 20 → rear = 1; enqueue 30 → rear = 2, `q = [10, 20, 30, , ]`.

Time complexity $O(1)$; space complexity $O(1)$ auxiliary.

Q7. What is a Circular Queue? How does it differ from a queue? [10 marks · D21]

A **circular queue** is a queue implemented in an array that is treated as a ring: the cell after $q[\text{MAX}-1]$ is $q[0]$. Pointer advancement uses modular arithmetic, $\text{rear} = (\text{rear} + 1) \% \text{MAX}$ and $\text{front} = (\text{front} + 1) \% \text{MAX}$, so cells vacated by deletion are reused and the array is fully utilised.

Conditions: - Empty: $\text{front} == -1$ - Full: $(\text{rear} + 1) \% \text{MAX} == \text{front}$ - One element: $\text{front} == \text{rear}$

Point	Linear queue	Circular queue
Array treated as	Straight line, last cell is the end	Ring, last cell joins the first
Pointer update	$\text{rear} = \text{rear} + 1$	$\text{rear} = (\text{rear} + 1) \% \text{MAX}$
Full condition	$\text{rear} == \text{MAX} - 1$	$(\text{rear} + 1) \% \text{MAX} == \text{front}$
Space utilisation	Cells before front are wasted	All cells reusable
False overflow	Yes — reports full while cells are free	No
Remedy needed	Shifting all elements left, $O(n)$	None
Implementation	Simpler	Needs modulo arithmetic and care over full/empty

Example with $\text{MAX} = 5$: after inserting 5 elements and deleting 2, a linear queue has $\text{front} = 2$, $\text{rear} = 4$ and refuses a sixth insertion although $q[0]$ and $q[1]$ are free. The circular queue sets $\text{rear} = (4 + 1) \% 5 = 0$ and stores the new element in $q[0]$.

Q8. What is a circular queue? How does it differ from a queue? Show insert and delete operations in a circular queue with the help of an example. [10 marks · D24]

Definition and the comparison table are as in Q7: a circular queue wraps the array round using $\%$ MAX , so no cell is wasted and no false overflow occurs.

```
#define MAX 5
int cq[MAX], front = -1, rear = -1;

void cEnqueue(int x)
{
    if ((rear + 1) % MAX == front) { printf("Queue Full"); return; }
    if (front == -1) front = 0;
    rear = (rear + 1) % MAX;
    cq[rear] = x;
}

int cDequeue(void)
{
    int x;
    if (front == -1) { printf("Queue Empty"); return -1; }
    x = cq[front];
    if (front == rear) front = rear = -1;          /* last element */
    else front = (front + 1) % MAX;
    return x;
}
```

Trace with $\text{MAX} = 5$, cells 0..4, starting empty ($\text{front} = \text{rear} = -1$):

Step	Operation	front	rear	Array contents (0..4)
1	insert 10	0	0	10, , , , _
2	insert 20	0	1	10, 20, , , _
3	insert 30	0	2	10, 20, 30, , _
4	insert 40	0	3	10, 20, 30, 40, _
5	delete → 10	1	3	_, 20, 30, 40, _
6	delete → 20	2	3	, , 30, 40, _
7	insert 50	2	4	, , 30, 40, 50
8	insert 60	2	0	60, _, 30, 40, 50
9	insert 70	—	—	Full: $(0+1)\%5 = 1 == \text{front?}$ no → rear = 1, cq[1] = 70, front = 2, rear = 1
10	insert 80	2	1	Rejected: $(1+1)\%5 = 2 == \text{front}$ → Queue Full

At step 8 the pointer wrapped from 4 to $(4+1)\%5 = 0$ and reused the cell vacated at step 5 — exactly what a linear queue cannot do. At step 10 four of five cells hold data and one cell is deliberately left free so that the full and empty tests remain distinguishable. Each operation is $O(1)$.

Q9. Write algorithm for deleting element from a circular queue. [5 marks · J26]

Assumptions: array cq[MAX]; front, rear global, both -1 when empty.

```
int cDequeue(void)
{
    int x;
    if (front == -1)                /* underflow */
    {
        printf("Circular Queue Empty");
        return -1;
    }
    x = cq[front];
    if (front == rear)              /* only one element left */
        front = rear = -1;
    else
        front = (front + 1) % MAX;  /* wrap around */
    return x;
}
```

Example, MAX = 5, cq = [60, _, 30, 40, 50], front = 2, rear = 0: delete returns 30, front = 3; delete returns 40, front = 4; delete returns 50, front = $(4+1)\%5 = 0$; delete returns 60 and since front == rear == 0 both are reset to -1.

Time complexity $O(1)$; space complexity $O(1)$.

Q10. What is a Dequeue? How does it differ from a Queue? [10 marks · J22]

A **deque** (deque, double-ended queue) is a linear list in which insertion and deletion are both permitted at **both** ends. It therefore supports four operations instead of two: insertFront,

`insertRear`, `deleteFront`, `deleteRear`, plus `isEmpty` and `isFull`. Implemented on a circular array of size `MAX` with `front` and `rear`:

- `insertRear`: `rear = (rear + 1) % MAX`
- `insertFront`: `front = (front - 1 + MAX) % MAX`
- `deleteFront`: `front = (front + 1) % MAX`
- `deleteRear`: `rear = (rear - 1 + MAX) % MAX`
- `full`: `(rear + 1) % MAX == front`; `empty`: `front == -1`

Two restricted variants: - **Input-restricted deque** — insertion allowed at one end only, deletion at both ends. - **Output-restricted deque** — insertion allowed at both ends, deletion at one end only.

Point	Queue	Deque
Insertion	At rear only	At both front and rear
Deletion	At front only	At both front and rear
Discipline	Strictly FIFO	FIFO or LIFO as required
Operations	2 primary	4 primary
Generality	Special case	Generalises both stack and queue
Use	Scheduling, buffering, BFS	Undo lists, palindrome testing, sliding-window maximum, job stealing

Example on a deque: `insertRear 10`, `insertRear 20`, `insertFront 5` gives 5, 10, 20; `deleteRear` removes 20; `deleteFront` removes 5, leaving 10. A plain queue could not have done either the `insertFront` or the `deleteRear`.

Q11. Explain the process of implementing two queues in an array. [10 marks · J24]

Two queues can share one array of size `MAX` so that the space unused by one is available to the other. The standard technique is to let the two queues grow **towards each other from the two ends**:

- Queue 1 occupies the low end: `front1/rear1` start at `-1` and `rear1` increases.
- Queue 2 occupies the high end: `front2/rear2` start at `MAX` and `rear2` decreases.
- The array is full when `rear1 + 1 == rear2`, i.e. the two queues meet.

```

#define MAX 10
int a[MAX];
int front1 = -1, rear1 = -1;      /* queue 1 grows upward */
int front2 = MAX, rear2 = MAX;    /* queue 2 grows downward */

void enqueue1(int x)
{
    if (rear1 + 1 == rear2) { printf("Overflow"); return; }
    if (front1 == -1) front1 = 0;
    a[++rear1] = x;
}

void enqueue2(int x)
{
    if (rear1 + 1 == rear2) { printf("Overflow"); return; }
    if (front2 == MAX) front2 = MAX - 1;
    a[--rear2] = x;
}

int dequeue1(void)
{
    if (front1 == -1 || front1 > rear1) { printf("Q1 empty"); return -1; }
    return a[front1++];
}

int dequeue2(void)
{
    if (front2 == MAX || front2 < rear2) { printf("Q2 empty"); return -1; }
    return a[front2--];
}

```

Layout for MAX = 10 after inserting 3 elements in Q1 and 2 in Q2:

index :	0	1	2	3	4	5	6	7	8	9
data :	A	B	C	-	-	-	-	-	Y	X
	^		^						^	^
	front1		rear1					rear2		front2

Advantage: one overflow test for both queues, and either queue may use up to MAX−1 cells if the other is empty — far better utilisation than two fixed half-arrays. The alternative method, dividing the array into two fixed halves of MAX/2 with wrap-around inside each half, is simpler but one queue can overflow while the other half is empty. Each operation remains O(1).

Sorting

Q1. Write an algorithm for bubble sort. Sort the following set of data in ascending order using bubble sort. Show all steps of application of algorithm: 100, 50, 60, 70, 150, 80 [10 marks · D21]

```
void bubbleSort(int a[], int n)
{
    int i, j, temp, swapped;
    for (i = 0; i < n - 1; i++)          /* n-1 passes */
    {
        swapped = 0;
        for (j = 0; j < n - 1 - i; j++) /* adjacent compares */
        {
            if (a[j] > a[j+1])          /* out of order */
            {
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
                swapped = 1;
            }
        }
        if (swapped == 0) break;        /* already sorted: early exit */
    }
}
```

Initial array: 100, 50, 60, 70, 150, 80 (n = 6).

Pass 1 (j = 0..4):

Compare	Action	Array after
100 > 50	swap	50, 100, 60, 70, 150, 80
100 > 60	swap	50, 60, 100, 70, 150, 80
100 > 70	swap	50, 60, 70, 100, 150, 80
100 < 150	no swap	50, 60, 70, 100, 150, 80
150 > 80	swap	50, 60, 70, 100, 80, 150

End of pass 1: **50, 60, 70, 100, 80, 150** — largest element 150 is in place.

Pass 2 (j = 0..3):

Compare	Action	Array after
50 < 60	no swap	50, 60, 70, 100, 80, 150
60 < 70	no swap	50, 60, 70, 100, 80, 150
70 < 100	no swap	50, 60, 70, 100, 80, 150
100 > 80	swap	50, 60, 70, 80, 100, 150

End of pass 2: **50, 60, 70, 80, 100, 150**.

Pass 3 ($j = 0..2$): $50 < 60$, $60 < 70$, $70 < 80$ — no swap occurs, `swapped == 0`, so the algorithm terminates early.

Sorted output: **50, 60, 70, 80, 100, 150**.

Complexity: best $O(n)$ (with the flag, already-sorted input), average and worst $O(n^2)$; space $O(1)$, in-place and stable.

Q2. Write an algorithm for Bubble sort. Sort the following set of data in ascending order using bubble sort. Show all steps of application of algorithm: 10, 5, 6, 7, 15, 8 Also, explain Best case, Average case and Worst case time complexities of bubble sort. [10 marks · D24]

The algorithm is as in Q1 above (nested loops with the `swapped` flag).

Initial array: 10, 5, 6, 7, 15, 8 ($n = 6$).

Pass 1:

Compare	Action	Array after
$10 > 5$	swap	5, 10, 6, 7, 15, 8
$10 > 6$	swap	5, 6, 10, 7, 15, 8
$10 > 7$	swap	5, 6, 7, 10, 15, 8
$10 < 15$	no swap	5, 6, 7, 10, 15, 8
$15 > 8$	swap	5, 6, 7, 10, 8, 15

End of pass 1: **5, 6, 7, 10, 8, 15**.

Pass 2:

Compare	Action	Array after
$5 < 6$	no swap	5, 6, 7, 10, 8, 15
$6 < 7$	no swap	5, 6, 7, 10, 8, 15
$7 < 10$	no swap	5, 6, 7, 10, 8, 15
$10 > 8$	swap	5, 6, 7, 8, 10, 15

End of pass 2: **5, 6, 7, 8, 10, 15**.

Pass 3: $5 < 6$, $6 < 7$, $7 < 8$ — no swaps, flag is 0, algorithm stops.

Sorted output: **5, 6, 7, 8, 10, 15**.

Complexities. - **Best case — $O(n)$.** The input is already sorted. The first pass makes $n-1$ comparisons and zero swaps, the flag stays 0 and the algorithm exits. Comparisons = $n-1$, swaps = 0. Without the flag the best case would also be $O(n^2)$. - **Average case — $O(n^2)$.** On a random permutation about half the pairs are inverted, giving roughly $n(n-1)/4$ swaps and $n(n-1)/2$ comparisons, both quadratic. - **Worst case — $O(n^2)$.** The input is in descending order. Every comparison causes a swap: comparisons = $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$ and the same number of swaps. For $n = 6$ that is 15 comparisons and 15 swaps.

Space complexity $O(1)$ (one temporary); the sort is in-place and stable.

Q3. Write an algorithm for Bubble Sort. Also, write its complexity. [10 marks · J23]

Bubble sort repeatedly compares adjacent elements and swaps those out of order, so after the i -th pass the i largest elements occupy the last i positions.

```
void bubbleSort(int a[], int n)
{
    int i, j, temp, swapped;
    for (i = 0; i < n - 1; i++)
    {
        swapped = 0;
        for (j = 0; j < n - 1 - i; j++)
            if (a[j] > a[j+1])
            {
                temp = a[j]; a[j] = a[j+1]; a[j+1] = temp;
                swapped = 1;
            }
        if (!swapped) break;
    }
}
```

Complexity derivation. The inner loop runs $n-1-i$ times for pass i , so the total number of comparisons is

$$\sum_{i=0}^{n-2} (n-1-i) = (n-1) + (n-2) + \dots + 1 = n(n-1)/2 = (n^2 - n)/2.$$

Hence $T(n) = O(n^2)$ for the average and worst cases. With the `swapped` flag the best case (sorted input) needs one pass of $n-1$ comparisons, so $T(n) = \Omega(n) = O(n)$. Number of swaps: 0 in the best case, $n(n-1)/2$ in the worst. Auxiliary space is a single temporary, $O(1)$. The sort is in-place, stable, and suitable only for small n or nearly sorted data.

Q4. Write an algorithm for implementation of Bubble sort. [10 marks · D23]

Assumptions: the array `a[0..n-1]` holds n integers; sorting is in ascending order and in place.

```

#include <stdio.h>
#define MAX 100

void bubbleSort(int a[], int n)
{
    int i, j, temp, swapped;

    for (i = 0; i < n - 1; i++)          /* at most n-1 passes */
    {
        swapped = 0;
        for (j = 0; j < n - 1 - i; j++)  /* compare neighbours */
        {
            if (a[j] > a[j+1])
            {
                temp = a[j];             /* exchange */
                a[j] = a[j+1];
                a[j+1] = temp;
                swapped = 1;
            }
        }
        if (swapped == 0)                /* no exchange in this pass */
            break;                       /* array already sorted */
    }
}

int main(void)
{
    int a[MAX], n, i;
    scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    bubbleSort(a, n);
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    return 0;
}

```

Working: in pass 1 the largest element "bubbles" to position $n-1$; in pass 2 the second largest reaches $n-2$; after at most $n-1$ passes the array is sorted. The reduced inner bound $n-1-i$ avoids re-examining the already-placed tail.

Short example: 4, 1, 3 $\rightarrow$ pass 1 gives 1, 3, 4 (two swaps); pass 2 gives 1, 3, 4 with no swap, so the flag stops the sort.

Time complexity: $O(n)$ best, $O(n^2)$ average and worst. Space complexity: $O(1)$, in-place and stable.

Q5. Sort the following lists using Bubble Sort Algorithm. Showing intermediate steps: 25 8 22 90 80 60 10 [10 marks · D25]

Algorithm as in Q4. Initial array: 25, 8, 22, 90, 80, 60, 10 ($n = 7$).

Pass 1 (comparisons $j = 0..5$):

Compare	Action	Array after
$25 > 8$	swap	8, 25, 22, 90, 80, 60, 10
$25 > 22$	swap	8, 22, 25, 90, 80, 60, 10
$25 < 90$	—	8, 22, 25, 90, 80, 60, 10
$90 > 80$	swap	8, 22, 25, 80, 90, 60, 10
$90 > 60$	swap	8, 22, 25, 80, 60, 90, 10
$90 > 10$	swap	8, 22, 25, 80, 60, 10, 90

End pass 1: **8, 22, 25, 80, 60, 10, 90**

Pass 2 ($j = 0..4$): $8 < 22$, $22 < 25$, $25 < 80$, $80 > 60$ swap, $80 > 10$ swap End pass 2: **8, 22, 25, 60, 10, 80, 90**

Pass 3 ($j = 0..3$): $8 < 22$, $22 < 25$, $25 < 60$, $60 > 10$ swap End pass 3: **8, 22, 25, 10, 60, 80, 90**

Pass 4 ($j = 0..2$): $8 < 22$, $22 < 25$, $25 > 10$ swap End pass 4: **8, 22, 10, 25, 60, 80, 90**

Pass 5 ($j = 0..1$): $8 < 22$, $22 > 10$ swap End pass 5: **8, 10, 22, 25, 60, 80, 90**

Pass 6 ($j = 0$): $8 < 10$, no swap — flag is 0, algorithm terminates.

Sorted output: **8, 10, 22, 25, 60, 80, 90**. Total comparisons $6+5+4+3+2+1 = 21$; swaps = 11.

Q6. Write an algorithm for Quick Sort. Sort the following set of data in ascending order using Quick Sort. Show all steps of application of algorithm: 110, 50, 60, 70, 150, 80 [10 marks · J22]

Quick sort is divide and conquer: choose a pivot, partition the array so that all elements $\leq$ pivot lie to its left and all $>$ pivot to its right, then sort the two parts recursively.

```

int partition(int a[], int low, int high)
{
    int pivot = a[low];           /* first element as pivot */
    int i = low + 1, j = high, temp;

    while (i <= j)
    {
        while (i <= high && a[i] <= pivot) i++;    /* scan right */
        while (j > low && a[j] > pivot) j--;    /* scan left */
        if (i < j) { temp = a[i]; a[i] = a[j]; a[j] = temp; }
    }
    temp = a[low]; a[low] = a[j]; a[j] = temp;    /* pivot to place */
    return j;                                     /* pivot index */
}

void quickSort(int a[], int low, int high)
{
    int p;
    if (low < high)
    {
        p = partition(a, low, high);
        quickSort(a, low, p - 1);    /* left sub-array */
        quickSort(a, p + 1, high);    /* right sub-array */
    }
}

```

Initial array (indices 0..5): 110, 50, 60, 70, 150, 80.

Level 1 — quickSort(0, 5), pivot = 110 - i scans right from index 1: $50 \leq 110$, $60 \leq 110$, $70 \leq 110$, stops at $a[4] = 150$. - j scans left from index 5: $a[5] = 80$ is not > 110 , so $j = 5$. - $i = 4 < j = 5 \rightarrow$ swap $a[4]$, $a[5] \rightarrow 110, 50, 60, 70, \mathbf{80}, \mathbf{150}$ - i continues: $a[4] = 80 \leq 110 \rightarrow i = 5$; $a[5] = 150 > 110 \rightarrow$ stop, $i = 5$. j: $a[5] = 150 > 110 \rightarrow j = 4$. $i > j$, loop ends. - Swap pivot $a[0]$ with $a[j] = a[4] \rightarrow \mathbf{80}, \mathbf{50}, \mathbf{60}, \mathbf{70}, \mathbf{110}, \mathbf{150}$; pivot 110 is fixed at index 4. - Sub-arrays: left $[80, 50, 60, 70]$ (0..3), right $[150]$ (5..5, already sorted).

Level 2 — quickSort(0, 3), pivot = 80 - i: 50, 60, 70 all $\leq 80 \rightarrow$ i runs past j. j: $a[3] = 70$ is not $> 80 \rightarrow j = 3$. - Swap $a[0]$ with $a[3] \rightarrow \mathbf{70}, \mathbf{50}, \mathbf{60}, \mathbf{80}$, pivot 80 fixed at index 3. - Array now: 70, 50, 60, 80, 110, 150. Sub-arrays: left $[70, 50, 60]$ (0..2), right empty.

Level 3 — quickSort(0, 2), pivot = 70 - $50 \leq 70$, $60 \leq 70 \rightarrow$ i past j; $j = 2$ (60 not > 70). Swap $a[0]$, $a[2] \rightarrow \mathbf{60}, \mathbf{50}, \mathbf{70}$, pivot 70 at index 2. - Array now: 60, 50, 70, 80, 110, 150. Sub-array left $[60, 50]$ (0..1).

Level 4 — quickSort(0, 1), pivot = 60 - $50 \leq 60 \rightarrow$ i past j; $j = 1$. Swap $a[0]$, $a[1] \rightarrow \mathbf{50}, \mathbf{60}$, pivot at index 1. Left sub-array is a single element.

Sorted output: **50, 60, 70, 80, 110, 150**.

Note that this input gives the worst case shape: the pivot always lands at the far right, producing partitions of size $n-1$ and 0. Complexity: best and average $O(n \log n)$, worst $O(n^2)$; space $O(\log n)$ average recursion stack, $O(n)$ worst.

Q7. Write an algorithm for quick sort. Sort the following set of data in ascending order using quick sort. Show all steps of application of the algorithm: 10, 5, 6, 7, 15, 8 Write the best case and worst case time complexities. [10 marks · J25]

Algorithm as in Q6 (partition plus recursive quickSort).

Initial array (indices 0..5): 10, 5, 6, 7, 15, 8.

Level 1 — quickSort(0, 5), pivot = a[0] = 10 - i from 1: $5 \leq 10$, $6 \leq 10$, $7 \leq 10$, stops at $a[4] = 15 \rightarrow i = 4$. - j from 5: $a[5] = 8$ is not $> 10 \rightarrow j = 5$. - $i < j \rightarrow$ swap $a[4], a[5] \rightarrow 10, 5, 6, 7, \mathbf{8}, \mathbf{15}$ - i continues: $a[4] = 8 \leq 10 \rightarrow i = 5$; $a[5] = 15 > 10 \rightarrow$ stop. j: $a[5] = 15 > 10 \rightarrow j = 4$. $i > j$, exit. - Swap pivot $a[0]$ with $a[4] \rightarrow \mathbf{8}, \mathbf{5}, \mathbf{6}, \mathbf{7}, \mathbf{10}, \mathbf{15}$; pivot 10 fixed at index 4. - Left $[8, 5, 6, 7]$ (0..3), right $[15]$ (5..5) sorted.

Level 2 — quickSort(0, 3), pivot = 8 - 5, 6, 7 all $\leq 8 \rightarrow i$ past j; $j = 3$ (7 not > 8). - Swap $a[0], a[3] \rightarrow \mathbf{7}, \mathbf{5}, \mathbf{6}, \mathbf{8}$; pivot 8 fixed at index 3. Array: 7, 5, 6, 8, 10, 15.

Level 3 — quickSort(0, 2), pivot = 7 - 5, 6 $\leq 7 \rightarrow i$ past j; $j = 2$. Swap $a[0], a[2] \rightarrow \mathbf{6}, \mathbf{5}, \mathbf{7}$; pivot 7 at index 2. Array: 6, 5, 7, 8, 10, 15.

Level 4 — quickSort(0, 1), pivot = 6 - 5 $\leq 6 \rightarrow j = 1$, swap $a[0], a[1] \rightarrow \mathbf{5}, \mathbf{6}$. Array: 5, 6, 7, 8, 10, 15. Recursion ends.

Sorted output: **5, 6, 7, 8, 10, 15**.

Best case — $O(n \log n)$. The pivot splits the array into two nearly equal halves at every level: $T(n) = 2T(n/2) + O(n)$, giving $\log_2 n$ levels each costing $O(n)$ for the partition scan, so $T(n) = O(n \log n)$. Average case is also $O(n \log n)$.

Worst case — $O(n^2)$. The pivot is the smallest or largest element at every level, so one partition has $n-1$ elements and the other none: $T(n) = T(n-1) + O(n) = O(n^2)$. This happens when the first element is chosen as pivot on data that is already sorted (ascending or descending) — as very nearly happened in the trace above. Remedies: random pivot, or median-of-three. Space: $O(\log n)$ stack on average, $O(n)$ in the worst case; quick sort is in-place but not stable.

Q8. Write an algorithm for implementation of Quick Sort. [10 marks · J24]

[Extraction note: the printed paper reads "Write an association for implementation of Quick Sort"; "association" is an OCR corruption of "algorithm".]

Assumptions: $a[0..n-1]$ holds n integers; the first element of each sub-array is the pivot; the call is `quickSort(a, 0, n-1)`.

```

int partition(int a[], int low, int high)
{
    int pivot = a[low], i = low + 1, j = high, temp;

    while (i <= j)
    {
        while (i <= high && a[i] <= pivot) i++;    /* find element > pivot */
        while (j > low && a[j] > pivot) j--;    /* find element <= pivot */
        if (i < j)
        {
            temp = a[i]; a[i] = a[j]; a[j] = temp;
        }
    }
    temp = a[low]; a[low] = a[j]; a[j] = temp;    /* pivot into position */
    return j;
}

void quickSort(int a[], int low, int high)
{
    int p;
    if (low < high)
    {
        p = partition(a, low, high);    /* divide */
        quickSort(a, low, p - 1);    /* conquer */
        quickSort(a, p + 1, high);
    }
}

```

Working: after `partition` returns `p`, `a[p]` is in its final sorted position, everything in `a[low..p-1]` is $\leq a[p]$ and everything in `a[p+1..high]` is $> a[p]$; no merge step is needed, which is why quick sort is in-place. The recursion bottoms out when `low >= high`.

Short example: 3, 7, 2 with pivot 3 $\rightarrow$ `i` stops at 7 (index 1), `j` stops at index 2 (2 is not > 3)... swap gives 3, 2, 7; then `i` = 2, `j` = 1, swap pivot with `a[1]` $\rightarrow$ 2, 3, 7, pivot at index 1, sub-arrays [2] and [7], done.

Time complexity: $O(n \log n)$ best and average, $O(n^2)$ worst (sorted input with a first-element pivot). Space: $O(\log n)$ average recursion depth, $O(n)$ worst; not stable.

Q9. Write an algorithm for implementation of Insertion Sort. [10 marks · J24]

Insertion sort builds the sorted region from left to right: at step `i` the element `a[i]` is lifted out as `key` and the larger elements to its left are shifted one place right until the correct slot for `key` is found.

Assumptions: `a[0..n-1]` of `n` integers, sorted ascending in place.

```

void insertionSort(int a[], int n)
{
    int i, j, key;

    for (i = 1; i < n; i++)          /* a[0] is a sorted list of one */
    {
        key = a[i];                  /* element to be inserted      */
        j = i - 1;
        while (j >= 0 && a[j] > key) /* shift larger elements right */
        {
            a[j+1] = a[j];
            j = j - 1;
        }
        a[j+1] = key;                /* drop key into its place    */
    }
}

```

The loop invariant is that after iteration i the sub-array $a[0..i]$ is sorted. The condition $j \geq 0$ guards the left boundary and $a[j] > \text{key}$ (strict) is what makes the sort stable, since equal elements are not moved past one another.

Short example on 5, 2, 4: $i = 1$, $\text{key} = 2$, shift 5 right $\rightarrow$ 2, 5, 4; $i = 2$, $\text{key} = 4$, shift 5 right $\rightarrow$ 2, 4, 5. Sorted.

Time complexity: best $O(n)$ when the input is already sorted (the `while` test fails immediately each time, $n-1$ comparisons, no shifts); average and worst $O(n^2)$ — reverse-sorted input needs $1 + 2 + \dots + (n-1) = n(n-1)/2$ comparisons and shifts. Space complexity $O(1)$; the sort is in-place, stable, online, and the preferred method for small or nearly sorted arrays.

Q10. Write algorithm for Insertion Sort. Sort the following list, using Insertion Sort, showing intermediate steps: 8 20 15 28 9 [10 marks · J26]

Algorithm as in Q9 above.

Initial list: 8, 20, 15, 28, 9 ($n = 5$). The sorted part is shown in bold.

Pass $i = 1$, $\text{key} = 20$. Compare with 8: $8 > 20$ is false, so no shift; 20 stays. Array: **8, 20**, 15, 28, 9

Pass $i = 2$, $\text{key} = 15$. Compare with 20: $20 > 15 \rightarrow$ shift 20 right (8, 20, 20, 28, 9); compare with 8: $8 > 15$ false $\rightarrow$ stop, place key at index 1. Array: **8, 15, 20**, 28, 9

Pass $i = 3$, $\text{key} = 28$. Compare with 20: $20 > 28$ false $\rightarrow$ no shift. Array: **8, 15, 20, 28**, 9

Pass $i = 4$, $\text{key} = 9$. Compare with 28 $\rightarrow$ shift (8, 15, 20, 28, 28); compare with 20 $\rightarrow$ shift (8, 15, 20, 20, 28); compare with 15 $\rightarrow$ shift (8, 15, 15, 20, 28); compare with 8: $8 > 9$ false $\rightarrow$ stop, place key at index 1. Array: **8, 9, 15, 20, 28**

Summary table:

Pass	key	Array state after the pass
start	—	8, 20, 15, 28, 9
1	20	8, 20, 15, 28, 9
2	15	8, 15, 20, 28, 9
3	28	8, 15, 20, 28, 9
4	9	8, 9, 15, 20, 28

Sorted output: **8, 9, 15, 20, 28**. Comparisons = $1 + 2 + 1 + 4 = 8$; shifts = $0 + 1 + 0 + 3 = 4$.

Complexity: best $O(n)$, average and worst $O(n^2)$; space $O(1)$; stable and in-place.

Linked lists

Q1. Write an algorithm to insert an element into a singly linked list. Make necessary assumptions. [10 marks · D21]

Assumptions: each node holds an integer and a pointer to the next node; `head` is the external pointer, NULL for an empty list; the last node's `next` is NULL; insertion is requested after a node containing a given key, and the three standard positions (beginning, end, after a given node) are all covered; memory comes from `malloc`.

```

struct node {
    int data;
    struct node *next;
};
struct node *head = NULL;

struct node *getNode(int x)
{
    struct node *p = (struct node *) malloc(sizeof(struct node));
    if (p == NULL) { printf("Memory not available"); exit(1); }
    p->data = x; p->next = NULL;
    return p;
}

void insertAtBeginning(int x)
{
    struct node *p = getNode(x);
    p->next = head;          /* new node points to old first node */
    head = p;               /* head now points to the new node */
}

void insertAtEnd(int x)
{
    struct node *p = getNode(x), *q;
    if (head == NULL) { head = p; return; } /* empty list */
    q = head;
    while (q->next != NULL) q = q->next;    /* walk to last node */
    q->next = p;
}

void insertAfter(int key, int x)
{
    struct node *p, *q = head;
    while (q != NULL && q->data != key) q = q->next;
    if (q == NULL) { printf("Key %d not found", key); return; }
    p = getNode(x);
    p->next = q->next;    /* order matters: link forward first */
    q->next = p;
}

```

Example: list $10 \rightarrow 20 \rightarrow 30$, `insertAfter(20, 25)` locates the node 20, sets the new node's next to 30 and then 20's next to the new node, giving $10 \rightarrow 20 \rightarrow 25 \rightarrow 30$. Reversing the two assignments would lose the rest of the list.

Time complexity: $O(1)$ for insertion at the beginning; $O(n)$ for insertion at the end or after a searched key. Space complexity $O(1)$ auxiliary (one node allocated).

Q2. Write an algorithm to delete an element from a Singly Linked List. Make necessary assumptions. [10 marks · J22]

Assumptions: singly linked list of integers with external pointer `head`; the node to delete is identified by its data value `key`; the first occurrence is removed; the freed node is returned to the heap with `free`.

```

struct node { int data; struct node *next; };
struct node *head = NULL;

void deleteNode(int key)
{
    struct node *curr = head, *prev = NULL;

    if (head == NULL)                /* case 1: empty list */
    {
        printf("List is empty");
        return;
    }

    if (head->data == key)            /* case 2: delete first node */
    {
        head = head->next;           /* may become NULL if single node */
        free(curr);
        return;
    }

    while (curr != NULL && curr->data != key) /* case 3: search */
    {
        prev = curr;
        curr = curr->next;
    }

    if (curr == NULL)                /* case 4: key absent */
    {
        printf("Element %d not found", key);
        return;
    }

    prev->next = curr->next;          /* bypass the node (works for tail too) */
    free(curr);
}

```

Boundary cases covered: empty list; deletion of the head; deletion of the only node (head becomes NULL); deletion of the last node (prev->next becomes NULL); key not present.

Example: 10 → 20 → 30 → NULL, deleteNode(20): prev = node(10), curr = node(20); prev->next = curr->next makes 10 point to 30; node 20 is freed, giving 10 → 30 → NULL.

Time complexity O(n) (O(1) if the node is the head); space complexity O(1).

Q3. What are Singly Linked Lists? Write an algorithm for implementation of a Singly Linked List.

[10 marks · D22]

A **singly linked list** is a linear collection of nodes in which each node contains a data field and a single pointer to the next node. The list is reached through an external pointer head; the last node's pointer is NULL. Nodes are allocated dynamically, so the list grows and shrinks at run time, insertion and deletion need no element shifting, but access is strictly sequential — the k-th element costs O(k) — and one pointer of extra space is paid per element.

```

struct node { int data; struct node *next; };
struct node *head = NULL;

struct node *create(int x)
{
    struct node *p = (struct node *) malloc(sizeof(struct node));
    p->data = x; p->next = NULL;
    return p;
}

void insertAtEnd(int x)                /* build the list */
{
    struct node *p = create(x), *q;
    if (head == NULL) { head = p; return; }
    for (q = head; q->next != NULL; q = q->next) ;
    q->next = p;
}

void insertAtBeginning(int x)
{
    struct node *p = create(x);
    p->next = head; head = p;
}

void deleteNode(int key)
{
    struct node *curr = head, *prev = NULL;
    if (head == NULL) { printf("Empty"); return; }
    if (head->data == key) { head = head->next; free(curr); return; }
    while (curr != NULL && curr->data != key) { prev = curr; curr = curr->next; }
    if (curr == NULL) { printf("Not found"); return; }
    prev->next = curr->next;
    free(curr);
}

struct node *search(int key)
{
    struct node *p = head;
    while (p != NULL && p->data != key) p = p->next;
    return p;                /* NULL if absent */
}

void display(void)
{
    struct node *p;
    for (p = head; p != NULL; p = p->next) printf("%d -> ", p->data);
    printf("NULL");
}

```

Example: insertAtEnd 10, 20, 30 gives head → 10 → 20 → 30 → NULL; insertAtBeginning 5 gives 5 → 10 → 20 → 30 → NULL; deleteNode(20) gives 5 → 10 → 30 → NULL.

Complexity: insertAtBeginning O(1); insertAtEnd, delete, search, display O(n); space O(n).

Q4. What are singly linked lists? Write an algorithm to count the number of items stored in a single linked list. [10 marks · J25]

A singly linked list is a chain of dynamically allocated nodes, each holding data and a pointer to its successor, with the last node pointing to NULL and the whole list accessed through the external pointer `head`. Because no count is stored in the structure, the number of items must be found by traversal.

Assumptions: head is NULL for an empty list; the traversal terminates on NULL.

```
struct node { int data; struct node *next; };
struct node *head = NULL;

int countItems(void)
{
    struct node *p = head;
    int count = 0;

    while (p != NULL)          /* terminate on NULL, not on a key */
    {
        count = count + 1;
        p = p->next;
    }
    return count;
}
```

Recursive version:

```
int countRec(struct node *p)
{
    if (p == NULL) return 0;
    return 1 + countRec(p->next);
}
```

Trace on head → 12 → 7 → 9 → 4 → NULL: p = 12 (count 1), p = 7 (2), p = 9 (3), p = 4 (4), p = NULL → loop ends, returns 4. For an empty list the loop body never executes and 0 is returned, which is the boundary case to state.

Time complexity $O(n)$ — every node is visited exactly once. Space complexity $O(1)$ for the iterative version, $O(n)$ stack for the recursive one. If counting is frequent, keep an integer `length` in a list header and update it on every insertion and deletion, making the count $O(1)$ — a space-time trade-off.

Q5. What is Link List? Write algorithms for inserting and deleting element in link list.

[10 marks · D25]

A **linked list** is a linear data structure whose elements, called nodes, are not stored contiguously; each node holds the data and the address of the next node. The list is accessed through an external pointer `head` and terminated by a NULL link. Its advantages over an array are dynamic size and $O(1)$ splicing; its costs are sequential access and one extra pointer per node.

```

struct node { int data; struct node *next; };
struct node *head = NULL;

/* ----- INSERTION ----- */
void insert(int x, int pos)      /* pos = 1 means insert at the beginning */
{
    struct node *p, *q;  int i;

    p = (struct node *) malloc(sizeof(struct node));
    if (p == NULL) { printf("Overflow"); return; }
    p->data = x;  p->next = NULL;

    if (pos == 1)                /* at the beginning */
    {
        p->next = head;
        head = p;
        return;
    }

    q = head;
    for (i = 1; i < pos - 1 && q != NULL; i++)  /* reach node pos-1 */
        q = q->next;

    if (q == NULL) { printf("Invalid position"); free(p); return; }

    p->next = q->next;                /* link forward first */
    q->next = p;                      /* then link backward */
}

/* ----- DELETION ----- */
void delete(int key)
{
    struct node *curr = head, *prev = NULL;

    if (head == NULL) { printf("Underflow: list empty"); return; }

    if (head->data == key)            /* first node */
    {
        head = head->next;
        free(curr);
        return;
    }

    while (curr != NULL && curr->data != key)
    {
        prev = curr;
        curr = curr->next;
    }
    if (curr == NULL) { printf("Element not found"); return; }

    prev->next = curr->next;          /* bypass and release */
    free(curr);
}

```

Example: empty list, insert(10, 1) → 10; insert(20, 2) → 10 → 20; insert(15, 2) → 10 → 15 → 20; delete(15) → 10 → 20; delete(10) → 20 (head moved).

Complexity: insertion at the head O(1), at position k O(k); deletion O(n) in general, O(1) at the head; auxiliary space O(1).

Q6. What are Doubly Linked Lists? Write an algorithm for implementation of a Doubly Linked List. [10 marks · asked 2×: J23, D23]

A **doubly linked list** is a linked list in which every node carries two pointers — `prev` to its predecessor and `next` to its successor — besides the data. The first node's `prev` and the last node's `next` are NULL. The list can be traversed in both directions, and a node can be deleted in $O(1)$ if a pointer to it is already held, because the predecessor is directly reachable. The costs are one extra pointer per node and the need to update two links on every side of a splice.

```

struct node {
    int data;
    struct node *prev, *next;
};
struct node *head = NULL;

struct node *create(int x)
{
    struct node *p = (struct node *) malloc(sizeof(struct node));
    p->data = x; p->prev = NULL; p->next = NULL;
    return p;
}

void insertAtBeginning(int x)
{
    struct node *p = create(x);
    if (head == NULL) { head = p; return; }
    p->next = head;
    head->prev = p;
    head = p;
}

void insertAtEnd(int x)
{
    struct node *p = create(x), *q;
    if (head == NULL) { head = p; return; }
    for (q = head; q->next != NULL; q = q->next) ;
    q->next = p;
    p->prev = q;
}

void insertAfter(int key, int x)
{
    struct node *p, *q = head;
    while (q != NULL && q->data != key) q = q->next;
    if (q == NULL) { printf("Key not found"); return; }
    p = create(x);
    p->next = q->next;
    p->prev = q;
    if (q->next != NULL) q->next->prev = p; /* only if not the tail */
    q->next = p;
}

void deleteNode(int key)
{
    struct node *p = head;
    while (p != NULL && p->data != key) p = p->next;
    if (p == NULL) { printf("Not found"); return; }
    if (p->prev != NULL) p->prev->next = p->next; else head = p->next;
    if (p->next != NULL) p->next->prev = p->prev;
    free(p);
}

void displayForward(void)
{
    struct node *p;
    for (p = head; p != NULL; p = p->next) printf("%d ", p->data);
}

```

Example: insertAtEnd 10, 20, 30 gives $\text{NULL} \leftarrow 10 \rightleftharpoons 20 \rightleftharpoons 30 \rightarrow \text{NULL}$. insertAfter(20, 25) sets 25's next to 30, 25's prev to 20, 30's prev to 25 and 20's next to 25 — four pointer assignments, all required.

Complexity: insertion at the head $O(1)$, at the end or after a key $O(n)$; deletion $O(n)$ to find plus $O(1)$ to splice; space $O(n)$ with 2 pointers per node.

Q7. Write algorithm for deleting element from a doubly linked list. [5 marks · J26]

Assumptions: doubly linked list with external pointer `head`; node has `data`, `prev`, `next`; the first matching value is deleted.

```
void deleteNode(int key)
{
    struct node *p = head;

    if (head == NULL) { printf("List empty"); return; }

    while (p != NULL && p->data != key) p = p->next;
    if (p == NULL) { printf("Element not found"); return; }

    if (p->prev == NULL)          /* deleting the first node */
        head = p->next;
    else
        p->prev->next = p->next;

    if (p->next != NULL)          /* not the last node */
        p->next->prev = p->prev;

    free(p);
}
```

The three boundary cases are: deleting the head (move head forward, new head's `prev` becomes `NULL`); deleting the tail (predecessor's `next` becomes `NULL`); deleting the only node (head becomes `NULL`). Both links must be repaired on each side of the removed node.

Example: $10 \rightleftharpoons 20 \rightleftharpoons 30$, delete 20 $\rightarrow$ 10's next becomes 30, 30's prev becomes 10, giving $10 \rightleftharpoons 30$.

Time complexity $O(n)$ to locate the node, $O(1)$ for the splice; space $O(1)$.

Q8. What are circular linked lists? Write an algorithm for insertion of an element into a circular linked list. [10 marks · J24]

A **circular linked list** is a linked list in which the last node points back to the first node instead of to `NULL`, so the list forms a ring and every node is reachable from every other node. Traversal must therefore terminate on the test `p->next == head`, never on `p == NULL` — this is the defining difference and the usual source of infinite loops. It suits round-robin scheduling, a cyclic buffer, and any application that must cycle repeatedly through a list.

Assumptions: singly circular list; `head` points to the first node; `head == NULL` means empty.

```

struct node { int data; struct node *next; };
struct node *head = NULL;

void insertAtBeginning(int x)
{
    struct node *p, *last;
    p = (struct node *) malloc(sizeof(struct node));
    p->data = x;

    if (head == NULL)                /* empty list: node points to itself */
    {
        p->next = p;
        head = p;
        return;
    }
    last = head;
    while (last->next != head) last = last->next; /* find last node */
    p->next = head;
    last->next = p;                /* close the ring */
    head = p;
}

void insertAtEnd(int x)
{
    struct node *p, *last;
    p = (struct node *) malloc(sizeof(struct node));
    p->data = x;

    if (head == NULL) { p->next = p; head = p; return; }

    last = head;
    while (last->next != head) last = last->next;
    last->next = p;
    p->next = head;                /* ring stays closed */
}

void insertAfter(int key, int x)
{
    struct node *p, *q = head;
    if (head == NULL) { printf("List empty"); return; }
    do {
        if (q->data == key) break;
        q = q->next;
    } while (q != head);
    if (q->data != key) { printf("Key not found"); return; }
    p = (struct node *) malloc(sizeof(struct node));
    p->data = x;
    p->next = q->next;
    q->next = p;
}

```

Example: insertAtEnd 10 → 10 points to itself. insertAtEnd 20 → 10 → 20 → back to 10. insertAtBeginning 5 → 5 → 10 → 20 → back to 5, with the last node 20 relinked to the new head 5.

Complexity: O(1) if a `last` pointer is maintained; O(n) as written because the last node must be located; space O(1) auxiliary.

Q9. What are circular linked lists? What is the procedure for inserting a new node at the end of circular linked list? [10 marks · D24]

A circular linked list is a list whose last node's pointer holds the address of the first node, forming a closed ring with no NULL link. Every traversal loop must stop on `p->next == head`.

Procedure for insertion at the end: 1. Allocate the new node and store the data in it. 2. If the list is empty (`head == NULL`), set `newnode->next = newnode` and `head = newnode`; the single node is its own successor. Stop. 3. Otherwise start at `head` and advance a pointer `last` while `last->next != head`, so that `last` ends on the current final node. 4. Set `last->next = newnode` — the old `last` node now points to the new node. 5. Set `newnode->next = head` — the ring is closed again. 6. `head` is **not** changed, because the new node went to the tail.

```
void insertAtEnd(int x)
{
    struct node *newnode, *last;

    newnode = (struct node *) malloc(sizeof(struct node));
    if (newnode == NULL) { printf("Memory not available"); return; }
    newnode->data = x;

    if (head == NULL)                /* step 2: empty list */
    {
        newnode->next = newnode;
        head = newnode;
        return;
    }

    last = head;                     /* step 3: locate last node */
    while (last->next != head)
        last = last->next;

    last->next = newnode;             /* step 4 */
    newnode->next = head;             /* step 5: close the ring */
}
```

Example: ring $10 \rightarrow 20 \rightarrow 30 \rightarrow (10)$. Insert 40: `last` walks 10, 20, 30 and stops at 30 because `30->next == head`; `30->next = 40` and `40->next = 10`, giving $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow (10)$.

If the implementation keeps an external pointer to the last node rather than the first, insertion at the end becomes $O(1)$: `newnode->next = last->next`; `last->next = newnode`; `last = newnode`; . As written above the traversal makes it $O(n)$ time, $O(1)$ space.

Searching

Q1. What is Linear Search? Write an algorithm for it. [10 marks · asked 2×: J23, D23]

Linear search (sequential search) examines the elements of a list one after another, from the first to the last, until the required key is found or the list is exhausted. It makes no assumption about the order of the data, so it works on unsorted arrays, linked lists and files, which is its only advantage over binary search.

```

int linearSearch(int a[], int n, int key)
{
    int i;
    for (i = 0; i < n; i++)
    {
        if (a[i] == key)
            return i;          /* found: return position */
    }
    return -1;                 /* not found */
}

```

Linked-list version:

```

struct node *linearSearchList(struct node *head, int key)
{
    struct node *p = head;
    while (p != NULL && p->data != key) p = p->next;
    return p;                  /* NULL if absent */
}

```

Example: $a = \{17, 4, 29, 8, 11\}$, $key = 8$. Compare $17 \neq 8$, $4 \neq 8$, $29 \neq 8$, $8 = 8 \rightarrow$ return index 3 after 4 comparisons. For $key = 5$ all 5 comparisons fail and -1 is returned.

Complexity. Basic operation: the comparison $a[i] == key$. - Best case: key at $a[0]$, 1 comparison $\rightarrow O(1)$. - Worst case: key at $a[n-1]$ or absent, n comparisons $\rightarrow O(n)$. - Average case (key equally likely at any of the n positions): $(1 + 2 + \dots + n)/n = (n+1)/2 \rightarrow O(n)$.

Space complexity $O(1)$. An improvement for unsorted data is the sentinel version, which stores the key at $a[n]$ and drops the loop-bound test, halving the comparisons per iteration without changing the $O(n)$ order.

Q2. What is Linear Search? How does it differ from Binary Search? [10 marks · J22]

Linear search scans the list sequentially from the first element until the key is found or the list ends; it requires no ordering of the data. Binary search works only on a **sorted** list: it compares the key with the middle element and discards one half of the list at each step.

Point	Linear search	Binary search
Precondition	None — data may be unsorted	Data must be sorted
Method	Sequential scan element by element	Repeated halving of the interval
Best case	1 comparison, $O(1)$	1 comparison, $O(1)$
Average / worst case	$(n+1)/2$ / n comparisons, $O(n)$	about $\log_2 n$ comparisons, $O(\log n)$
Data structure	Array, linked list, file	Array or any structure giving $O(1)$ access to the middle
Comparisons for $n = 1024$	up to 1024	at most 11
Insert/delete cost on the structure	Cheap, order need not be kept	Expensive, order must be maintained
Implementation	Very simple, one loop	Needs low, high, mid and a three-way compare
Best used for	Small or unsorted or frequently changing data	Large, static, sorted data

Example on $a = \{11, 22, 33, 44, 55, 66, 77, 88\}$, $key = 66$. Linear search needs 6 comparisons. Binary search: $low = 0$, $high = 7$, $mid = 3$ ($44 < 66$) $\rightarrow low = 4$; $mid = 5$ ($66 = 66$) $\rightarrow$ found in 2 comparisons.

Conclusion: binary search is asymptotically far better but the cost of keeping the data sorted may outweigh the saving when the data changes often or n is small.

Q3. What is Binary Search? Write an algorithm for it. [10 marks · D22]

Binary search finds a key in a list that is already **sorted in ascending order** (this precondition must be stated before the code). It compares the key with the middle element: if they are equal the search succeeds; if the key is smaller the whole upper half is discarded; if larger, the lower half is discarded. Each step halves the search space.

```
int binarySearch(int a[], int n, int key)
{
    int low = 0, high = n - 1, mid;

    while (low <= high)
    {
        mid = (low + high) / 2;
        if (key == a[mid])
            return mid;           /* success */
        else if (key < a[mid])
            high = mid - 1;       /* discard the right half */
        else
            low = mid + 1;        /* discard the left half */
    }
    return -1;                   /* key not present */
}
```

Recursive version:

```
int binarySearchRec(int a[], int low, int high, int key)
{
    int mid;
    if (low > high) return -1;
    mid = (low + high) / 2;
    if (key == a[mid]) return mid;
    else if (key < a[mid]) return binarySearchRec(a, low, mid - 1, key);
    else return binarySearchRec(a, mid + 1, high, key);
}
```

Example: $a = \{10, 20, 30, 40, 50, 60, 70, 80\}$, $key = 70$ — $low\ 0/high\ 7/mid\ 3$ ($40 < 70$) $\rightarrow low\ 4$; $low\ 4/high\ 7/mid\ 5$ ($60 < 70$) $\rightarrow low\ 6$; $low\ 6/high\ 7/mid\ 6$ ($70 = 70$) $\rightarrow$ return 6.

Complexity: the interval length after k iterations is $n/2^k$, and the loop ends when $n/2^k = 1$, so $k = \log_2 n \rightarrow O(\log_2 n)$ worst and average, $O(1)$ best. Space $O(1)$ iterative, $O(\log n)$ stack recursive.

Q4. Write binary search algorithm. [5 marks · D25]

Precondition: the array $a[0..n-1]$ must be sorted in ascending order.

```
int binarySearch(int a[], int n, int key)
{
    int low = 0, high = n - 1, mid;

    while (low <= high)
    {
        mid = low + (high - low) / 2;    /* avoids overflow */
        if (a[mid] == key) return mid;
        if (key < a[mid]) high = mid - 1;
        else low = mid + 1;
    }
    return -1;
}
```

Example on $\{2, 5, 8, 12, 16, 23, 38\}$, $\text{key} = 23$: $\text{mid} = 3 \rightarrow 12 < 23$, $\text{low} = 4$; $\text{mid} = 5 \rightarrow 23$ found at index 5, in 2 comparisons rather than 6.

Time complexity $O(\log_2 n)$ worst case, $O(1)$ best; space complexity $O(1)$.

Q5. What is Binary Search? Explain it with an example. [10 marks · D21]

Binary search is a divide-and-conquer search method for a list **sorted in ascending order**. Three indices are used: low (first candidate), high (last candidate) and $\text{mid} = (\text{low} + \text{high})/2$. The key is compared with $a[\text{mid}]$ and one of three things happens: equality ends the search; $\text{key} < a[\text{mid}]$ means the key, if present, lies in $a[\text{low}..\text{mid}-1]$ so $\text{high} = \text{mid} - 1$; $\text{key} > a[\text{mid}]$ means it lies in $a[\text{mid}+1..\text{high}]$ so $\text{low} = \text{mid} + 1$. The loop ends when the key is found or when $\text{low} > \text{high}$, which proves the key is absent.

Successful search. $a = \{11, 22, 33, 44, 55, 66, 77, 88\}$, $n = 8$, $\text{key} = 22$.

Step	low	high	mid	a[mid]	Comparison	Action
1	0	7	3	44	$22 < 44$	$\text{high} = 2$
2	0	2	1	22	$22 = 22$	found at index 1

Found in 2 comparisons; linear search would have needed 2 as well here, but for $\text{key} = 77$: $\text{mid} 3$ ($44 < 77$) $\rightarrow \text{low} 4$; $\text{mid} 5$ ($66 < 77$) $\rightarrow \text{low} 6$; $\text{mid} 6$ (77) $\rightarrow$ found in 3 comparisons against 7 for linear search.

Unsuccessful search. Same array, $\text{key} = 50$.

Step	low	high	mid	a[mid]	Comparison	Action
1	0	7	3	44	$50 > 44$	$\text{low} = 4$
2	4	7	5	66	$50 < 66$	$\text{high} = 4$
3	4	4	4	55	$50 < 55$	$\text{high} = 3$
4	$\text{low} = 4 > \text{high} = 3$					return -1

Complexity: after k halvings the interval holds $n/2^k$ elements and the search ends at 1, so $k = \log_2 n \rightarrow O(\log n)$; for $n = 1,000,000$ at most 20 comparisons are needed. Space $O(1)$. Limitation: the

array must be sorted and must support $O(1)$ indexing, so binary search is unsuitable for a linked list.

Hashing

Q1. What is Hashing? Write a short note on it. [10 marks · D22]

Hashing is a technique that computes the address of a record directly from its key, so that a record can be stored and retrieved in $O(1)$ average time without searching. A **hash function** h maps a key k to an index $h(k)$ in the range $0..m-1$ of a table of m slots, called the **hash table**.

Hash functions. - **Division method:** $h(k) = k \bmod m$. Simple and the most used; m should be prime and not a power of 2, so that all bits of the key influence the address. - **Mid-square method:** square the key and take the middle digits, e.g. $k = 3121 \rightarrow k^2 = 9740641 \rightarrow$ middle three digits 406. - **Folding method:** split the key into parts of equal length and add them, e.g. $123456789 \rightarrow 123 + 456 + 789 = 1368 \rightarrow h = 1368 \bmod m$. - **Multiplication method:** $h(k) = \lfloor m \cdot (k \cdot A \bmod 1) \rfloor$ with $0 < A < 1$.

Load factor $\alpha = n/m$, the ratio of stored records to slots. The expected cost of a successful search is about $(1 + \alpha)/2$ for chaining; performance degrades as α approaches 1, so the table is usually rehashed into a larger table when α exceeds about 0.7.

Collision. Two distinct keys with the same hash value collide; collisions are unavoidable when the key space exceeds m . They are resolved by **open addressing** (linear probing, quadratic probing, double hashing) or **chaining** (a linked list per slot).

Example, $m = 10$, $h(k) = k \bmod 10$ and keys 12, 44, 13, 88, 23, 94, 11, 39: $h = 2, 4, 3, 8, 3, 4, 1, 9$ — 23 collides with 13 and 94 with 44.

Applications: symbol tables in compilers, database indexes, caches, dictionaries and sets, password storage, direct file organisation. Advantage: $O(1)$ average insert, search and delete. Disadvantage: no ordered traversal, worst case $O(n)$ if every key collides, and extra space must be left free.

Q2. What is hashing? How is a hash function utilized to map data into a hash table? Explain with an example. [10 marks · D24]

Hashing converts a key into a table address by arithmetic rather than by comparison, giving $O(1)$ average access. The three parts are the **key**, the **hash function** h , and the **hash table** of m slots.

The mapping process: 1. Choose the table size m (a prime, with some slack so the load factor stays below about 0.7). 2. Choose a hash function that is fast to compute and distributes keys uniformly over $0..m-1$. 3. For each key compute $i = h(k)$. 4. If slot i is free, store the record there. 5. If slot i is occupied by a different key, a collision has occurred; resolve it by probing or chaining. 6. Search repeats steps 3-5: compute $h(k)$ and inspect the slot, following the same probe sequence or chain until the key or an empty slot is found.

Worked example. $m = 10$, $h(k) = k \bmod 10$, keys inserted in the order 12, 44, 13, 88, 23, 94, 11, 39.

$h(12) = 2$, $h(44) = 4$, $h(13) = 3$, $h(88) = 8$, $h(23) = 3$ (collision with 13), $h(94) = 4$ (collision with 44), $h(11) = 1$, $h(39) = 9$.

Resolved by **linear probing**, $h'(k, i) = (h(k) + i) \bmod 10$:

Slot	0	1	2	3	4	5	6	7	8	9
Key	—	11	12	13	44	23	94	—	88	39

23 probed slot 3 (busy), 4 (busy), 5 (free) — 2 extra probes; 94 probed 4, 5, 6 — 2 extra probes.

Resolved by **chaining**, each slot heading a linked list:

```
0 : NULL
1 : 11 -> NULL
2 : 12 -> NULL
3 : 13 -> 23 -> NULL
4 : 44 -> 94 -> NULL
8 : 88 -> NULL
9 : 39 -> NULL
```

Searching 23: compute $h = 3$, walk the chain at slot 3, compare 13 then 23 → found in 2 comparisons. Load factor $\alpha = 8/10 = 0.8$, which is already high for open addressing.

Q3. Explain what is collision in the context of 'hashing'. What methods can be employed to address collisions? [10 marks · J25]

A **collision** occurs when two distinct keys $k_1 \neq k_2$ hash to the same slot, $h(k_1) = h(k_2)$. Because the universe of keys is normally much larger than the table size m , the pigeonhole principle makes collisions unavoidable; a good hash function reduces their frequency but cannot eliminate them. The probability rises with the load factor $\alpha = n/m$.

Example, $m = 10$, $h(k) = k \bmod 10$: keys 13 and 23 both map to slot 3, and 44 and 94 both map to slot 4.

Family 1 — Open addressing (closed hashing). All records live in the table itself; on a collision a probe sequence looks for another free slot. - **Linear probing:** $h'(k, i) = (h(k) + i) \bmod m$, $i = 0, 1, 2, \dots$ Simple and cache-friendly, but suffers **primary clustering**: long runs of occupied slots that lengthen all later probes. - **Quadratic probing:** $h'(k, i) = (h(k) + c_1i + c_2i^2) \bmod m$. Removes primary clustering but causes **secondary clustering** (keys with the same home slot follow the same path) and may not visit every slot. - **Double hashing:** $h'(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$ with $h_2(k)$ never 0 and relatively prime to m , e.g. $h_2(k) = 1 + (k \bmod (m-1))$. The best probe distribution of the three; behaves almost like uniform hashing. - Deletion needs a "deleted" marker, otherwise a probe chain is broken.

Inserting 12, 44, 13, 88, 23, 94 with linear probing gives: slot 2 → 12, 4 → 44, 3 → 13, 8 → 88, 23 → slot 5, 94 → slot 6.

Family 2 — Chaining (open hashing). Each slot holds the head of a linked list of all keys that hash to it.

```
3 : 13 -> 23 -> NULL
4 : 44 -> 94 -> NULL
```

Point	Open addressing	Chaining
Storage	Records inside the table	Table of pointers plus nodes
Load factor	Must stay < 1	May exceed 1
Deletion	Needs tombstones	Simple unlink
Extra space	None	One pointer per node
Worst case search	$O(n)$ with clustering	$O(n)$, length of one chain
Average successful search	$\approx (1/\alpha) \cdot \ln(1/(1-\alpha))$	$1 + \alpha/2$

Other techniques: **separate overflow area** (a common overflow block, used in direct file organisation), **coalesced hashing**, **rehashing** into a larger prime-sized table when α grows, and **bucket hashing** where each slot holds several records.

File organisation

Q1. What is need of file organisation? What are key drivers for file organization? Explain indexed sequential file organization. [10 marks · D25]

Need. A file holds far more records than can be kept in main memory, and every access costs a disk seek, which is millions of times slower than a memory reference. File organisation is the arrangement of records in secondary storage, together with the access method used to reach them, chosen so that the number of block transfers for the expected pattern of operations is as small as possible. Without a deliberate organisation, finding one record means reading the whole file.

Key drivers for choosing an organisation: 1. **Access type** required — sequential, random, or both. 2. **Volume and volatility** — file size, and how often records are inserted or deleted. 3. **Activity ratio** — the fraction of records touched in a typical run; high ratios favour sequential, low ratios favour direct. 4. **Response time** requirement — batch processing versus on-line enquiry. 5. **Storage cost and space utilisation**, including room for overflow. 6. **Ease of update and maintenance** (reorganisation cost). 7. **Storage device characteristics** — tape permits only sequential access, disk permits direct access.

Indexed sequential file organisation (ISAM). Records are stored sequentially, physically ordered on the primary key, and a separate **index** is maintained mapping key values to block addresses. The index is itself ordered and often multi-level (a master index pointing to cylinder indexes pointing to track indexes).

INDEX (sparse, one entry per data block)	DATA AREA (sorted on key)
+-----+-----+	+-----+
key 100 blk 1 ----->	010 030 070 100 blk 1
key 240 blk 2 ----->	130 180 200 240 blk 2
key 400 blk 3 ----->	290 310 360 400 blk 3
+-----+-----+	+-----+
	OVERFLOW AREA 155 ...

To read record 180: search the index, find that $180 \leq 240$ so it lies in block 2, read block 2 and search inside it — two block accesses instead of scanning the file.

It supports **both** sequential processing (read the data area in order, for reports) and random access (through the index) — the property that distinguishes it from the other two organisations. Insertions go into an **overflow area** linked from the home block; as overflow grows, access degrades and the file must be periodically **reorganised**.

Advantages: both access modes, fast random retrieval, keys kept in order. Disadvantages: index storage and maintenance overhead, degradation from overflow chains, periodic reorganisation needed, and access is still slower than direct hashing for pure random retrieval.

Q2. What is Direct File Organization (DFO)? Explain the use of hashing and hash function in DFO with the help of an example. [10 marks · J26]

Direct (random, or hashed) file organisation stores each record at a disk address computed directly from its key, so a record is retrieved in essentially one block access with no index to search and no sequential scanning. There is no relationship between the key order and the physical order of records, which is the price paid for the speed.

Role of hashing. A **hash function** h maps the record key to a **bucket address** in the range $0..m-1$, where a bucket is one block (or a few blocks) holding several records. The steps are:

1. The file is allocated m buckets, each able to hold b records.
2. For a record with key k , compute the bucket address $i = h(k)$, commonly by the division method $h(k) = k \bmod m$ with m prime.
3. Read bucket i ; if it has room, write the record there.
4. If bucket i is full, a **bucket overflow** has occurred; the record goes into an overflow area and is linked from bucket i (chained overflow), or to the next free bucket (open addressing).
5. To retrieve a record, compute $h(k)$, read bucket i , and search the few records inside it; if not found, follow the overflow chain.
6. Deletion removes the record from the bucket and relinks the chain.

Example. $m = 10$ buckets numbered $0..9$, bucket capacity $b = 2$ records, $h(k) = k \bmod 10$. Insert employee numbers 1025, 1038, 1042, 1055, 1062, 1072, 1048, 1082.

Key	$h(k) = k \bmod 10$	Bucket	Note
1025	5	5	slot 1
1038	8	8	slot 1
1042	2	2	slot 1
1055	5	5	slot 2 — bucket 5 now full
1062	2	2	slot 2 — bucket 2 now full
1072	2	2	overflow → overflow area, chained from bucket 2
1048	8	8	slot 2
1082	2	2	overflow → chained after 1072

```
bucket 2 : [1042 | 1062] --> overflow: [1072 | 1082]
bucket 5 : [1025 | 1055]
bucket 8 : [1038 | 1048]
```

Retrieving 1082: $h = 2$, read bucket 2, compare 1042 and 1062, follow the overflow pointer, compare 1072 then 1082 → found in 2 block accesses.

Advantages: fastest random access (close to one access), no index to store or maintain, suitable for on-line enquiry such as banking and reservation systems. Disadvantages: sequential processing in key order is not possible without sorting the whole file, space is wasted in partly filled buckets, performance degrades as buckets overflow (keep the load factor below about 0.7–0.8), and the file must be reorganised when it grows.

Q3. What is sequential file organization? Explain its advantages. [5 marks · J26]

In **sequential file organisation** records are written one after another in the physical order in which they arrive, or sorted on a key field, and must be read in that same order from the start of the file. There is no index and no computed address; to reach the k -th record the first $k-1$ must be passed. New records are appended, or the file is merged with a sorted transaction file to produce a new master file; deletion is done by marking the record and recovering the space at the next reorganisation.

Advantages: - Simplest organisation to design, implement and understand. - No index or pointer overhead, so storage utilisation is the highest of all organisations. - Very efficient when the **activity ratio is high**, i.e. most records are processed in one run, as in payroll, billing or monthly report generation. - Works on any storage device, including magnetic tape, which supports nothing else. - Sequential reading of consecutive blocks is the fastest possible transfer mode on disk, since no seeks are needed between blocks. - If the file is sorted on the key, ordered reports need no further sorting.

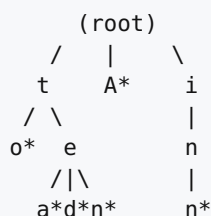
Disadvantages, for contrast: random access to a single record costs $O(n)$ block reads on average, insertion in key order requires rewriting the file, and it is unsuitable for on-line enquiry.

Asked once

Q1. What are Tries? List the main characteristics of tries. [10 marks · J24]

A **trie** (from *retrieval*, also called a prefix tree or digital tree) is an ordered tree used to store a set of strings, in which the position of a node in the tree, not any value stored in it, encodes the key. The root represents the empty string; every edge is labelled with one character, and the key associated with a node is the concatenation of the edge labels on the path from the root. A node is marked as terminal if the string spelled out on its path is a key in the set.

Example storing {to, tea, ted, ten, A, inn}: the root has children **t**, **A**, **i**; **t** has child **o** (terminal: "to") and child **e**, which has children **a**, **d**, **n** (terminal: "tea", "ted", "ten").



Main characteristics: 1. Keys are stored by **path**, not inside nodes; no node holds a complete key. 2. All descendants of a node share the **common prefix** represented by that node, so a trie is the natural structure for prefix queries. 3. The root represents the empty string and is at level 0; a node at level k represents a prefix of length k . 4. Search, insertion and deletion take **$O(m)$** time for a key of length m , independent of the number n of keys stored — unlike a BST, which costs $O(m \log n)$. 5. Each node has up to R children where R is the alphabet size (26, or 128 for ASCII),

stored as an array or a map; the array form is fast but space-hungry. 6. No collisions, and keys come out in **lexicographic order** on a pre-order traversal — an advantage over hashing. 7. Space can be large because of null child pointers; compressed variants (**Patricia trie / radix tree**, where chains of single-child nodes are merged, and the **suffix tree**) fix this. 8. Terminal nodes are flagged, and a key may be a prefix of another key.

Applications: dictionary and spell-checker lookup, auto-complete and predictive text, longest-prefix matching in IP routing tables, T9 input, and string-matching with suffix tries.

Q2. What is Red-Black tree? Write its properties. [5 marks · J26]

A **Red-Black tree** is a self-balancing binary search tree in which each node carries one extra bit of colour, red or black. The colouring rules keep the height within $2 \cdot \log_2(n+1)$, so search, insertion and deletion are all $O(\log n)$ in the worst case. Balance is restored after an insertion or deletion by recolouring and by at most two rotations.

Properties: 1. Every node is either **red** or **black**. 2. The **root is black**. 3. Every leaf (the NIL sentinel) is **black**. 4. A red node cannot have a red child — **no two consecutive red nodes** on any path (so a red node's parent and children are black). 5. For every node, all simple paths from that node down to its descendant NIL leaves contain the **same number of black nodes** (its black height). 6. Consequently the height of a tree of n internal nodes is at most $2 \cdot \log_2(n+1)$, and the longest root-to-leaf path is at most twice the shortest.

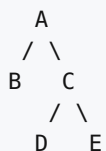
Compared with an AVL tree, a Red-Black tree is less strictly balanced, so lookups are slightly slower but insertions and deletions need fewer rotations; it is the structure behind C++ `std::map`, Java `TreeMap` and many Linux kernel schedulers.

Q3. What is a full tree? What is a complete tree? Explain the implementation of a tree using linked list. [10 marks · J24]

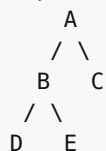
A **full (strictly binary) tree** is a binary tree in which every node has either 0 or 2 children — no node has exactly one child. A full binary tree with n internal nodes has $n+1$ leaves and $2n+1$ nodes in all.

A **complete binary tree** is a binary tree in which every level is completely filled except possibly the last, and the nodes on the last level are filled from left to right. A complete binary tree of height h has between 2^h and $2^{h+1}-1$ nodes, and its height is $\lfloor \log_2 n \rfloor$. A **perfect** binary tree is both full and complete, with exactly $2^{h+1}-1$ nodes.

Full (not complete)



Complete (not full)



Implementation using a linked list. Each node is a dynamically allocated record holding the data and pointers to its children; the tree is reached through an external pointer `root`, and an absent child is NULL. For a binary tree, two pointers suffice:

```

struct node {
    int data;
    struct node *left, *right;
};
struct node *root = NULL;

struct node *newNode(int x)
{
    struct node *p = (struct node *) malloc(sizeof(struct node));
    p->data = x; p->left = p->right = NULL;
    return p;
}

struct node *insert(struct node *t, int x)    /* BST-style insertion */
{
    if (t == NULL) return newNode(x);
    if (x < t->data) t->left = insert(t->left, x);
    else            t->right = insert(t->right, x);
    return t;
}

void inorder(struct node *t)
{
    if (t != NULL) { inorder(t->left); printf("%d ", t->data); inorder(t->right); }
}

```

For a **general tree** of arbitrary degree the **left-child / right-sibling** representation is used, so that two pointers still suffice regardless of the number of children:

```

struct gnode {
    int data;
    struct gnode *firstChild, *nextSibling;
};

```

A node's children form a linked list threaded through `nextSibling`, reached from `firstChild`. This is exactly the transformation used to convert a general tree into a binary tree.

Advantages of the linked representation over the array (`left = 2i`, `right = 2i+1`) representation: no wasted cells for a skewed tree, and the tree grows dynamically. Disadvantage: pointer space, and no $O(1)$ access to a node by index. Traversal is $O(n)$ time with $O(h)$ stack space.

Q4. What is a B-tree and what are its properties? Create a B-tree of order 5, if the following list of keys are inserted into the empty B-tree in the order:
25, 10, 15, 28, 40, 30, 5, 2, 35, 42, 50 [10 marks · D24]

A **B-tree** of order m is a balanced multi-way search tree designed for disk storage, in which one node is one disk block, so that the height — and hence the number of block accesses per search — is minimised.

Properties of a B-tree of order m: 1. Every node holds at most $m-1$ keys and at most m child pointers. 2. Every node except the root and the leaves holds at least $\lceil m/2 \rceil$ children, i.e. at least $\lceil m/2 \rceil - 1$ keys. 3. The root has at least 2 children unless it is a leaf. 4. All leaves appear at the **same level** — the tree is perfectly height-balanced. 5. Keys within a node are in ascending order, and the subtree between two keys holds values lying between them. 6. A node with k keys has exactly $k+1$ children. 7. Growth is upward: an overflowing node is **split** and its median key is promoted to the parent; the height increases only when the root splits. 8. Search, insertion and deletion are all $O(\log \lceil m/2 \rceil n)$.

For **order 5**: maximum 4 keys per node, minimum 2 keys in a non-root node, split when a 5th key arrives, promoting the median.

Construction, step by step.

Insert 25, 10, 15, 28 — all go into the single root leaf, kept sorted:

```
[10 15 25 28]
```

Insert 40 — the node would hold 10 15 25 28 40, which is 5 keys. **Split** on the median 25, which is promoted to a new root:

```

      [25]
     /  \
  [10 15] [28 40]
```

Insert 30 — belongs in the right leaf:

```

      [25]
     /  \
  [10 15] [28 30 40]
```

Insert 5 — left leaf:

```

      [25]
     /  \
  [5 10 15] [28 30 40]
```

Insert 2 — left leaf, now full with 4 keys:

```

      [25]
     /  \
  [2 5 10 15] [28 30 40]
```

Insert 35 — right leaf, now full:

```

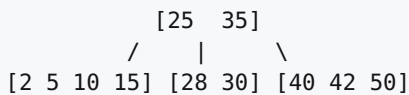
      [25]
     /  \
  [2 5 10 15] [28 30 35 40]
```

Insert 42 — the right leaf would hold 28 30 35 40 42, five keys. **Split** on the median 35, promoted into the root, which becomes [25 35]:

```

      [25 35]
     /  |  \
  [2 5 10 15] [28 30] [40 42]
```

Insert 50 — goes into the rightmost leaf:



Final B-tree of order 5: root [25, 35] with three leaves [2, 5, 10, 15], [28, 30] and [40, 42, 50]. Height 1, all leaves at the same level, every leaf holding between 2 and 4 keys — all the order-5 rules are satisfied. Two splits occurred, on inserting 40 and 42.

Q5. Write all pairs shortest paths algorithm. [10 marks · D23]

The all-pairs shortest path problem asks for the shortest distance between **every** ordered pair of vertices. Floyd-Warshall solves it by dynamic programming: it considers each vertex k in turn as a possible intermediate point and asks whether going $i \rightarrow k \rightarrow j$ is shorter than the best $i \rightarrow j$ path found so far.

Assumptions: n vertices numbered $0..n-1$; $\text{cost}[i][j]$ is the edge weight, 0 on the diagonal and INF (a very large number) where no edge exists; negative edges are allowed but no negative cycle.

```
#define INF 9999

void floydWarshall(int cost[][MAX], int n)
{
    int D[MAX][MAX], P[MAX][MAX], i, j, k;

    for (i = 0; i < n; i++)           /* initialise */
        for (j = 0; j < n; j++)
        {
            D[i][j] = cost[i][j];
            P[i][j] = (cost[i][j] != INF && i != j) ? i : -1; /* predecessor */
        }

    for (k = 0; k < n; k++)           /* intermediate vertex */
        for (i = 0; i < n; i++)       /* source */
            for (j = 0; j < n; j++)   /* destination */
                if (D[i][k] + D[k][j] < D[i][j])
                {
                    D[i][j] = D[i][k] + D[k][j];
                    P[i][j] = P[k][j];
                }

    for (i = 0; i < n; i++)
        if (D[i][i] < 0) { printf("Negative cycle present"); return; }
}
```

The recurrence is $D^k[i][j] = \min(D^{k-1}[i][j], D^{k-1}[i][k] + D^{k-1}[k][j])$, with D^0 = the cost matrix. After the k -loop finishes for all k , $D[i][j]$ is the shortest distance from i to j using any intermediate vertices. The path itself is recovered by following the predecessor matrix P backwards from j to i .

Example on 3 vertices with $\text{cost}(0 \rightarrow 1) = 4$, $\text{cost}(0 \rightarrow 2) = 11$, $\text{cost}(1 \rightarrow 2) = 2$: with $k = 1$, $D[0][2] = \min(11, 4 + 2) = 6$, so the matrix is updated and $P[0][2] = 1$.

Time complexity $O(n^3)$ with three nested loops; space complexity $O(n^2)$ for the two matrices. Unlike Dijkstra, Floyd-Warshall tolerates **negative edge weights** (though not negative cycles) and gives all pairs in one run; running Dijkstra from every vertex costs $O(n \cdot E \log n)$ and fails on negative edges.

Q6. Describe Dijkstra's algorithm for finding the shortest path in a weighted graph. How does it manage graph with negative edge costs? Explain with the help of an example. [10 marks · D24]

Dijkstra's algorithm finds the shortest paths from one source vertex s to every other vertex of a weighted graph with **non-negative** edge weights. It is greedy: it keeps a set S of vertices whose shortest distance is finalised, and repeatedly moves into S the unvisited vertex with the smallest tentative distance, then relaxes the edges leaving it.

```
void dijkstra(int cost[][MAX], int n, int s)
{
    int dist[MAX], pred[MAX], visited[MAX], i, j, u, min;

    for (i = 0; i < n; i++) { dist[i] = INF; pred[i] = -1; visited[i] = 0; }
    dist[s] = 0;

    for (i = 0; i < n - 1; i++)
    {
        min = INF; u = -1;
        for (j = 0; j < n; j++) /* pick nearest unvisited */
            if (!visited[j] && dist[j] < min) { min = dist[j]; u = j; }
        if (u == -1) break;
        visited[u] = 1; /* distance is now final */

        for (j = 0; j < n; j++) /* relax edges from u */
            if (!visited[j] && cost[u][j] != INF &&
                dist[u] + cost[u][j] < dist[j])
            {
                dist[j] = dist[u] + cost[u][j];
                pred[j] = u;
            }
    }
}
```

Negative edge costs. Dijkstra **fails** on graphs with negative edge weights. Its correctness rests on the assumption that once a vertex is removed from the queue with the minimum tentative distance, no shorter route to it can be found later — which holds only if every edge adds a non-negative amount. A negative edge can shorten a path through an already-finalised vertex, and the algorithm never revisits it.

Example: vertices A, B, C with $A \rightarrow B = 4$, $A \rightarrow C = 5$ and $C \rightarrow B = -3$. Dijkstra from A sets $\text{dist}[B] = 4$ and $\text{dist}[C] = 5$, then finalises B (distance 4) because $4 < 5$. Later C is processed and the edge $C \rightarrow B$ gives $5 + (-3) = 2 < 4$, but B is already in S and is never updated, so the algorithm reports $\text{dist}[B] = 4$ while the true shortest distance is 2.

How it is managed: use **Bellman-Ford** instead, which relaxes all E edges $V-1$ times in $O(V \cdot E)$ and additionally detects negative cycles, or **Floyd-Warshall** in $O(V^3)$ for all pairs; or apply **Johnson's algorithm**, which reweights edges with Bellman-Ford potentials to make them non-negative and then runs Dijkstra. Note that no algorithm can define a shortest path in a graph containing a **negative cycle**, since the path cost can be driven to $-\infty$.

Complexity of Dijkstra: $O(V^2)$ with the array implementation above, $O((V + E) \log V)$ with a binary heap; space $O(V)$.

Q7. What is meant by Minimum Cost Spanning Tree (MCST)? How can you find it? [10 marks · J22]

A **spanning tree** of a connected, undirected graph $G = (V, E)$ is a subgraph that is a tree and includes every vertex of G ; it has exactly $|V| - 1$ edges and contains no cycle. A **minimum cost spanning tree** is a spanning tree whose total edge weight is the least among all spanning trees of G . It need not be unique, but all MCSTs of a graph have the same total weight. A graph with n vertices can have up to $n^{(n-2)}$ spanning trees, so exhaustive search is not an option.

How it is found — two greedy algorithms.

Kruskal's algorithm (edge-based): 1. Sort all E edges in non-decreasing order of weight. 2. Start with a forest of $|V|$ single-vertex trees and an empty tree T . 3. Take the next cheapest edge; if its two endpoints lie in **different** components (tested with union-find), add it to T and union the components; otherwise discard it because it would form a cycle. 4. Stop when T has $|V| - 1$ edges. Complexity $O(E \log E)$ for the sort, which dominates.

Prim's algorithm (vertex-based): 1. Start from any vertex; let $S = \{\text{start}\}$. 2. Repeatedly choose the cheapest edge with one endpoint in S and the other outside S (the minimum edge crossing the cut), add it to T and add its far endpoint to S . 3. Stop when $S = V$, after $|V| - 1$ edges. Complexity $O(V^2)$ with an array, $O(E \log V)$ with a heap.

Worked example. Vertices A, B, C, D with weights $AB = 1, BC = 2, AC = 4, CD = 3, BD = 5$. Kruskal sorts the edges 1, 2, 3, 4, 5: take AB (1), take BC (2), take CD (3), now $3 = |V| - 1$ edges so stop; AC and BD would have created cycles. MCST = $\{AB, BC, CD\}$ with total cost 6. Prim starting at A : cheapest edge from $\{A\}$ is AB (1); from $\{A, B\}$ it is BC (2); from $\{A, B, C\}$ it is CD (3) — the same tree.

Applications: laying out a minimum-length network of cables, pipelines, roads or circuit wiring; cluster analysis; approximation algorithms for the travelling salesman problem; network broadcast routing.

Q8. Write algorithm for Depth First Search. [5 marks · J26]

DFS explores a graph by going as deep as possible along one branch before backtracking. It uses a **stack** (explicitly, or implicitly via recursion) and a **visited** array to avoid revisiting vertices in a cyclic graph.

```

int visited[MAX];                                /* all 0 initially */

void dfs(int adj[][MAX], int n, int v)           /* recursive */
{
    int w;
    visited[v] = 1;
    printf("%d ", v);                            /* visit v */
    for (w = 0; w < n; w++)
        if (adj[v][w] == 1 && !visited[w])
            dfs(adj, n, w);
}

void dfsIterative(int adj[][MAX], int n, int s) /* explicit stack */
{
    int stack[MAX], top = -1, v, w;
    stack[++top] = s;
    while (top != -1)
    {
        v = stack[top--];
        if (!visited[v])
        {
            visited[v] = 1; printf("%d ", v);
            for (w = n - 1; w >= 0; w--)
                if (adj[v][w] == 1 && !visited[w]) stack[++top] = w;
        }
    }
}

```

Example on edges A-B, A-C, B-D, C-D starting at A: visit A, go to B, go to D, D's neighbour C is unvisited so visit C, then backtrack — order A, B, D, C.

Time complexity $O(V + E)$ with an adjacency list, $O(V^2)$ with an adjacency matrix; space $O(V)$ for the stack and the visited array.

Q9. Explain stack implementation using a linked list. [10 marks · D24]

In a **linked stack** each element is a dynamically allocated node holding the data and a pointer to the node below it; a single external pointer `top` addresses the most recently pushed node, and the bottom node's pointer is NULL. Both push and pop act at the **head** of the list only, which is why both are $O(1)$ with no traversal.

```

struct node { int data; struct node *next; };
struct node *top = NULL;

void push(int x)
{
    struct node *p = (struct node *) malloc(sizeof(struct node));
    if (p == NULL) { printf("Heap exhausted"); return; } /* no fixed overflow */
    p->data = x;
    p->next = top; /* new node points to old top */
    top = p; /* top moves to the new node */
}

int pop(void)
{
    struct node *p; int x;
    if (top == NULL) { printf("Stack Underflow"); return -1; }
    p = top;
    x = p->data;
    top = top->next; /* top moves down */
    free(p);
    return x;
}

int peek(void)
{
    if (top == NULL) { printf("Stack empty"); return -1; }
    return top->data;
}

int isEmpty(void) { return (top == NULL); }

```

Example: push 10 → top → 10 → NULL; push 20 → top → 20 → 10 → NULL; push 30 → top → 30 → 20 → 10 → NULL; pop returns 30 and top → 20 → 10 → NULL.

Point	Array stack	Linked stack
Size	Fixed at compile time (MAX)	Grows and shrinks at run time
Overflow	top == MAX-1	Only when malloc fails
Underflow	top == -1	top == NULL
Extra space	None per element	One pointer per element
Access cost	O(1), contiguous and cache-friendly	O(1), but pointer chasing
Memory use	May waste unused cells	Exactly what is needed

The key points: there is **no fixed overflow condition** for a linked stack, the underflow test is `top == NULL`, and the trade-off for the flexible size is one extra pointer per element plus the cost of `malloc/free`. Both operations are O(1) in time and the stack of n elements occupies O(n) space.

Q10. Explain the operations that are performed on Stacks. Write an algorithm to push an element to the stack. [10 marks · D22]

A **stack** is a linear list in which insertion and deletion both take place at one end, the **top**, giving Last-In-First-Out order.

Operations:

Operation	Effect	Condition	Cost
<code>createStack()</code>	Set <code>top = -1</code>	—	$O(1)$
<code>push(x)</code>	Insert <code>x</code> at the top	Overflow if <code>top == MAX-1</code>	$O(1)$
<code>pop()</code>	Remove and return the top element	Underflow if <code>top == -1</code>	$O(1)$
<code>peek() / top()</code>	Return the top element without removing it	Underflow	$O(1)$
<code>isEmpty()</code>	True when <code>top == -1</code>	—	$O(1)$
<code>isFull()</code>	True when <code>top == MAX-1</code>	—	$O(1)$

Assumptions: array `s[MAX]` of integers; `top` is global and initialised to `-1`, so that `top` always holds the index of the current top element and `-1` denotes an empty stack.

```
#define MAX 50
int s[MAX];
int top = -1;

void push(int x)
{
    if (top == MAX - 1)          /* overflow test first */
    {
        printf("Stack Overflow: cannot push %d", x);
        return;
    }
    top = top + 1;               /* make room */
    s[top] = x;                 /* store value */
}
```

Example with `MAX = 4`: `push 10` → `top = 0`, `s = [10]`; `push 20` → `top = 1`, `s = [10, 20]`; `push 30` → `top = 2`; `push 40` → `top = 3`, stack full; `push 50` → overflow reported and the stack is unchanged.

Time complexity $O(1)$; space complexity $O(1)$ auxiliary, $O(\text{MAX})$ for the stack itself. **Applications of stacks:** expression conversion and evaluation, function call and recursion management, undo mechanisms, backtracking, and the explicit stack in non-recursive tree traversal and DFS.

Q11. What is stack? Write algorithm to check whether stack is empty or not. [5 marks · J26]

A stack is a linear LIFO data structure in which both insertion (push) and deletion (pop) occur at one end called the top. With an array implementation `top` is initialised to `-1`, and an empty stack is precisely the state `top == -1`; with a linked implementation it is `top == NULL`.

```
#define MAX 50
int s[MAX];
int top = -1;

int isEmpty(void)                /* array implementation */
{
    if (top == -1)
        return 1;               /* stack is empty */
    else
        return 0;               /* stack is not empty */
}
```

Equivalently `return (top == -1);`. For the linked version:

```
int isEmpty(void) { return (top == NULL); }
```

Example: after `createStack()` `top = -1` so `isEmpty()` returns 1; after `push(10)` `top = 0` so it returns 0; after `pop()` `top` is back to `-1` and it returns 1 again. The test is used by `pop` and `peek` to detect **underflow** before touching the array.

Time complexity $O(1)$; space complexity $O(1)$.

Q12. Explain the process of implementing two stacks in a single dimensional array.

[10 marks · D23]

Two stacks can share one array of size `MAX` so that space unused by one is available to the other. The two stacks **grow towards each other from the two ends**:

- Stack 1 starts at index 0 and grows **upwards**: `top1` is initialised to `-1` and incremented on push.
- Stack 2 starts at index `MAX-1` and grows **downwards**: `top2` is initialised to `MAX` and decremented on push.
- The shared space is exhausted — and only then — when the two tops become adjacent: the overflow condition is `top1 + 1 == top2`.
- Stack 1 is empty when `top1 == -1`; stack 2 is empty when `top2 == MAX`.

```
#define MAX 10
int a[MAX];
int top1 = -1, top2 = MAX;

void push1(int x)
{
    if (top1 + 1 == top2) { printf("Stack Overflow"); return; }
    a[++top1] = x;
}

void push2(int x)
{
    if (top1 + 1 == top2) { printf("Stack Overflow"); return; }
    a[--top2] = x;
}

int pop1(void)
{
    if (top1 == -1) { printf("Stack 1 Underflow"); return -1; }
    return a[top1--];
}

int pop2(void)
{
    if (top2 == MAX) { printf("Stack 2 Underflow"); return -1; }
    return a[top2++];
}
```

Layout for `MAX = 10` after pushing A, B, C on stack 1 and X, Y on stack 2:

index :	0	1	2	3	4	5	6	7	8	9
data :	A	B	C	-	-	-	-	-	Y	X
			^						^	
			top1				top2			
	stack 1 grows -->					<-- grows stack 2				

Advantage. Either stack may occupy up to `MAX-1` cells if the other is empty, so overflow is declared only when the array is genuinely full. Dividing the array into two fixed halves of `MAX/2`

instead would declare overflow on one stack while the other half stood empty, wasting up to half the memory. All four operations remain $O(1)$ in time and the scheme uses $O(\text{MAX})$ space with only two index variables. The technique generalises poorly to three or more stacks, which need either fixed partitions or periodic shifting.

Q13. Write an algorithm to reverse a string. [10 marks · D21]

Assumptions: the string is a character array terminated by the null character `'\0'`; it is reversed **in place** by the two-pointer swap method, and the terminator stays at the end.

```
int stringLength(char s[])
{
    int n = 0;
    while (s[n] != '\0') n++;          /* count up to the terminator */
    return n;
}

void reverseString(char s[])
{
    int i, j;
    char temp;

    i = 0;
    j = stringLength(s) - 1;           /* last character, not '\0' */

    while (i < j)
    {
        temp = s[i];                   /* swap the two ends */
        s[i] = s[j];
        s[j] = temp;
        i = i + 1;                     /* move inward */
        j = j - 1;
    }
}
```

Trace on "DATA" ($n = 4, i = 0, j = 3$):

Step	i	j	String
start	0	3	D A T A
swap s[0], s[3]	1	2	A A T D
swap s[1], s[2]	2	1	A T A D
$i < j$ false	—	—	stop

Result "ATAD". For an odd-length string the middle character is left untouched because the loop exits when $i == j$.

An alternative version copies into a second array back to front, or pushes every character onto a stack and pops them off, both of which cost $O(n)$ extra space; the two-pointer method is preferred.

Time complexity $O(n)$ — $n/2$ swaps plus n steps to find the length. Space complexity $O(1)$, one temporary character.

Q14. Write an algorithm that accepts two strings, S1 and S2, as input and then find whether S1 is a substring of S2. [10 marks · J22]

Assumptions: both strings are null-terminated character arrays; the comparison is case-sensitive; the function returns the index in S2 at which the first occurrence of S1 begins, or -1 if S1 does not occur; the empty S1 is taken to occur at position 0.

```
int length(char s[])
{
    int n = 0;
    while (s[n] != '\0') n++;
    return n;
}

int isSubstring(char s1[], char s2[])          /* is s1 inside s2 ? */
{
    int m, n, i, j;

    m = length(s1);                          /* pattern length */
    n = length(s2);                          /* text length */

    if (m == 0) return 0;
    if (m > n) return -1;                    /* pattern longer than text */

    for (i = 0; i <= n - m; i++)             /* every possible start position */
    {
        j = 0;
        while (j < m && s2[i + j] == s1[j]) /* match character by character */
            j = j + 1;

        if (j == m)                          /* all m characters matched */
            return i;                        /* found at position i */
    }
    return -1;                               /* not a substring */
}
```

Trace with S1 = "DAT", S2 = "ADATA" (m = 3, n = 5, i runs 0..2): - i = 0: s2[0] = 'A' ≠ 'D' → j = 0, fail. - i = 1: s2[1] = 'D' = 'D', s2[2] = 'A' = 'A', s2[3] = 'T' = 'T' → j = 3 = m → return 1.

So "DAT" is a substring of "ADATA" starting at index 1. With S1 = "TAD" every start position fails and -1 is returned.

Time complexity: the outer loop runs $n-m+1$ times and the inner up to m times, so the worst case is $O(m \cdot n)$ — this is the naive or brute-force string matcher. Best case $O(n)$ when the first character rarely matches. Space complexity $O(1)$. Faster algorithms exist — KMP in $O(m+n)$ using a prefix table, and Boyer-Moore, both buying speed with $O(m)$ extra space.

Q15. What is meant by Big-O notation? Explain with an example. [10 marks · J22]

Answered in full under **Complexity and asymptotic notation, Q6.**

Q16. What is meant by worst case time complexity and best case time complexity? Explain with an example. [10 marks · D23]

Answered in full under **Complexity and asymptotic notation, Q8.**

Q17. Explain the process of calculation of storage complexity with an example. [10 marks · J24]

Answered in full under **Complexity and asymptotic notation, Q5.**

Q18. What is a Dequeue? How does it differ from a Queue? [10 marks · J22]

Answered in full under **Queues, Q10.**

Q19. Explain the process of implementing two queues in an array. [10 marks · J24]

Answered in full under **Queues, Q11.**

Q20. Write algorithm for deleting element from a circular queue. [5 marks · J26]

Answered in full under **Queues, Q9.**

Q21. What is Queue? Write algorithm for addition of an element to the Queue. [5 marks · J26]

Answered in full under **Queues, Q6.**

Q22. What are singly linked lists? Write an algorithm to count the number of items stored in a single linked list. [10 marks · J25]

Answered in full under **Linked lists, Q4.**

Q23. Write algorithm for deleting element from a doubly linked list. [5 marks · J26]

Answered in full under **Linked lists, Q7.**

Q24. Explain what is collision in the context of 'hashing'. What methods can be employed to address collisions? [10 marks · J25]

Answered in full under **Hashing, Q3.**

Q25. What is sequential file organization? Explain its advantages. [5 marks · J26]

Answered in full under **File organisation, Q3.**

Q26. What is need of file organisation? What are key drivers for file organization? Explain indexed sequential file organization. [10 marks · D25]

Answered in full under **File organisation, Q1.**

Q27. What is Direct File Organization (DFO)? Explain the use of hashing and hash function in DFO with the help of an example. [10 marks · J26]

Answered in full under **File organisation, Q2.**

A **spanning tree** T of a connected undirected graph $G = (V, E)$ is a subgraph of G that (i) contains every vertex of V , (ii) is connected, and (iii) contains no cycle. It therefore has exactly $|V| - 1$ edges, and adding any further edge of G creates exactly one cycle while removing any edge of T disconnects it. A disconnected graph has no spanning tree, only a spanning forest. A graph usually has many spanning trees — the complete graph on n vertices has $n^{(n-2)}$ of them by Cayley's formula.

A **minimum cost spanning tree (MCST)** is a spanning tree whose sum of edge weights is minimum. Kruskal's and Prim's greedy algorithms both construct one, in $O(E \log E)$ and $O(E \log V)$ respectively.

Example: for the graph on A, B, C, D with $AB = 1, BC = 2, CD = 3, AC = 4, BD = 5$, the edge set $\{AB, BC, CD\}$ is a spanning tree of weight 6 and is minimal; $\{AB, AC, CD\}$ is also a spanning tree but has weight 8.

Applications of spanning trees: 1. Minimum-cost network design — laying telephone, electrical, cable TV, water or road networks connecting all sites with least total length. 2. Printed-circuit and VLSI wire routing, to minimise total wire length. 3. Broadcast and multicast routing in computer networks; the IEEE 802.1D **spanning tree protocol** removes loops from a switched LAN. 4. Cluster analysis — deleting the $k-1$ most expensive MCST edges yields k clusters; also used in image segmentation. 5. Approximation algorithms, e.g. the 2-approximation for the metric travelling salesman problem built from an MCST. 6. Civil-engineering and pipeline network planning, and handwriting or feature recognition. 7. As a building block in network reliability analysis and in maze generation.